A Structural Approach to Interpretable Deep Learning

Technical Supplement

Jakob Zimmermann

Technische Universität Berlin

Contents

List of Figures ii

List of Tables iii

S.1 Extended Game-Theoretic Derivations 1
S.1.1 Notation and Conventions . . . 1
S.1.2 Stopping Game . . . 2
S.1.3 Routing Game: Log-Space Formulation . . . 13
S.1.4 Analytical Modes . . . 24
S.1.5 Softplus Variant of the Stopping Game . . . 26

S.2 Extended Attention-Routing Derivations 30
S.2.1 State space . . . 30
S.2.2 Legendre–Fenchel transform of the KL divergence . . . 31
S.2.3 Value-routing equilibrium . . . 32
S.2.4 $\alpha\beta$-***L**ayer-wise **R**elevance **P**ropagation* (LRP) recovery for the attention block . . . 33
S.2.5 Risk-Averse Attention via ADF . . . 35

S.3 Extended Trajectory Derivations 36
S.3.1 Layered Markov Process Foundations . . . 36
S.3.2 Hellinger Distance from Bhattacharyya Coefficients . . . 38
S.3.3 Hellinger Identities and Proofs . . . 39
S.3.4 Proof of the Per-Pixel Decomposition . . . 41
S.3.5 Conditioned-Survival Trajectory Laws . . . 42
S.3.6 Randomization Bounds . . . 45

S.4 Extended Experimental Results and Protocol 47
S.4.1 Full Evaluation Results for the Split Methods . . . 47
S.4.2 Stabilization Ablations for the Split Methods . . . 54
S.4.3 Implementation and Evaluation Details . . . 58
S.4.4 Additional Results . . . 61
S.4.5 Numerical Diagnostic of the Transformer DC Construction . . . 76
S.4.6 Formal Verification Details . . . 77

List of References 79

List of Figures

S.1.1 Stopping Game: trajectories and local stopping decisions 6
S.1.2 Routing Game: a local routing subgame 17

S.4.1 Additional VGG-16 saliency map comparisons 48
S.4.2 ViT-B/16 qualitative comparison on six ImageNet-S examples 76

List of Tables

S.1.1 Notation for games and trajectories . . . 2
S.1.2 Modes: default value, effect and cost per image . . . 24

S.4.1 Comprehensive interpretability evaluation: VGG-16 . . . 49
S.4.2 Comprehensive interpretability evaluation: ResNet-18 . . . 50
S.4.3 Randomization sanity check for VGG-16 split methods . . . 52
S.4.4 Mean attribution and model-splitting times . . . 53
S.4.5 Alpha ablation for SplitGrad and SplitCAM . . . 55
S.4.6 Scale versus shift stabilization for SplitLRP . . . 56
S.4.7 Alpha ablation without forward stabilization . . . 56
S.4.8 Alpha ablation with shift normalization . . . 57
S.4.9 Per-method validation grids . . . 60
S.4.10 ViT-B/16: validation-selected configurations . . . 62
S.4.11 VGG-16: validation-selected configurations . . . 63
S.4.12 ResNet-50: validation-selected configurations . . . 64
S.4.13 ViT-B/16: full ImageNet-S results with confidence intervals . . . 64
S.4.14 VGG-16: full ImageNet-S results with confidence intervals . . . 65
S.4.15 ResNet-50: full ImageNet-S results with confidence intervals . . . 66
S.4.16 ViT-B/16 on the Pascal VOC 2012 test pool . . . 68
S.4.17 ViT-B/16: one-factor ablations on ImageNet-S . . . 69
S.4.18 VGG-16: one-factor ablations on ImageNet-S . . . 71
S.4.19 ResNet-50: one-factor ablations on ImageNet-S . . . 72
S.4.20 Pointing game: Quantus tie rule and matched granularity . . . 74
S.4.21 ViT-B/16 Routing Game attention rule ablation . . . 75

S.1 Extended Game-Theoretic Derivations

S.1.1 Notation and Conventions

Throughout, $p, q, r \in \{+, -\}$ denote player or turn labels of the two players P^+ and P^-, and p' denotes the opponent of p ($+' = -$, $-' = +$); Table S.1.1 lists all symbols, grouped as network, games, Routing Game and attention, modes and smooth activations, and Hellinger distance; each has a single meaning. We write $\delta_{p,q} = 1$ if $p = q$ and 0 otherwise, the sign operator $\xi_{p,q} = 2\delta_{p,q} - 1$ ($+1$ if $p = q$, -1 if $p \neq q$), and the XOR operator $p \oplus q = +$ if $p = q$ and $-$ if $p \neq q$; it is commutative and associative, with $p \oplus p = +$, $p \oplus + = p$, $p \oplus - = p'$, and $p' \oplus q = (p \oplus q)'$. **Boundary conventions.** We fix the selected backward value $\mathrm{ReLU}'(0) \coloneqq 0$ (an algorithmic convention, since ReLU has no classical derivative at zero); equivalently, the indicator $\mathbf{1}[z > 0]$ used throughout returns 0 at $z = 0$. For a finite tuple $(x_1, \ldots, x_m)$, $\arg\max_k x_k \coloneqq \min\{k : x_k = \max_l x_l\}$, the smallest index attaining the maximum. We reserve the word *trajectory* for sequences of game states $\zeta = (\zeta_0, \zeta_1, \ldots)$ and use *path* for sequences of neuron indices.

Tab. S.1.1: Notation. Each symbol has one meaning throughout the game and trajectory chapters, except the local attention symbols $Q_t, K_k, e_{tk}, s_{tk}, m, s, c, T_n$ and the token index t, defined in Section 5.5.

Symbol	Meaning	Symbol	Meaning
l, L	layers; input 0 ($a^{(0)} = x$), output L	$W_{ij}^{(l)}$, $b_j^{(l)}$	weight from i (layer $l-1$) to j (layer l); bias
$z_j^{(l)}$, $a_j^{(l)}$	$\sum_i a_i^{(l-1)} W_{ij}^{(l)} + b_j^{(l)}$; $\mathrm{ReLU}(z_j^{(l)})$	$f(x) = a_u^{(L)}$	scalar activation output
$[t]^{\pm}$, $x_k^{\pm}$	$\max\{\pm t, 0\}$, so $t = [t]^+ - [t]^-$; $x_k^{\pm} = [x_k]^{\pm}$	$W^{(l,\pm)}$, $b^{(l,\pm)}$	$[W^{(l)}]^{\pm}$, $[b^{(l)}]^{\pm}$ (sign split)
$z_j^{(l,\pm)}$, $a_{\mathrm{stop},j}^{(l,\pm)}$	sign split of z; Stopping Decomposition	$\nabla^2 f$	Hessian
p, q, r; p'	players / turns in $\{+,-\}$; opponent	$\kappa \in \{+,-\}$	stream sign of a linear routing state
$s_{j,q}^{(l,\mathrm{act})}$, $s_{k,q}^{(0)}$	activation state (q in turn); input terminal	$s_{j,q,\kappa}^{(l,\mathrm{lin})}$	RG linear routing state
$\dagger$	cemetery (absorbing) state	$\perp_{\mathrm{lin}}$	outside action (log-payoff $\log \varepsilon$)
$r_p(s,a)$	payoff of player p	π, $\pi^\star$	policy; equilibrium (SPNE) policy
$\tilde{V}_p(s)$, $\tilde{Q}_p(s,a)$	SG player-specific value, action value	$V_p(s)$	game value (player p minus p')
$Q_r(s,i)$	RG log-gain of routing to predecessor i	$\Sigma_j^{(l,\kappa)}$	$\sum_i [a_i^{(l-1)} W_{ij}^{(l)}]^\kappa + [b_j^{(l)}]^\kappa$
$\gamma_j^{(l)}$	SG discount $\sum_i \lvert W_{ij}^{(l)} \rvert$	γ_{O}	Oracle discount (2 add./branch, 1 max-pool)
$\gamma(s)$	per-state discount: SG $\gamma_j^{(l)}$ or γ_{O}; RG $\in \{2\alpha, 2\beta, 1\}$	ζ	trajectory $(\zeta_0, \zeta_1, \dots)$
Π_x	equilibrium trajectory law at input x	$d_t(\zeta)$	cumulative trajectory discount
$\Gamma_x(s)$	discounted occupation measure of state s	$\mathscr{H}$, $\mathscr{H}_\theta$	Shannon entropy; binary form
τ	RG temperature (entropy weight)	α, β	$\alpha\beta$-LRP weights, $\alpha - \beta = 1$
ε	LRP stabilizer	$R_i^{(l)}$	LRP relevance
A	attention matrix (held fixed)	v, W_v	Value vectors, projection (sup. "val")
n_h	number of attention heads	N	equivariant-averaging samples
λ	activation-side risk aversion ($\lambda \le 0$)	λ_{sm}	attention-logit risk aversion
μ, σ, σ^2	ADF mean, std.; input variance	λ_{ent}	fan-in entropy-bias mode
sp_θ, sp'_θ	Softplus $\theta \log(1+e^{z/\theta})$; its gate	Φ, ϕ	normal CDF, density
θ; $\tilde{\Phi}$	Softplus temperature; soft gate $\Phi(\mu/\sigma + \lambda)$	$\mathscr{R}_\Phi$	probit regularizer $\phi(\Phi^{-1}(\pi))$
H, $\mathrm{H}_{\mathrm{surv}}$	Hellinger distance; survival-conditioned	BC	Bhattacharyya coefficient
$T_M^{(l)}$	backward kernel of model M	$m_M^{(l)}(s)$	marginal occupation probability
$g^{(l)}(s)$	overlap mass (backward pass)	$c^{(l)}(s)$	continuation mass (forward pass)
$\mathrm{H}^2(s)$	per-terminal-state disagreement	$\psi_M^{(l)}(s)$	survival probability

S.1.2 Stopping Game

This section realizes the activations of the Stopping Decomposition (S.1.4) as player-specific game values of a backward two-player Stopping Game on an extended network graph and demonstrates that gradients of the original network can be interpreted as integrals over game trajectories.

S.1.2.1 Stopping Decomposition

Throughout we consider a feedforward ReLU network of L layers with input layer 0, $a^{(0)} = x$, pre-activations $z_j^{(l)} = \sum_i a_i^{(l-1)} W_{ij}^{(l)} + b_j^{(l)}$, where $W_{ij}^{(l)}$ connects neuron i in layer $l-1$ to neuron j in layer l, activations $a_j^{(l)} = \mathrm{ReLU}(z_j^{(l)})$, and a *single scalar activation output* $f(x) = a_u^{(L)}$. An ungated affine classifier head is treated separately in Section 4.5.

Canonical sign split. We use the element-wise positive/negative decomposition of the original network's weights and biases,

$$W^{(l,+)} := [W^{(l)}]^+, \qquad W^{(l,-)} := [W^{(l)}]^-, \qquad b^{(l,+)} := [b^{(l)}]^+, \qquad b^{(l,-)} := [b^{(l)}]^-, \tag{S.1.1}$$

which maps non-negative activation pairs $(a_i^{(l-1,+)}, a_i^{(l-1,-)})$ to the pair of pre-activations at neuron j,

$$z_j^{(l,+)} := b_j^{(l,+)} + \sum_i W_{ij}^{(l,+)} a_i^{(l-1,+)} + \sum_i W_{ij}^{(l,-)} a_i^{(l-1,-)}, \tag{S.1.2}$$

$$z_j^{(l,-)} := b_j^{(l,-)} + \sum_i W_{ij}^{(l,+)} a_i^{(l-1,-)} + \sum_i W_{ij}^{(l,-)} a_i^{(l-1,+)}. \tag{S.1.3}$$

When $a_i^{(l-1,+)} - a_i^{(l-1,-)} = a_i^{(l-1)}$ for all i, this yields the identity $z_j^{(l,+)} - z_j^{(l,-)} = b_j^{(l)} + \sum_i (W_{ij}^{(l,+)} - W_{ij}^{(l,-)})(a_i^{(l-1,+)} - a_i^{(l-1,-)}) = z_j^{(l)}$, so the layer-$l$ pre-activation pair already encodes the original network's pre-activation through its difference.

The **Stopping decomposition function** $f_{\text{stop}} \colon \mathbb{R}^2_{\geq 0} \to \mathbb{R}^2_{\geq 0}$ is

$$f_{\text{stop}}(z^+, z^-) \;:=\; \big(\mathbf{1}[z^+ - z^- > 0]\, z^+,\; \mathbf{1}[z^+ - z^- > 0]\, z^-\big). \tag{S.1.4}$$

Applied at every neuron and propagated by (S.1.2)–(S.1.3), it defines the **Stopping Decomposition** activations $\big(a_{\text{stop},j}^{(l,+)}, a_{\text{stop},j}^{(l,-)}\big) = f_{\text{stop}}\big(z_j^{(l,+)}, z_j^{(l,-)}\big)$, with the input-layer initialization

$$a_{\text{stop},k}^{(0,\pm)} = x_k^{\pm}. \tag{S.1.5}$$

Theorem S.1.1 (Layer-wise Recovery of the Original Activations, `Lean_55`). *For every layer l and neuron j,*

$$a_{\text{stop},j}^{(l,+)} - a_{\text{stop},j}^{(l,-)} = a_j^{(l)} = \mathit{ReLU}\big(z_j^{(l)}\big). \tag{S.1.6}$$

Proof. Induction on l. At $l = 0$, $x_k^+ - x_k^- = x_k = a_k^{(0)}$ by (S.1.5). At layer $l \geq 1$ the identity above gives $z_j^{(l,+)} - z_j^{(l,-)} = z_j^{(l)}$, hence $a_{\text{stop},j}^{(l,+)} - a_{\text{stop},j}^{(l,-)} = \mathbf{1}\{z_j^{(l)} > 0\}\big(z_j^{(l,+)} - z_j^{(l,-)}\big) = \mathrm{ReLU}(z_j^{(l)}) = a_j^{(l)}$. □

ReLU networks compute continuous piecewise linear functions (Mukherjee et al., 2018), which admit a ***Difference of Convex*** (DC) decomposition (Zimmermann and Loho, 2026); it uses the same sign split (S.1.2)–(S.1.3) with the pairwise activation $(\max\{z^+, z^-\}, z^-)$ in place of f_{stop} and yields components that are convex in x. The Stopping Decomposition instead gates both streams by the same indicator, so its components are not convex, but on each linear region they are non-negative affine, hence monotone, in (x^+, x^-).

Parity path decomposition of the split streams. We can decompose the Stopping Decomposition activations into small contributions coming from individual paths through the network: each non-negative decomposed activation at neuron j is the sum, over all backward paths from j to the input, of the absolute weight product along the path times either the positive part $x_{P_0}^+$ or the negative part $x_{P_0}^-$ of the input pixel where the path terminates, with the choice between $x_{P_0}^+$ and $x_{P_0}^-$ determined by the parity of the number of negative weights traversed.

Lemma S.1.2 (Path view on the Stopping Decomposition, Lean_56). *Fix a feedforward ReLU network with weights $W^{(l)}$ (biases ignored; see remark at end of proof). For neuron j in layer $l \geq 1$, call a* backward path *from j to the input layer a list of neuron indices*

$$P = (P_l, P_{l-1}, \ldots, P_0), \qquad P_l = j, \tag{S.1.7}$$

with P_t indexing a neuron in layer t. The edges of P use the weights $W^{(t)}_{P_{t-1},P_t}$ for $t = 1, \ldots, l$. Define

$$w(P) \coloneqq \prod_{t=1}^{l} W^{(t)}_{P_{t-1},P_t}, \qquad |w|(P) \coloneqq \prod_{t=1}^{l} |W^{(t)}_{P_{t-1},P_t}|, \tag{S.1.8}$$

the parity sign

$$\operatorname{sgn}(P) \in \{+,-\}, \qquad \operatorname{sgn}(P) = + \iff \#\big\{t : W^{(t)}_{P_{t-1},P_t} < 0\big\} \text{ is even}, \tag{S.1.9}$$

and the gate factor *(1 iff every gate on P at intermediate layers $1, \ldots, l-1$ is open)*

$$G_{<l}(P) \coloneqq \prod_{t=1}^{l-1} \mathbf{1}\big[z^{(t)}_{P_t} > 0\big]. \tag{S.1.10}$$

Then the decomposed pre-activations (S.1.2)–(S.1.3) admit the path-sum representation

$$z_j^{(l,+)} = \sum_{P:\, P_l = j} G_{<l}(P)\, |w|(P)\, x_{P_0}^{\operatorname{sgn}(P)}, \tag{S.1.11}$$

$$z_j^{(l,-)} = \sum_{P:\, P_l = j} G_{<l}(P)\, |w|(P)\, x_{P_0}^{\operatorname{sgn}(P)'}. \tag{S.1.12}$$

Proof. We prove (S.1.11)–(S.1.12) by induction on l.

Base case ($l = 1$): By the input convention (S.1.5), $a^{(0,+)}_{\text{stop},k} = x_k^+$ and $a^{(0,-)}_{\text{stop},k} = x_k^-$. Using (S.1.2) with biases omitted,

$$z_j^{(1,+)} = \sum_k \big(W_{kj}^{(1,+)} x_k^+ + W_{kj}^{(1,-)} x_k^-\big). \tag{S.1.13}$$

Backward paths with $P_1 = j$ and $P_0 = k$ are in bijection with predecessors k; each has $|w|(P) = |W_{kj}^{(1)}|$, $\operatorname{sgn}(P) = +$ iff $W_{kj}^{(1)} > 0$, and $G_{<1}(P) = 1$ (empty product). Splitting by the sign of $W_{kj}^{(1)}$:

- $W_{kj}^{(1)} > 0$: $W_{kj}^{(1,+)} = |W_{kj}^{(1)}|$ contributes $|W_{kj}^{(1)}| x_k^+$ with $\operatorname{sgn}(P) = +$, matching $|w|(P)\, x_{P_0}^{\operatorname{sgn}(P)}$.
- $W_{kj}^{(1)} < 0$: $W_{kj}^{(1,-)} = |W_{kj}^{(1)}|$ contributes $|W_{kj}^{(1)}| x_k^-$ with $\operatorname{sgn}(P) = -$, again matching.

The argument for $z_j^{(1,-)}$ is identical with the parity swapped, establishing (S.1.12).

Inductive step ($l \geq 2$): The decomposition recursion (S.1.2) takes *Stopping Decomposition* activations from layer $l-1$ as inputs,

$$z_j^{(l,+)} = \sum_i \big(W_{ij}^{(l,+)}\, a^{(l-1,+)}_{\text{stop},i} + W_{ij}^{(l,-)}\, a^{(l-1,-)}_{\text{stop},i}\big), \tag{S.1.14}$$

and $a^{(l-1,\pm)}_{\text{stop},i} = \mathbf{1}[z_i^{(l-1)} > 0]\, z_i^{(l-1,\pm)}$. Hence, at each predecessor i, the gate indicator $\mathbf{1}[z_i^{(l-1)} > 0]$ multiplies the inductive representation of $z_i^{(l-1,\pm)}$, upgrading $G_{<l-1}(P')$ to $G_{\le l-1}(P')$ for every sub-path P' ending at i.

Every backward path P with $P_l = j$ and $P_{l-1} = i$ is uniquely obtained by prepending j to a sub-path $P' = (P'_{l-1}, \dots, P'_0)$ with $P'_{l-1} = i$; the prepended edge uses $W^{(l)}_{ij}$, contributing $|W^{(l)}_{ij}|$ to $|w|(P)$ and either preserving or flipping parity depending on $\operatorname{sgn}(W^{(l)}_{ij})$. Crucially, the new gate factor $G_{<l}(P)$ for P equals $G_{\le l-1}(P')$: up to layer $l-1$ the gates are those along P', and there is no gate at layer l in the $G_{<l}$ product.

Case $W^{(l)}_{ij} > 0$. Then $W^{(l,+)}_{ij} = |W^{(l)}_{ij}|$ and $W^{(l,-)}_{ij} = 0$. Only $W^{(l,+)}_{ij}\, a^{(l-1,+)}_{\text{stop},i}$ contributes, and the new edge preserves parity, $\operatorname{sgn}(P) = \operatorname{sgn}(P')$. Using the induction hypothesis under the updated gate factor,

$$\begin{aligned} W^{(l,+)}_{ij}\, a^{(l-1,+)}_{\text{stop},i} &= |W^{(l)}_{ij}| \sum_{P' :\, P'_{l-1}=i} G_{\le l-1}(P')\, |w|(P')\, x_{P'_0}^{\operatorname{sgn}(P')} \\ &= \sum_{P :\, P_l=j,\, P_{l-1}=i} G_{<l}(P)\, |w|(P)\, x_{P_0}^{\operatorname{sgn}(P)}. \end{aligned} \tag{S.1.15}$$

Case $W^{(l)}_{ij} < 0$. Then $W^{(l,+)}_{ij} = 0$ and $W^{(l,-)}_{ij} = |W^{(l)}_{ij}|$. Only $W^{(l,-)}_{ij}\, a^{(l-1,-)}_{\text{stop},i}$ contributes, and the new edge flips parity, $\operatorname{sgn}(P) = \operatorname{sgn}(P')'$, so $\operatorname{sgn}(P') = \operatorname{sgn}(P)'$, and the induction hypothesis for $z_i^{(l-1,-)}$ (with $x_{P'_0}^{\operatorname{sgn}(P')'} = x_{P_0}^{\operatorname{sgn}(P)}$) gives the same path sum as in the previous case. *Case* $W^{(l)}_{ij} = 0$. Both terms vanish and no path through i is prepended.

Summing over i yields (S.1.11). The same argument with positive/negative decomposed activations swapped gives (S.1.12).

Signed recovery. Using $x_k^+ - x_k^- = x_k$ and $\operatorname{sgn}(P) \in \{+,-\}$,

$$z_j^{(l,+)} - z_j^{(l,-)} = \sum_P G_{<l}(P)\, |w|(P) \left(x_{P_0}^{\operatorname{sgn}(P)} - x_{P_0}^{\operatorname{sgn}(P)'}\right) = \sum_P G_{<l}(P)\, w(P)\, x_{P_0}, \tag{S.1.16}$$

which is the standard active-path-sum representation of $z_j^{(l)}$ (paths through an inactive intermediate neuron contribute zero).

Biases. Adjoining a bias "stub path" at each intermediate layer l'—starting from (l,j), terminating at the bias node of neuron k' at layer l', with weight product taken over the $l - l'$ edges above and an appropriate gate factor $G_{<l}$ restricted to those edges—handles the bias terms by the same parity rule; the argument above goes through verbatim after absorbing these stub contributions into the path sums. □

S.1.2.2 Game Definition

Relation to the network games of Gaubert & Vlassopoulos (2025). The Stopping Game adapts the game of Gaubert and Vlassopoulos to the Stopping Decomposition. We keep their state-space structure —per-neuron ± states, the bias as the continuation and network input as final payoff, transitions driven by the positive and negative parts of the weights, a cemetery state, and a stop-versus-continue action. Two changes are load-bearing for attribution: (i) the original formulation tracks only one game value, which equals $\pm a_j^{(l)}$; we additionally maintain the non-negative payoff per player, and the original

game value is recovered as their difference. (ii) We replace the signed terminal values $\pm x_k$ by the non-negative pair x_k^+, x_k^-, which keeps every player-specific payoff non-negative and exposes the parity decomposition of Lemma S.1.2.

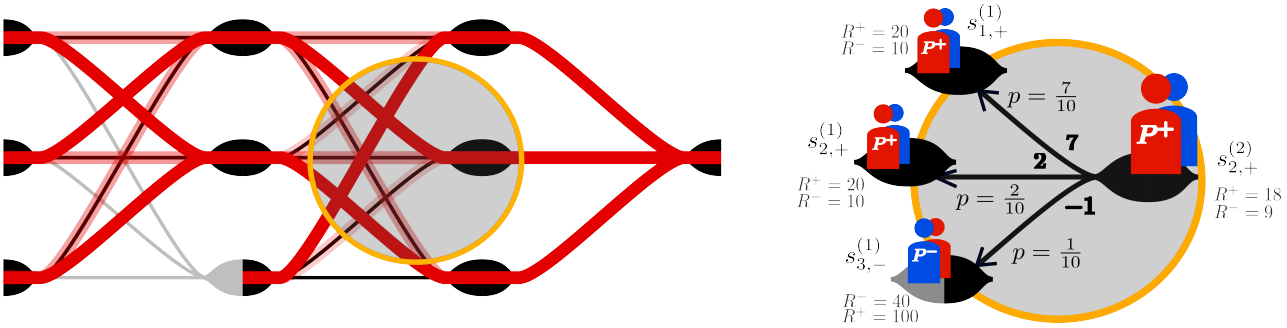


Fig. S.1.1: Stopping Game: trajectory distribution and local stopping decisions on a toy subnetwork. *Left:* trajectory distribution visualized on the network graph. *Right:* local game around $s^{(2)}_{2,+}$. Player P^+ decides whether to stop or continue. Under continuation, transitions are to $s^{(1)}_{1,+}$ with probability 7/10, to $s^{(1)}_{2,+}$ with probability 2/10 (both turn unchanged), and to $s^{(1)}_{3,-}$ with probability 1/10 (turn switches to P^-, reflecting the negative sign of the weight). The labels $R^\pm$ are the players' expected payoffs $\tilde{Q}_\pm(\cdot, \text{cont})$ (at $s^{(2)}_{2,+}$ without the discount $\gamma_2^{(2)} = 10$). At $s^{(1)}_{3,-}$, continuing yields expected payoffs R^-=40 and R^+=100, giving continuation advantage $R^- - R^+ = -60 < 0$; therefore P^- stops (so $V_-(s^{(1)}_{3,-}) = 0$), shown by the grayed-out half-neuron. Correspondingly, in the left panel the trajectories terminate at that state (the cemetery state is omitted). Since P^- stops in $s^{(1)}_{3,-}$, traversals to this state contribute zero continuation payoff to both players. Hence, at $s^{(2,\text{act})}_{2,+}$ the positive player has larger expected payoff under continuation than his opponent and chooses to continue.

Definition S.1.1 (Stopping Game State Space). The game is played on the following states:

- **Activation States:** $s^{(l,\text{act})}_{j,q}$ where layer $l \in \{1, \dots, L\}$ and player $q \in \{+,-\}$ is in turn at neuron j.
- **Terminal States:** $s^{(0)}_{k,q}$ where k is an input dimension and $q \in \{+,-\}$ is the final turn label.
- **Cemetery State:** †, an absorbing state reached upon stopping.

We adopt the convention that, when a transition kernel or recursion at layer l references the predecessor state $s^{(l-1,\text{act})}_{i,r}$, the symbol denotes the activation state at layer $l-1 \geq 1$ as defined above and, in the boundary case $l = 1$, is interpreted as the terminal state $s^{(0)}_{i,r}$.

Definition S.1.2 (Payoffs, Transitions, and Value Recursion). Fix an input x.

Payoffs. The player-specific *payoff* r_p at activation states is paid out depending on the action chosen by the active player, and paid out at terminal and cemetery states without prerequisites:

- Cemetery: $r_p(\dagger) \coloneqq 0$.
- Active state: $r_p(s^{(l,\text{act})}_{j,q}, \text{stop}) \coloneqq 0$ (stopping pays nothing) and $r_p(s^{(l,\text{act})}_{j,q}, \text{cont}) \coloneqq b^{(l,p\oplus q)}_j$ (continuation pays a positive bias to the active player q—covered by the case $p \oplus q = +$—and the absolute value of a negative bias to its opponent q'—covered by the case $p \oplus q = -$).
- Terminal: $r_p(s^{(0)}_{k,q}) \coloneqq x_k^{p\oplus q}$ (at terminal states the payoff depends on state only—the game ends here. A positive input is paid to the active player q—covered by the case $p \oplus q = +$—whereas the absolute value of a negative input is paid to its opponent q'—covered by the case $p \oplus q = -$).

Transition kernel. Stopping at state $s_{j,p}^{(l,\mathrm{act})}$ deterministically sends the game to the cemetery; continuation transports backward by one layer with transition probabilities depending on the fraction of the respective absolute network weight compared to the sum of absolute incoming weights at neuron j of layer l. In case of a positive weight, the active player p stays in turn, in case of a negative weight, the turn switches to its opponent p'.

$$\mathbb{P}\big(\dagger \,\big|\, s_{j,q}^{(l,\mathrm{act})}, \mathrm{stop}\big) \coloneqq 1, \tag{S.1.17}$$

$$\mathbb{P}\big(s_{i,r}^{(l-1,\mathrm{act})} \,\big|\, s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) \coloneqq \frac{W_{ij}^{(l,q\oplus r)}}{\gamma_j^{(l)}}, \qquad \gamma_j^{(l)} \coloneqq \sum_i \big(W_{ij}^{(l,+)} + W_{ij}^{(l,-)}\big) = \sum_i \big|W_{ij}^{(l)}\big|, \tag{S.1.18}$$

where $\gamma_j^{(l)}$ is the local structural discount that restores unit row-sum (if $\gamma_j^{(l)} = 0$, continuation leads to $\dagger$). For understanding the idea of the occupation measure, we remark that those $\gamma_j^{(l)}$ are going to weight the trajectories in the occupation measure.

Policy. A pure, i.e., deterministic policy $\pi = (\pi_+, \pi_-)$ assigns each active state $s_{j,q}^{(l,\mathrm{act})}$ a binary stop/continue decision by the player in turn,

$$\pi_q\big(s_{j,q}^{(l,\mathrm{act})}\big) \in \{0,1\}, \qquad 1 = \mathrm{cont},\ 0 = \mathrm{stop}. \tag{S.1.19}$$

Player-specific action and game values. For each policy π, the player-specific **action value** $\tilde{Q}_p^\pi(s,a)$ is the expected discounted payoff for player p starting at s with action a and following π thereafter, defined by the standard Bellman equation

$$\tilde{Q}_p^\pi(s,a) \coloneqq r_p(s,a) + \gamma(s) \textstyle\sum_{s'} \mathbb{P}\big(s' \,\big|\, s,a\big)\, \tilde{V}_p^\pi(s'), \tag{S.1.20}$$

where $\tilde{V}_p^\pi$ is the player-specific game value defined below. We use tildes to reserve V_p^π for the advantage further down, which already carries a subtracted $\tilde{V}_{p'}^\pi$. At active states (S.1.20) specializes, using (S.1.17)–(S.1.18), to

$$\tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{stop}\big) = 0 + 1 \cdot \tilde{V}_p^\pi(\dagger) = 0 \tag{S.1.21}$$

$$\tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) = b_j^{(l,p\oplus q)} + \gamma_j^{(l)} \textstyle\sum_{s'} \mathbb{P}\big(s' \,\big|\, s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big)\, \tilde{V}_p^\pi(s') \tag{S.1.22}$$

$$= b_j^{(l,p\oplus q)} + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\, \tilde{V}_p^\pi\big(s_{i,r}^{(l-1,\mathrm{act})}\big). \tag{S.1.23}$$

The **player-specific game value** $\tilde{V}_p^\pi(s)$ is the action value averaged by the policy. At terminal and cemetery states it equals the local payoff there:

$$\tilde{V}_p^\pi\big(s_{k,q}^{(0)}\big) \coloneqq x_k^{p\oplus q}, \qquad \tilde{V}_p^\pi(\dagger) \coloneqq 0. \tag{S.1.24}$$

At active states,

$$\tilde{V}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}\big) \coloneqq \pi_q\big(s_{j,q}^{(l,\mathrm{act})}\big)\, \tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) + \big(1 - \pi_q\big(s_{j,q}^{(l,\mathrm{act})}\big)\big)\, \tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{stop}\big). \tag{S.1.25}$$

Since π_q is binary, (S.1.25) selects either $\tilde{Q}_p^\pi(\cdot, \mathrm{cont})$ or $\tilde{Q}_p^\pi(\cdot, \mathrm{stop}) = 0$.

Game value. The quantity the player in turn maximizes at each active state is the advantage of player p at s,

$$V_p^{\pi}(s) \coloneqq \tilde{V}_p^{\pi}(s) - \tilde{V}_{p'}^{\pi}(s). \tag{S.1.26}$$

Figure S.1.1 shows a worked example of the induced trajectory distribution and the local stop/continue logic on a small toy network.

Theorem S.1.3 (Forward Equivalence for Stopping Game, `Lean_57`). *The game admits a Subgame Perfect Nash Equilibrium (SPNE) where both players follow the same pure strategy:*

$$\pi_q^{\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = \mathbf{1}\big[z_j^{(l)} > 0\big]. \tag{S.1.27}$$

Under this equilibrium, the player-specific game values and the game values satisfy

$$\tilde{V}_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = a_{\mathrm{stop},j}^{(l,p\oplus q)}, \qquad V_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = \xi_{p,q}\, a_j^{(l)}. \tag{S.1.28}$$

Remark S.1.1 (Uniqueness of the SPNE up to tie-breaking). It is easy to see in the following proof that the SPNE (S.1.27) is unique up to tie-breaking at states where continuing yields zero advantage $\tilde{Q}_q^{\pi^\star}(\cdot,\mathrm{cont}) - \tilde{Q}_{q'}^{\pi^\star}(\cdot,\mathrm{cont}) = z_j^{(l)} = 0$—representing neurons of pre-activation zero. The pinned Stopping-Game attribution code selects continuation at exact zero in its hard backward mask, while the Routing Game and Hellinger diagnostic use strict positivity. These choices give the same original forward activation at a tie but can give different selected backward quantities there. The theorem here uses stopping at ties. We refer to the policy (S.1.27) by **stop-at-tie-breaks** equilibrium.

Proof. We proceed by backward induction on the layer index l.

Base Case ($l = 0$): By (S.1.24), $\tilde{V}_p^{\pi^\star}(s_{k,q}^{(0)}) = x_k^{p\oplus q} = a_{\mathrm{stop},k}^{(0,p\oplus q)}$ (input convention (S.1.5)). The game value is

$$V_p^{\pi^\star}\big(s_{k,q}^{(0)}\big) = x_k^{p\oplus q} - x_k^{p'\oplus q} = \xi_{p,q}\, x_k, \tag{S.1.29}$$

using $x_k^+ - x_k^- = x_k$ and $x_k^- - x_k^+ = -x_k$.

Inductive Step: Assume the subgames starting at states up to layer $l-1$ admit the claimed SPNE and (S.1.28) holds at layer $l-1$. Consider an active state $s_{j,q}^{(l,\mathrm{act})}$. The active player q chooses $a \in \{\mathrm{stop},\mathrm{cont}\}$ to maximize the advantage $\tilde{Q}_q^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})},a) - \tilde{Q}_{q'}^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})},a)$. For $a = \mathrm{stop}$ this advantage is 0 by (S.1.21); for $a = \mathrm{cont}$, using (S.1.22),

$$\tilde{Q}_q^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})},\mathrm{cont}\big) - \tilde{Q}_{q'}^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})},\mathrm{cont}\big) = \big(b_j^{(l,q\oplus q)} - b_j^{(l,q'\oplus q)}\big) + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\, V_q^{\pi^\star}\big(s_{i,r}^{(l-1,\mathrm{act})}\big) \tag{S.1.30}$$

$$= \big(b_j^{(l,+)} - b_j^{(l,-)}\big) + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\, V_q^{\pi^\star}\big(s_{i,r}^{(l-1,\mathrm{act})}\big). \tag{S.1.31}$$

By the inductive hypothesis $V_q^{\pi^\star}(s_{i,r}^{(l-1,\mathrm{act})}) = \xi_{q,r}\, a_i^{(l-1)}$, with $\xi_{q,q} = +1$ and $\xi_{q,q'} = -1$,

$$\tilde{Q}_q^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) - \tilde{Q}_{q'}^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) = b_j^{(l)} + \sum_i \big(W_{ij}^{(l,q\oplus q)}\, \xi_{q,q} + W_{ij}^{(l,q\oplus q')}\, \xi_{q,q'}\big) a_i^{(l-1)} \tag{S.1.32}$$

$$= b_j^{(l)} + \sum_i W_{ij}^{(l)}\, a_i^{(l-1)} = z_j^{(l)}. \tag{S.1.33}$$

Thus, player q prefers *continue* in case $z_j^{(l)} > 0$ and prefers *stop* in case $z_j^{(l)} < 0$, and is indifferent at $z_j^{(l)} = 0$, where we tie-break in favor of *stop*. The rule $\pi_q^\star(s_{j,q}^{(l,\mathrm{act})}) = \mathbf{1}[z_j^{(l)} > 0]$ together with the player policies of earlier layers thus constitutes an SPNE of the subgame starting at state $s_{j,q}^{(l,\mathrm{act})}$.

To verify (S.1.28), substitute $\pi_q^\star$ into (S.1.25) and use the inductive hypothesis $\tilde{V}_p^{\pi^\star}(s_{i,r}^{(l-1,\mathrm{act})}) = a_{\mathrm{stop},i}^{(l-1,p\oplus r)}$ in (S.1.22):

$$\tilde{V}_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = \mathbf{1}\big[z_j^{(l)} > 0\big]\, \tilde{Q}_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) \tag{S.1.34}$$

$$= \mathbf{1}\big[z_j^{(l)} > 0\big] \left[b_j^{(l,p\oplus q)} + \sum_i \big(W_{ij}^{(l,+)}\, a_{\mathrm{stop},i}^{(l-1,p\oplus q)} + W_{ij}^{(l,-)}\, a_{\mathrm{stop},i}^{(l-1,p\oplus q')}\big) \right]. \tag{S.1.35}$$

If $p = q$, the bracket is $b_j^{(l,+)} + \sum_i \big(W_{ij}^{(l,+)}\, a_{\mathrm{stop},i}^{(l-1,+)} + W_{ij}^{(l,-)}\, a_{\mathrm{stop},i}^{(l-1,-)}\big) = z_j^{(l,+)}$. If $p = q'$, the bracket is $b_j^{(l,-)} + \sum_i \big(W_{ij}^{(l,+)}\, a_{\mathrm{stop},i}^{(l-1,-)} + W_{ij}^{(l,-)}\, a_{\mathrm{stop},i}^{(l-1,+)}\big) = z_j^{(l,-)}$. In both cases, $\tilde{V}_p^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}) = \mathbf{1}[z_j^{(l)} > 0]\, z_j^{(l,p\oplus q)} = a_{\mathrm{stop},j}^{(l,p\oplus q)}$. Finally,

$$V_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = a_{\mathrm{stop},j}^{(l,p\oplus q)} - a_{\mathrm{stop},j}^{(l,(p\oplus q)')} = \xi_{p,q}\, a_j^{(l)}. \quad \square \tag{S.1.36}$$

$\square$

S.1.2.3 Skip Connections and Max Pooling

Architectures such as ResNets and CNNs with pooling contain structural operators beyond single-parent affine maps. To handle these natively in the Stopping Game, we add an **Addition Oracle** for skip connections and a deterministic **Max-Pooling Decision State** for pooling, mirroring the analogous constructions in the Routing Game (Section S.1.3.1).

Definition S.1.3 (Addition Oracle). For a residual node $z = x + y$, the *addition state* $s_{z,q}^{\mathrm{add}}$ has active player $q \in \{+,-\}$ and operand states $s_{x,q}, s_{y,q}$ at the two summands. The transition kernel is the uniform Oracle

$$\mathbb{P}\big(s_{x,q} \,\big|\, s_{z,q}^{\mathrm{add}}\big) := \tfrac{1}{2}, \qquad \mathbb{P}\big(s_{y,q} \,\big|\, s_{z,q}^{\mathrm{add}}\big) := \tfrac{1}{2}, \tag{S.1.37}$$

with player q retained at the chosen operand. The immediate payoff is $r_p(s_{z,q}^{\mathrm{add}}) := 0$ for both $p \in \{+,-\}$, and the structural discount is $\gamma_{\mathrm{O}} := 2$.

Proposition S.1.4 (Forward Equivalence for Addition, `Lean_58`). *The player-specific state value at the addition state equals the sum of the operand state values,*

$$\tilde{V}_p^{\pi}\big(s_{z,q}^{\mathrm{add}}\big) \;=\; \tilde{V}_p^{\pi}(s_{x,q}) + \tilde{V}_p^{\pi}(s_{y,q}). \tag{S.1.38}$$

Consequently, if the operand states recover the Stopping-Decomposition activations—$\tilde{V}_p^{\pi^\star}(s_{x,q}) = x^{(p\oplus q)}$ *and* $\tilde{V}_p^{\pi^\star}(s_{y,q}) = y^{(p\oplus q)}$*—then* $\tilde{V}_p^{\pi^\star}(s^{\mathrm{add}}_{z,q}) = z^{(p\oplus q)}$*, preserving the forward equivalence of Theorem S.1.3 across the residual sum.*

Proof. Apply the Bellman recursion (S.1.20)–(S.1.25) at $s^{\mathrm{add}}_{z,q}$ with $r_p = 0$, the uniform Oracle kernel, and $\gamma_{\mathrm{O}} = 2$ from Definition S.1.3:

$$\tilde{V}_p^{\pi}\big(s^{\mathrm{add}}_{z,q}\big) \;=\; 0 + \gamma_{\mathrm{O}} \cdot \Big(\tfrac{1}{2}\,\tilde{V}_p^{\pi}(s_{x,q}) + \tfrac{1}{2}\,\tilde{V}_p^{\pi}(s_{y,q})\Big) \;=\; \tilde{V}_p^{\pi}(s_{x,q}) + \tilde{V}_p^{\pi}(s_{y,q}), \tag{S.1.39}$$

which is the first identity. Substituting the operand assumption and using $z^{(p\oplus q)} = x^{(p\oplus q)} + y^{(p\oplus q)}$ from the additive convention (S.1.2)–(S.1.3) of Section S.1.2.1 yields the second. □

On the uniform 1/2 split. The $(\frac{1}{2}, \frac{1}{2})$ Oracle with $\gamma_{\mathrm{O}} = 2$ is not forced by the architecture; it is a convention, the simplest choice inside a one-parameter family of conservation-preserving Oracles $(\rho, 1-\rho)$ with matching discounts $(1/\rho, 1/(1-\rho))$ attached to the branch taken (with the single state discount γ_{O} of (S.1.20) and d_t, $\rho\gamma_{\mathrm{O}} = (1-\rho)\gamma_{\mathrm{O}} = 1$ forces $\rho = \frac{1}{2}$). The uniform split is the maximum-entropy choice—the one that imposes *no prior* on which summand carries more responsibility. In the Stopping Game, Proposition S.1.4 and Theorems S.1.3 and S.1.6 hold for any such split with branch discounts, since the proofs only use that routing probability times Oracle discount equals 1 on each branch; only the trajectory law Π_x (and with it the Hellinger distance of Section 8.3) depends on the split. This invariance does not extend to the Routing Game (Section S.1.3.1), where the split changes the relevance assigned to each summand.

Max pooling is simpler, because the forward operator already selects a single predecessor.

Definition S.1.4 (Max-Pooling Decision State). For a pooled scalar output $z = \max\{x_1, \dots, x_m\}$, the *pooling state* $s^{\max}_{z,q}$ has active player $q \in \{+,-\}$ and admissible action set $\mathscr{A}(s^{\max}_{z,q}) = \{1, \dots, m\}$ indexing the operand states $s_{x_1,q}, \dots, s_{x_m,q}$. Under action k, the transition is deterministic,

$$\mathbb{P}\big(s_{x_k,q} \,\big|\, s^{\max}_{z,q}, k\big) \coloneqq 1, \tag{S.1.40}$$

with player q retained at the chosen operand. The immediate payoff is $r_p(s^{\max}_{z,q}, k) \coloneqq 0$ for both $p \in \{+,-\}$, and the structural discount is $\gamma_{\mathrm{O}} \coloneqq 1$.

Tie-breaking at the pooling state. Whenever multiple operands x_k attain the maximum, the active player is indifferent over the corresponding actions, and any pure best response over the tied indices yields the same forward value but a different equilibrium policy and hence a different occupation measure on the operand states. The *stop-at-tie-breaks equilibrium*, fixed by the boundary conventions of Section S.1.1, resolves this by the convention $k^\star \coloneqq \min\{k : x_k = \max_l x_l\}$, i.e., the active player picks the smallest index among the operands attaining the maximal payoff. All gradient and occupation-measure statements below are stated for this equilibrium.

Proposition S.1.5 (Forward Equivalence for Max Pooling, `Lean_59`). *Assume the operand states recover the Stopping-Decomposition activations,* $\tilde{V}_p^{\pi^\star}(s_{x_k,q}) = x_k^{(p\oplus q)}$ *for all* k*, with non-negative operand activations* $x_k \geq 0$*. Then at the SPNE of the subgame rooted at* $s^{\max}_{z,q}$*,*

$$\tilde{V}_p^{\pi^\star}\big(s^{\max}_{z,q}\big) \;=\; z^{(p\oplus q)}, \tag{S.1.41}$$

so the pooling state inherits the same Stopping-Decomposition identity as its winning operand and the forward equivalence of Theorem S.1.3 extends across the pooling node.

Proof. The active player q chooses the operand that maximizes its game value $V_q^{\pi^\star} = \tilde{V}_q^{\pi^\star} - \tilde{V}_{q'}^{\pi^\star}$. By the operand assumption, $V_q^{\pi^\star}(s_{x_k,q}) = \xi_{q,q}\, x_k = x_k$ for every k, so the maximizer is $k^\star \in \arg\max_k x_k$ (ties resolved by the smallest index). The Bellman recursion (S.1.20)–(S.1.25) with $r_p = 0$, the deterministic transition, and $\gamma_{\mathrm{O}} = 1$ from Definition S.1.4 gives

$$\tilde{V}_p^{\pi^\star}\big(s_{z,q}^{\max}\big) \;=\; 0 + 1 \cdot \tilde{V}_p^{\pi^\star}(s_{x_{k^\star},q}) \;=\; x_{k^\star}^{(p\oplus q)}. \tag{S.1.42}$$

Since $z = x_{k^\star}$, the right-hand side equals $z^{(p\oplus q)}$. □

S.1.2.4 Discounted Occupation Measures and Gradient Representations

To connect game trajectories with neural network gradients, we define the **discounted occupation measure**. This measure quantifies the expected influence of a neuron on the game's outcome, weighted by the discount factors accumulated along all possible game trajectories (including those created by skip connections and max-pooling nodes).

Definition S.1.5 (Game trajectory and trajectory law, `Lean_60`). Fix an input x and a root state s. A *game trajectory* rooted at s is a finite sequence of game states

$$\zeta \;=\; (\zeta_0, \zeta_1, \ldots, \zeta_T), \qquad \zeta_0 = s, \tag{S.1.43}$$

that follows the admissible transitions of the game (Definition S.1.2 together with the addition/max-pooling Oracle splits of Definitions S.1.3–S.1.4) until it terminates either at a layer-0 terminal state $\zeta_T = s_{k,r}^{(0)}$ (reaching an input dimension k) or at the cemetery state $\zeta_T = \dagger$ (reached the first time a player chooses *stop*). The set of all such trajectories is the trajectory *ground set* $\mathscr{T}_s$. Composing the transition kernel (S.1.17)–(S.1.18), the Oracle splits, and a (possibly stochastic) policy π at every activation state defines a Markov kernel on game states, which together with the deterministic start at s induces a probability distribution $\Pi_x^\pi(\cdot \mid s)$ on $\mathscr{T}_s$. Trajectories that pass through closed gates—neurons j with $\mathbf{1}[z_j^{(l)} > 0] = 0$—are admissible elements of $\mathscr{T}_s$, but under the equilibrium policy $\pi^\star$ the player at the activation state stops there, so any trajectory continuing past a closed gate carries probability zero. We write $\Pi_x(\cdot \mid s) \coloneqq \Pi_x^{\pi^\star}(\cdot \mid s)$ for the equilibrium *trajectory law* and $\Pi_x \coloneqq \Pi_x(\cdot \mid s_0)$ for the law rooted at the output state $s_0 \coloneqq s_{u,+}^{(L,\mathrm{act})}$.

Definition S.1.6 (Discounted Occupation Measure, `Lean_61`). Let $\Pi_x(\cdot \mid s)$ be the equilibrium trajectory law of Definition S.1.5. Define the cumulative discount

$$d_t(\zeta) \coloneqq \prod_{m=0}^{t-1} \gamma(\zeta_m), \qquad d_0(\zeta) = 1, \tag{S.1.44}$$

where $\gamma(\zeta_m) = \gamma_{\mathrm{O}} = 2$ for addition Oracle states, $\gamma(\zeta_m) = \gamma_{\mathrm{O}} = 1$ for max-pooling states, and $\gamma(\zeta_m) = \gamma_j^{(l)}$ for continuation at activation states $\zeta_m = s_{j,q}^{(l,\mathrm{act})}$ (per-neuron weight-magnitude discount). The *discounted occupation measure* of state v for trajectories rooted at s is

$$\Gamma_x(v \mid s) \coloneqq \mathbb{E}_{\zeta\sim\Pi_x(\cdot|s)}\left[\sum_t d_t(\zeta)\, \mathbf{1}\{\zeta_t = v\}\right], \qquad \Gamma_x(v) \coloneqq \Gamma_x(v \mid s_0). \tag{S.1.45}$$

Theorem S.1.6 (Activation–Occupation Correspondence, Lean_62). *Fix a neuron state $s = s^{(m,\mathrm{act})}_{j,q}$ and a layer $l \le m$ (for $l = 0$, $s^{(0,\mathrm{act})}_{i,\cdot}$ denotes the terminal state $s^{(0)}_{i,\cdot}$). Under the stop-at-tie-breaks equilibrium fixed by the boundary conventions of Section S.1.1 (uniquely fixing the SPNE at ReLU gates with $z = 0$ and at max-pooling states with multiple maximizers), the discounted occupation measures exactly represent, in a multi-trajectory ReLU network (e.g., a ResNet or a CNN with max pooling), the local Stopping-Decomposition and scalar activation derivatives on open cells with nonzero ReLU pre-activations and unique max-pooling winners. At ties, the same formulas represent the backward quantities computed using the selected gates and winners; they do not assert classical differentiability of the individual stopping streams, which can be discontinuous there:*

1) ***Stopping-Decomposition Gradient:***

$$\frac{\partial a^{(m,p\oplus q)}_{\mathrm{stop},j}}{\partial a^{(l,k)}_{\mathrm{stop},i}} \;=\; \Gamma_x\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \;\big|\; s\big). \tag{S.1.46}$$

2) ***Network Activation Gradient.*** *The ordinary-network gradient of $a^{(m)}_j$ with respect to the scalar activation $a^{(l)}_i$ is*

$$\frac{\partial a^{(m)}_j}{\partial a^{(l)}_i} \;=\; \xi_{q,+}\big(\Gamma_x(s^{(l,\mathrm{act})}_{i,+} \mid s) - \Gamma_x(s^{(l,\mathrm{act})}_{i,-} \mid s)\big). \tag{S.1.47}$$

Proof. We establish Part 1 by backward induction on the layer gap $m - l$ and derive Part 2 from it. Since x is fixed, we write $\Gamma(v \mid s) \coloneqq \Gamma_x(v \mid s)$.

Part 2. By Theorem S.1.3, $a^{(l)}_i = a^{(l,+)}_{\mathrm{stop},i} - a^{(l,-)}_{\mathrm{stop},i}$ and $a^{(m)}_j = a^{(m,+)}_{\mathrm{stop},j} - a^{(m,-)}_{\mathrm{stop},j}$. Differentiating $a^{(m)}_j$ along the $+$ direction (perturb $a^{(l,+)}_{\mathrm{stop},i}$, keep $a^{(l,-)}_{\mathrm{stop},i}$ fixed) and applying Part 1 at $k = +$,

$$\frac{\partial a^{(m)}_j}{\partial a^{(l)}_i} = \frac{\partial a^{(m,+)}_{\mathrm{stop},j}}{\partial a^{(l,+)}_{\mathrm{stop},i}} - \frac{\partial a^{(m,-)}_{\mathrm{stop},j}}{\partial a^{(l,+)}_{\mathrm{stop},i}} = \Gamma\big(s^{(l,\mathrm{act})}_{i,q} \;\big|\; s\big) - \Gamma\big(s^{(l,\mathrm{act})}_{i,q'} \;\big|\; s\big) = \xi_{q,+}\big(\Gamma(s^{(l,\mathrm{act})}_{i,+} \mid s) - \Gamma(s^{(l,\mathrm{act})}_{i,-} \mid s)\big), \tag{S.1.48}$$

using $q \oplus + = q$ and $q' \oplus + = q'$ in the second equality.

Part 1. The decomposition recursion of Theorem S.1.3 writes the upper-layer Stopping-Decomposition activation as

$$a^{(m,p\oplus q)}_{\mathrm{stop},j} = \mathbf{1}\big[z^{(m)}_j > 0\big]\left[b^{(m,p\oplus q)}_j + \sum_h \Big(W^{(m,+)}_{hj}\, a^{(m-1,p\oplus q)}_{\mathrm{stop},h} + W^{(m,-)}_{hj}\, a^{(m-1,p\oplus q')}_{\mathrm{stop},h}\Big)\right]. \tag{S.1.49}$$

Base case ($m = l$). If $s = s^{(l,\mathrm{act})}_{i,r}$ already lies at the target neuron and decomposed activation, then

$$\frac{\partial a^{(l,p\oplus r)}_{\mathrm{stop},i}}{\partial a^{(l,k)}_{\mathrm{stop},i}} = \delta_{p\oplus r,\,k} = \mathbf{1}\{r = p \oplus k\} = \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \;\big|\; s^{(l,\mathrm{act})}_{i,r}\big). \tag{S.1.50}$$

Inductive step (linear routing, $m > l$). Differentiating (S.1.49) with respect to $a^{(l,k)}_{\mathrm{stop},i}$ (the bias $b^{(m,p\oplus q)}_j$ does not depend on lower-layer activations) and substituting the inductive hypothesis at layer $m-1$,

$$\frac{\partial a^{(m,p\oplus q)}_{\mathrm{stop},j}}{\partial a^{(l,k)}_{\mathrm{stop},i}} = \mathbf{1}\big[z^{(m)}_j > 0\big] \sum_h \Big(W^{(m,+)}_{hj}\, \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s^{(m-1,\mathrm{act})}_{h,q}\big) + W^{(m,-)}_{hj}\, \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s^{(m-1,\mathrm{act})}_{h,q'}\big)\Big). \tag{S.1.51}$$

On the game side, the one-step occupation recursion at $s = s^{(m,\mathrm{act})}_{j,q}$ uses transition probabilities $W^{(m,q\oplus r)}_{hj}/\gamma^{(m)}_j$ together with the matching cumulative-discount factor $\gamma^{(m)}_j$ and the equilibrium gate $\mathbf{1}[z^{(m)}_j > 0]$:

$$\Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s\big) = \mathbf{1}\big[z^{(m)}_j > 0\big] \sum_{h,r} W^{(m,q\oplus r)}_{hj}\, \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s^{(m-1,\mathrm{act})}_{h,r}\big). \tag{S.1.52}$$

The two right-hand sides agree, completing the inductive step at linear routing layers.

Addition-node step. For an addition node $z = x + y$, the additivity of the Stopping Decomposition from Section S.1.2.1 gives $a^{(p\oplus q)}_{\mathrm{stop},z} = a^{(p\oplus q)}_{\mathrm{stop},x} + a^{(p\oplus q)}_{\mathrm{stop},y}$, so by the inductive hypothesis,

$$\frac{\partial a^{(p\oplus q)}_{\mathrm{stop},z}}{\partial a^{(l,k)}_{\mathrm{stop},i}} = \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{x,q}\big) + \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{y,q}\big). \tag{S.1.53}$$

The Oracle split of Definition S.1.3 sends mass $\frac{1}{2}$ to each branch and applies the discount $\gamma_{\mathrm{O}} = 2$, so the occupation measure satisfies the matching identity

$$\Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s^{\mathrm{add}}_{z,q}\big) = 2 \cdot \tfrac{1}{2}\, \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{x,q}\big) + 2 \cdot \tfrac{1}{2}\, \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{y,q}\big). \tag{S.1.54}$$

Max-pooling step. For a max-pooling node $z = \max\{x_1, \ldots, x_m\}$ with winner $k^\star$, Proposition S.1.5 gives $a^{(p\oplus q)}_{\mathrm{stop},z} = a^{(p\oplus q)}_{\mathrm{stop},x_{k^\star}}$, and the inductive hypothesis yields

$$\frac{\partial a^{(p\oplus q)}_{\mathrm{stop},z}}{\partial a^{(l,k)}_{\mathrm{stop},i}} = \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{x_{k^\star},q}\big). \tag{S.1.55}$$

The deterministic transition with unit discount in Definition S.1.4 gives the matching identity $\Gamma(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s^{\max}_{z,q}) = \Gamma(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{x_{k^\star},q})$. Hence, the correspondence also holds across skip connections and max-pooling nodes, completing the induction. □

S.1.3 Routing Game: Log-Space Formulation

This section makes the two-player payoff structure of the Routing Game explicit. Its global network interpretation uses non-negative activation outputs; the GELU discussion later concerns local gates conditioned on a fixed forward pass. The recursion used in Section 4.3.2 is the logarithmic representation of an entropy-regularized two-player game, and admits a Gibbs subgame-perfect equilibrium for every $\tau > 0$.

S.1.3.1 Exact Routing Kernel Behind $\alpha\beta$-*Layer-wise Relevance Propagation* (LRP)

The linear routing subgames are entropy-regularized one-step maximization problems whose closed form is given by the Legendre–Fenchel identity for Shannon entropy $\mathscr{H}(\pi) \coloneqq -\sum_i \pi(i) \log \pi(i)$ (Boyd and Vandenberghe, 2004): for every $\tau > 0$, $n \in \mathbb{N}$, and payoff vector $r \in \mathbb{R}^n$,

$$\max_{\pi \in \Delta_n} \Big\{ \textstyle\sum_i \pi(i)\, r_i + \tau\, \mathscr{H}(\pi) \Big\} = \tau \log \textstyle\sum_i \exp(r_i/\tau), \qquad \pi^\star(i) \propto \exp(r_i/\tau). \tag{S.1.56}$$

We write

$$\ell(z) \coloneqq \begin{cases} \log z, & z > 0, \\ -\infty, & z \le 0, \end{cases} \qquad \exp(-\infty) \coloneqq 0, \tag{S.1.57}$$

so that $\exp(\ell(x)) = \mathrm{ReLU}(x)$ for any $x \in \mathbb{R}$; we use this fact for notational convenience. Zero or negative routed mass thus receives the stopping value $-\infty$. At every activation state we explicitly choose *stop whenever the two log-space action values tie*. Hence continuing is selected exactly when its log value is strictly greater than $-\infty$, equivalently when the signed routed mass is positive. The ReLU-gating itself is fully modeled by the player policies at states modeling the ReLU gates—just like in the Stopping Game.

State Space and Game Mechanics. To correctly model the interaction between player turns and stream signs, we define a structured state space:

- **Activation States:** $\mathscr{S}_{act}^{(l)} = \{s_{j,p}^{(l,act)}\}$, where $p \in \{+,-\}$ is the player in turn.
- **Linear Routing States:** $\mathscr{S}_{lin}^{(l)} = \{s_{j,q,\kappa}^{(l,lin)}\}$, where q is the player turn and $\kappa \in \{+,-\}$ is the stream sign.

At $s_{j,p}^{(l,act)}$, the active player p chooses between *stop* and *continue*. Under the *continue* action, the environment's transition kernel is uniform over the two sign branches,

$$\mathbb{P}\big(s_{j,p,+}^{(l,lin)} \mid s_{j,p}^{(l,act)}, \text{continue}\big) = \tfrac{1}{2}, \qquad \mathbb{P}\big(s_{j,p',-}^{(l,lin)} \mid s_{j,p}^{(l,act)}, \text{continue}\big) = \tfrac{1}{2}, \tag{S.1.58}$$

where the first branch keeps player p in control evaluating $\kappa = +$ weights and the second switches the turn to opponent p' evaluating $\kappa = -$ weights. To compensate for the $1/2$ branching, the environment applies an Oracle structural discount of $\gamma_{\mathrm{O}} = 2$ (distinct from the per-neuron weight-magnitude discount $\gamma_j^{(l)}$ used in the Stopping Game).

Definition S.1.7 (Entropy-Regularized Two-Player Routing Game). Fix an input x and a stabilizer $\varepsilon \ge 0$. At each activation state $s_{j,p}^{(l,act)}$, the player in turn chooses *stop* or *continue* with similar mechanics as in the Stopping Game (Section S.1.2) reflecting the open-closed state of a ReLU gate: stopping sends the game to the cemetery † giving immediate payoff $\ell(0) = -\infty$—and continuation transitions backward with structural discount $\gamma = 2$ and the uniform $1/2$ sign-branch kernel given above. Here happens the critical step: gains of the opponent on the negative stream state $s_{j,p',-}^{(l,\mathrm{lin})}$ shall be counted as losses of the active player p. At each linear routing state $s_{j,q,\kappa}^{(l,lin)}$, the player in turn instead chooses a mixed action $\nu \in \Delta(\mathrm{Pred}_j^{(l)} \cup \{\mathrm{bias}, \perp_{\mathrm{lin}}\})$ over the predecessors of neuron j augmented by a terminal *bias node* with immediate payoff $\ell([b_j^{(l)}]^\kappa)$ and a fictitious *outside option* $\perp_{\mathrm{lin}}$ which routes the game's mass to the cemetery with immediate payoff $\ell(\varepsilon)$; trajectories entering either

are absorbed. Let $\mathscr{H}(\nu) \coloneqq -\sum_a \nu(a)\log\nu(a)$ be Shannon entropy over this augmented action set. At $\varepsilon = 0$ the outside option carries $\ell(0) = -\infty$ and is dominated by every positive-mass predecessor, so the augmented game reduces to the Routing Game over $\mathrm{Pred}_j^{(l)} \cup \{\text{bias}\}$.

An admissible profile chooses a pure stop/continue action at each activation state and a probability vector at each linear routing state. Under any profile, evaluate the displayed recursions using those choices, without taking maxima: use the chosen activation action and the chosen routing vector's expected log score plus entropy. A zero-probability term contributes zero even when its score is $-\infty$; positive probability on a $-\infty$ score makes the routing value $-\infty$.

Throughout, $\mathbf{V}(\cdot) = (V_p(\cdot), V_{p'}(\cdot))^\top$ denotes the game's backward-induction player-specific log-value function, defined jointly at linear routing states by (S.1.63) and at activation states by (S.1.66) below. The recursion is well-posed because the layered state graph is a finite DAG: the $l = 1$ case of (S.1.59) needs no lower-layer $\mathbf{V}$, and every subsequent layer uses only $\mathbf{V}$ at strictly lower layers.

For a **linear routing state** at $l \geq 2$ the active player continuing to the ith neuron's activation state receives an immediate payoff of the logarithm of the respective weight summed with the respective log-value function of the next state, whereas in the first layer the final payoff is paid out to the active player. The inactive player always obtains an immediate payoff of $\ell(0) = -\infty$. Let us denote by $Q_r(s,a)$ player r's gain for the game value in case action a actually happens, whereas $\bar{Q}_r$ denotes the mixture of the differences between the payoffs of the opponents including the entropy regularization term given the mixed strategy chosen by the active player.

$$Q_r\big(s_{j,q,\kappa}^{(l,lin)}, i\big) \coloneqq \begin{cases} \ell\big(W_{ij}^{(l,\kappa)}\big) + V_r\big(s_{i,q}^{(l-1,act)}\big), & r = q,\ l \geq 2, \\ \ell\big([x_i W_{ij}^{(1)}]^\kappa\big), & r = q,\ l = 1, \\ -\infty, & r = q'. \end{cases} \tag{S.1.59}$$

The outside option $\perp_{\mathrm{lin}}$ and the bias node have log-scores $Q_r(s_{j,q,\kappa}^{(l,\mathrm{lin})}, \perp_{\mathrm{lin}}) \coloneqq \ell(\varepsilon)$ and $Q_r(s_{j,q,\kappa}^{(l,\mathrm{lin})}, \mathrm{bias}) \coloneqq \ell([b_j^{(l)}]^\kappa)$ for $r = q$ and $-\infty$ for $r = q'$; they route the ε-fraction and the bias share of the mass to absorbing terminals instead of to any predecessor. If player q uses mixed action ν at this state, the induced *individual routed masses* are

$$\begin{aligned} \bar{Q}_r\big(s_{j,q,\kappa}^{(l,lin)}; \nu\big) \coloneqq \sum_a \nu(a)\, \ell\left(\exp\left(Q_r\big(s_{j,q,\kappa}^{(l,lin)}, a\big)\right) - \exp\left(Q_{r'}\big(s_{j,q,\kappa}^{(l,lin)}, a\big)\right)\right) \\ + \tau\mathscr{H}(\nu), \qquad r \in \{q, q'\}, \end{aligned} \tag{S.1.60}$$

which, as $\exp(-\infty) = 0$, evaluates to

$$\bar{Q}_q\big(s_{j,q,\kappa}^{(l,lin)}; \nu\big) = \sum_a \nu(a)\, Q_q\big(s_{j,q,\kappa}^{(l,lin)}, a\big) + \tau\mathscr{H}(\nu) \tag{S.1.61}$$

$$\bar{Q}_{q'}\big(s_{j,q,\kappa}^{(l,lin)}; \nu\big) = -\infty. \tag{S.1.62}$$

If $W_{ij}^{(l,\kappa)} = 0$ for every predecessor i and $[b_j^{(l)}]^\kappa = 0$ (i.e. the stream-κ part of the fan-in to neuron j is identically zero), then $Q_q(s_{j,q,\kappa}^{(l,\mathrm{lin})}, i) = -\infty$ for all i as well and the value (S.1.63) equals $\ell(\varepsilon)$. If $\varepsilon = 0$ and every action scores $-\infty$ (e.g. $\Sigma_j^{(l,\kappa)} = 0$ at $\tau = 1$), every ν is a best response; we fix $\nu = \delta_{\perp_{\mathrm{lin}}}$ (absorption), matching the convention $0/0 \coloneqq 0$ in (S.1.85). We define the corresponding equilibrium

linear-game values by

$$V_q^\star\big(s_{j,q,\kappa}^{(l,lin)}\big) \coloneqq \max_{\nu\in\Delta(\mathrm{Pred}_j^{(l)}\cup\{\mathrm{bias},\perp_{\mathrm{lin}}\})} \bar{Q}_q\big(s_{j,q,\kappa}^{(l,lin)};\nu\big)$$
$$= \max_{\nu\in\Delta(\mathrm{Pred}_j^{(l)}\cup\{\mathrm{bias},\perp_{\mathrm{lin}}\})} \left\{\sum_a \nu(a)\, Q_q\big(s_{j,q,\kappa}^{(l,lin)},a\big) + \tau\mathscr{H}(\nu)\right\}$$
$$= \tau\cdot\ell\left(\varepsilon^{\frac{1}{\tau}} + \big([b_j^{(l)}]^\kappa\big)^{\frac{1}{\tau}} + \sum_i \exp\Big(Q_q(s_{j,q,\kappa}^{(l,lin)},i)\Big)^{\frac{1}{\tau}}\right), \tag{S.1.63}$$
$$V_{q'}^\star\big(s_{j,q,\kappa}^{(l,lin)}\big) \coloneqq -\infty, \tag{S.1.64}$$

where we used (S.1.56). At $\tau = 1$ the associated Gibbs policy assigns the outside option the probability $\varepsilon/(\varepsilon + \sum_{a\neq\perp_{\mathrm{lin}}} \exp Q_q(\cdots,a))$ and distributes the remaining mass over the predecessors and the bias node in proportion to $\exp Q_q$.

For an **activation state**, continuing exposes both sign branches. Let $s_+ \coloneqq s_{j,p,+}^{(l,\mathrm{lin})}$ and $s_- \coloneqq s_{j,p',-}^{(l,\mathrm{lin})}$. If player p chooses *continue* at $s_{j,p}^{(l,act)}$, the game transitions to s_+ or to s_- with equal probability, while *stop* sends the game to the cemetery with game value $V_r(\dagger) = \ell(0) = -\infty$. Taking the expectation over the uniform sign-branch kernel ($\frac{1}{2}$ each) and respecting the discount $\gamma_O = 2$ interpreted in logarithmic space, the continuation value becomes

$$Q_r\big(s_{j,p}^{(l,act)},\mathrm{continue}\big) \coloneqq \ell\left(\sum_{s'\in\{s_+,s_-\}} \mathbb{P}(s' \mid s_{j,p}^{(l,act)})\cdot\gamma_{\mathrm{O}}\cdot\Big(\exp V_r(s') - \exp V_{r'}(s')\Big)\right)$$
$$= \ell\left(\xi_{r,p}\Big(\exp\left(V_p\left(s_+\right)\right) - \exp\left(V_{p'}\left(s_-\right)\right)\Big)\right). \tag{S.1.65}$$

Stopping yields $-\infty$ for both players. We select *stop at every activation-state tie in log space*, so the selected policy and active-player value are

$$\pi_p^\star(\mathrm{continue} \mid s_{j,p}^{(l,\mathrm{act})}) \coloneqq \mathbf{1}\Big[Q_p(s_{j,p}^{(l,\mathrm{act})},\mathrm{continue}) > -\infty\Big], \tag{S.1.66}$$
$$V_p(s_{j,p}^{(l,\mathrm{act})}) \coloneqq \max\{-\infty, Q_p(s_{j,p}^{(l,\mathrm{act})},\mathrm{continue})\}. \tag{S.1.67}$$

Writing $D_p = \exp V_p(s_+) - \exp V_{p'}(s_-)$, this selects continuing exactly when $D_p > 0$. When $D_p \leq 0$, both actions have log value $-\infty$ and the convention selects stopping. If continuing is selected, the opponent's continuation value is $-\infty$; if stopping is selected, both players receive $-\infty$. Thus in either case

$$V_{p'}(s_{j,p}^{(l,\mathrm{act})}) = -\infty. \tag{S.1.68}$$

Figure S.1.2 shows a worked example of the local routing subgame.

In the following let us denote, with $a^{(0)} = x$,

$$\Sigma_j^{(l,\kappa)} \coloneqq \textstyle\sum_i [a_i^{(l-1)} W_{ij}^{(l)}]^\kappa + [b_j^{(l)}]^\kappa, \tag{S.1.69}$$

so that $z_j^{(l)} = \Sigma_j^{(l,+)} - \Sigma_j^{(l,-)}$; for $l \geq 2$, $a^{(l-1)} \geq 0$ gives $[a_i^{(l-1)} W_{ij}^{(l)}]^\kappa = a_i^{(l-1)} W_{ij}^{(l,\kappa)}$.

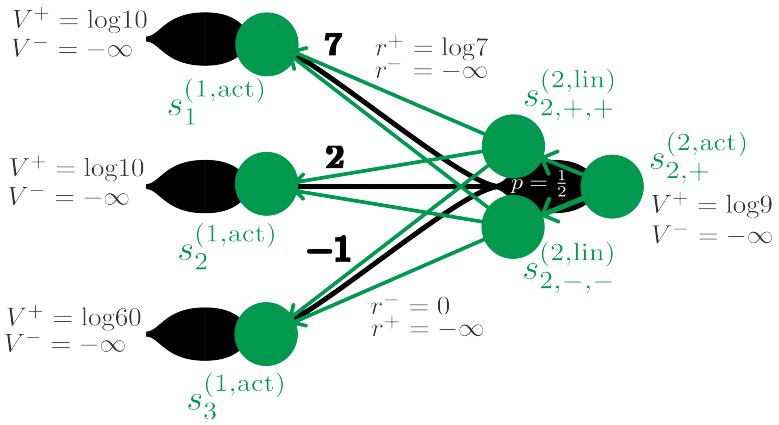


Fig. S.1.2: Routing Game: local routing subgame around $s_{2,+}^{(2,\mathrm{act})}$. This uses the same toy subnetwork and numerical values as Figure S.1.1. At this activation state, P^+ is in turn and continues because the routed mass is positive ($V^+ = \log 9 > -\infty$ in the depicted example). The game then branches with probability 1/2 to $s_{2,+,+}^{(2,\mathrm{lin})}$ (turn stays with P^+) and with probability 1/2 to $s_{2,-,-}^{(2,\mathrm{lin})}$ (turn switches to P^-). In each linear state, the active player samples a predecessor according to the Gibbs policy from the immediate log-payoff and game value. Example: along $s_{2,+,+}^{(2,\mathrm{lin})} \to s_{1,+}^{(1,\mathrm{act})}$, the player on the positive stream state gets immediate payoff $\log W_{12}^{(2,+)} = \log 7$ and P^- gets $-\infty$. Edges with zero routed mass have payoff $\log W_{12}^{(2,-)} = \log 0 = -\infty$. For readability, the $\pm$ activation copies in layer 1 are collapsed, and the intermediate payoff $\log 2$ on the transition from position 2 to 2 is omitted.

Theorem S.1.7 (SPNE and Forward Equivalence of the Routing Game, `Lean_63`).

(i) *SPNE existence for every $\tau > 0$ and $\varepsilon \geq 0$. For every $\tau > 0$ and every $\varepsilon \geq 0$, the entropy-regularized two-player Routing Gameof Definition S.1.7 (with linear-state entropy weight $\tau\mathscr{H}$ and outside-option stabilizer ε) admits a subgame-perfect Nash equilibrium (see (S.1.74)–(S.1.75) in the proof for the explicit form of the equilibrium policy). We write $\cdot^\star$ for equilibrium quantities throughout.*

(ii) *Forward equivalence at the anchor $\tau = 1$, $\varepsilon \geq 0$. At $\tau = 1$ and $\varepsilon \geq 0$, the equilibrium quantities recover the network forward pass: for every layer l, neuron j, player labels p,q,r, and stream sign $\kappa \in \{+,-\}$,*

$$V_r\big(s_{j,q,\kappa}^{(l,lin)}\big) = \ell\big(\delta_{r,q}\,(\Sigma_j^{(l,\kappa)} + \varepsilon)\big), \tag{S.1.70}$$

$$Q_r\left(s_{j,p}^{(l,act)}, \mathit{continue}\right) = \ell\left(\xi_{r,p} \cdot z_j^{(l)}\right), \tag{S.1.71}$$

$$V_r\big(s_{j,p}^{(l,act)}\big) = \ell\big(\delta_{r,p}\ \cdot a_j^{(l)}\big), \tag{S.1.72}$$

$$\pi_p^\star\big(s_{j,p}^{(l,act)}\big) = \mathbf{1}[z_j^{(l)} > 0]. \tag{S.1.73}$$

Remark S.1.2 (Uniqueness). The linear-state Gibbs best response (S.1.74) is the unique maximizer of the strictly concave entropy-regularized objective whenever some action has a finite score (otherwise Definition S.1.7 fixes $\perp_{\mathrm{lin}}$). At activation states the active player is indifferent whenever the signed continuation mass $\exp V_p(s_+) - \exp V_{p'}(s_-)$ is nonpositive ($z_j^{(l)} \leq 0$ at $\tau = 1$), because both log-space action values are then $-\infty$. Thus the unconstrained best-response set can contain several policies. The theorem specifies the tie-breaker $\pi_p^\star = \mathbf{1}[Q_p(\cdot,\mathrm{continue}) > -\infty]$, which at $\tau = 1$ reduces to $\mathbf{1}[z_j^{(l)} > 0]$. Just like in the ***S**topping **G**ame* (SG) (Section S.1.2), we refer to this SPNE as the **stop-at-tie-breaks**

equilibrium. This canonical choice also selects the absorbing outside action whenever every routing action has zero mass; without these conventions, both activation log ties and empty routing streams permit multiple optimal choices.

Proof. **SPNE existence for every $\tau > 0$.** The layered state graph is a finite DAG: linear routing states at layer l transition to activation states at layer $l-1$ (or to terminal input positions at $l=1$), and activation states at layer l transition to linear routing states at the same layer l or to the cemetery $\dagger$. We construct the equilibrium by backward induction on the layer index l.

At layer $l = 1$ the linear routing subgames have no unresolved game values, so the action values Q_q in (S.1.59) are fully specified by the input x. At each linear state the active player solves $\max_{\nu \in \Delta(\mathrm{Pred}_j^{(1)})} \{\sum_i \nu(i)\, Q_q(s_{j,q,\kappa}^{(1,lin)}, i) + \tau\, \mathscr{H}(\nu)\}$, which is strictly concave on the simplex face supported on finite-score actions whenever that face is nonempty. If every score is $-\infty$, every distribution is optimal and we select the absorbing outside action as above. Identity (S.1.56) yields the unique Gibbs best response in the nonempty finite-score case, and simplifying $\exp \circ \ell$ in (S.1.59) gives the explicit form

$$\pi_q^\star\big(i \,\big|\, s_{j,q,\kappa}^{(l,lin)}\big) \propto \begin{cases} \big(W_{ij}^{(l,\kappa)}\big)^{1/\tau} \exp\big(V_q(s_{i,q}^{(l-1,act)})/\tau\big), & l \geq 2, \\ \big([x_i W_{ij}^{(1)}]^\kappa\big)^{1/\tau}, & l = 1, \end{cases} \tag{S.1.74}$$

with zero mass on routes of zero or wrong sign and closed-form value

$$V_q(s_{j,q,\kappa}^{(1,lin)}) = \tau \log \sum_i \exp(Q_q(s_{j,q,\kappa}^{(1,lin)}, i)/\tau).$$

With these values in hand, the activation-state subgame at layer $l = 1$ is a binary choice between stop (log-payoff $\ell(0) = -\infty$) and continue (S.1.65); comparing log-space action values and choosing stopping at every tie, the active player's best response is the pure stop/continue strategy

$$\pi_p^\star\big(s_{j,p}^{(l,act)}\big) = \mathbf{1}\big[Q_p(s_{j,p}^{(l,act)}, \text{continue}) > -\infty\big]. \tag{S.1.75}$$

Assume the equilibrium has been constructed at all layers $\leq l-1$, yielding definite values $V_r(s_{i,q}^{(l-1,act)})$. At layer l the linear-state action values Q_q in (S.1.59) are then specified, and the same strict-concavity argument gives the Gibbs best response (S.1.74) with the analogous log-sum-exp value. The layer-l activation-state best response is again the binary hard comparison (S.1.75). Because each subgame's best response is taken to be optimal given the game values from lower layers, the composed strategy profile is subgame-perfect. This proves existence of an SPNE for every $\tau > 0$. The argument is unchanged under the bias-node and outside-option augmentation of Definition S.1.7: the linear-state action set gains the extra elements bias and $\perp_{\mathrm{lin}}$ with constant log-scores $\ell([b_j^{(l)}]^\kappa)$ and $\log \varepsilon$ (finite or $-\infty$), the strict-concavity / Legendre–Fenchel step goes through verbatim on the augmented simplex, and the closed-form value picks up the extra terms $([b_j^{(l)}]^\kappa)^{1/\tau}$ and $\varepsilon^{1/\tau}$ inside the log-sum-exp. Hence, an SPNE exists for every $(\tau, \varepsilon) \in \mathbb{R}_{>0} \times \mathbb{R}_{\geq 0}$.

Forward equivalence at $\tau = 1$. We now specialize to $\tau = 1$ and show by induction on l that the equilibrium quantities recover the network forward pass.

Base case ($l = 1$): Identical to the inductive step below, with the fused-input action values $Q_q(s_{j,q,\kappa}^{(1,lin)}, i) = \ell([x_i W_{ij}^{(1)}]^\kappa)$ of (S.1.59) in place of $\ell(a_i^{(l-1)} W_{ij}^{(l,\kappa)})$.

Inductive step ($l \geq 2$): Assume $V_q(s_{i,q}^{(l-1,act)}) = \ell(a_i^{(l-1)})$ for all predecessors i and both player labels q. Since the assumed non-negative activation functions satisfy $a_i^{(l-1)} \geq 0$, the linear routing-state action values become

$$Q_q\big(s_{j,q,\kappa}^{(l,lin)}, i\big) = \ell\big(W_{ij}^{(l,\kappa)}\big) + \ell\big(a_i^{(l-1)}\big) = \ell\big(a_i^{(l-1)} W_{ij}^{(l,\kappa)}\big). \tag{S.1.76}$$

Applying (S.1.56) to the linear routing subgame gives the Gibbs best response (S.1.74) and

$$V_q\big(s_{j,q,\kappa}^{(l,lin)}\big) = \ell\left(\sum_a \exp\big(Q_q(s_{j,q,\kappa}^{(l,lin)}, a)\big)\right) = \ell\left(\sum_i a_i^{(l-1)} W_{ij}^{(l,\kappa)} + [b_j^{(l)}]^\kappa + \varepsilon\right) = \ell(\Sigma_j^{(l,\kappa)} + \varepsilon), \tag{S.1.77}$$

At the activation state the active player's continuation advantage is again $z_j^{(l)}$

$$Q_p\big(s_{j,p}^{(l,act)}, \text{continue}\big) = \ell\left(\Sigma_j^{(l,+)} - \Sigma_j^{(l,-)}\right) = \ell(z_j^{(l)}), \tag{S.1.78}$$

whereas for the opponent

$$Q_{p'}\left(s_{j,p}^{(l,\text{act})}, \text{continue}\right) = \ell\left(\Sigma_j^{(l,-)} - \Sigma_j^{(l,+)}\right) = \ell(-z_j^{(l)}), \tag{S.1.79}$$

again confirming (S.1.71). Thus, the best response is (S.1.75). Finally,

$$V_r\big(s_{j,p}^{(l,act)}\big) = \ell\big(\delta_{r,p}\, \mathbf{1}[z_j^{(l)} > 0]\, z_j^{(l)}\big) = \ell\big(\delta_{r,p}\, a_j^{(l)}\big), \tag{S.1.80}$$

proving (S.1.72). Since each subgame has an optimal action chosen by the player in turn, the resulting strategy profile is subgame-perfect. This completes the induction. □

Log-Space Addition Subgame, Skip Connections, and Max Pooling

Residual additions $z = x + y$ and max-pooling nodes $z = \max_k x_k$ enter the ***R**outing **G**ame* (RG) through the addition state $s_{z,p}^{\text{add}}$ (uniform Oracle to the operand states, player p retained, $\gamma_{\text{O}} = 2$) and the pooling state $s_{z,p}^{\max}$ (deterministic choice of an operand, player p retained, $\gamma_{\text{O}} = 1$, ties broken by the smallest index) of Section S.1.2.3, with the values written in log space:

$$V_r(s_{z,p}^{\text{add}}) = \ell\big(\exp V_r(s_{x,p}) + \exp V_r(s_{y,p})\big), \qquad V_p(s_{z,p}^{\max}) = \max_k V_p(s_{x_k,p}). \tag{S.1.81}$$

If the operand states recover their (non-negative) activations, $V_p(s_{x,p}) = \ell(a_x)$ etc., then $V_p(s_{z,p}^{\text{add}}) = \ell(a_x + a_y) = \ell(a_z)$ and $V_p(s_{z,p}^{\max}) = \ell(a_{x_{k^\star}}) = \ell(a_z)$, so forward equivalence extends across both nodes. Two things differ from the SG. First, $\gamma_{\text{O}} = 2$ enters only the values: the trajectory discount (S.1.83) of the addition transition is 1, so each operand receives half of the addition-state occupation (Theorem S.1.8, item 2). Second, the relevance split therefore equals the Oracle probabilities: a $(\rho, 1-\rho)$ Oracle with value discounts $(1/\rho, 1/(1-\rho))$ leaves all forward values unchanged but splits the relevance as $(\rho, 1-\rho)$, so the uniform split is a convention of the RG rule. The implementation passes the full relevance to both operands (trajectory discount 2); since every backward trajectory of a ResNet or ***Vi**sion **T**ransformer* (ViT) that reaches the input crosses each of its B addition nodes exactly once, this rescales the input occupation by the constant 2^B and leaves normalized maps unchanged.

S.1.3.2 Backward Path Integral and $\alpha\beta$-LRP Recovery

We now show that the projected discounted occupation measure induced by the SPNE on the layered routing graph realizes the $\alpha\beta$-LRP-ε relevance vector of (4.1). To keep the game side and the attribution side strictly separate, we use distinct symbols throughout this subsection:

- the *game-side* object is the discounted occupation measure $\Gamma_x(\,\cdot\,)$ on the states of the routing graph (Definition S.1.6), now driven by the SPNE trajectory law Π_x rooted at $s_{u,+}^{(L,\mathrm{act})}$ with unit initial mass: $\Gamma_x(s_{u,+}^{(L,\mathrm{act})}) = 1$ and $\Gamma_x(s_{j,p}^{(L,\mathrm{act})}) = 0$ for every other output activation state. With non-negative branch discounts (cf. (S.1.83)) Γ_x is itself non-negative; we then form the per-neuron difference of the two player copies,

$$\Gamma_j^{(l)} \;\coloneqq\; \Gamma_x\big(s_{j,+}^{(l,\mathrm{act})}\big) \;-\; \Gamma_x\big(s_{j,-}^{(l,\mathrm{act})}\big). \tag{S.1.82}$$

- the *attribution-side* object is the $\alpha\beta$-LRP-ε relevance vector $R^{(l)}$ (the $\alpha\beta$ rule of Bach et al. (2015) with the stabilization convention adopted here, see below), a deterministic backward DP on the network's linear layers seeded by a chosen output mass $R_u^{(L)}$ and recursing via (S.1.85) below.

The recovery theorem (Theorem S.1.8) shows that, under the SPNE policy and the following trajectory discount (S.1.83), the per-neuron difference $\Gamma_j^{(l)}$ satisfies the same recursion as $R_j^{(l)}$, hence equals $R_j^{(l)}/R_u^{(L)}$.

Trajectory discount. We track sign-dependent branch discounts governed by the stream-balance mode (α, β) $(\alpha - \beta = 1)$. Let $\zeta = (\zeta_0, \zeta_1, \dots)$ be a backward trajectory on linear routing, activation, addition or max-pooling states. Define the per-state discount γ as a non-negative function of the entered state,

$$\gamma(s) \coloneqq \begin{cases} 2\alpha, & s = s_{j,q,+}^{(l,lin)} \text{ for some } j,q,l, \\ 2\beta, & s = s_{j,q,-}^{(l,lin)} \text{ for some } j,q,l, \\ 1, & \text{otherwise (activation, addition Oracle, max-pool).} \end{cases} \tag{S.1.83}$$

The cumulative discount along the trajectory up to time t is the product

$$d_t(\zeta) \coloneqq \prod_{m=1}^{t} \gamma(\zeta_m), \qquad d_0(\zeta) = 1 \text{ (empty product).} \tag{S.1.84}$$

The signed minus of the $\alpha\beta$-LRP recursion is produced by the difference (S.1.82) between the two player copies.

Remark S.1.3 (Choice of starting player). Rooting Π_x at $s_{u,-}^{(L,\mathrm{act})}$ instead of $s_{u,+}^{(L,\mathrm{act})}$ swaps the two output-layer occupations and therefore negates $\Gamma_j^{(l)}$ at every layer. We fix the canonical convention that the game starts at the $+$-state.

Remark S.1.4 (Equivalent signed-discount sum formulation). The same recovery theorem can be phrased with the signed trajectory discount $\gamma(s_{j,q,+}^{(l,\mathrm{lin})}) = 2\alpha$, $\gamma(s_{j,q,-}^{(l,\mathrm{lin})}) = -2\beta$ and the per-neuron *sum* $\Gamma_j^{(l)} = \Gamma_x(s_{j,+}^{(l,\mathrm{act})}) + \Gamma_x(s_{j,-}^{(l,\mathrm{act})})$ in place of (S.1.82). We have chosen the non-negative-discount version

because it is easier to comprehend—Γ_x remains a genuine non-negative occupation measure, and the LRP sign is generated by an explicit algebraic difference rather than absorbed into the trajectory weights.

Attribution-side: $\alpha\beta$-LRP-ε relevance. Independently of the game, the $\alpha\beta$-LRP-ε relevance vector $R^{(l)}$ is defined recursively from $R_j^{(L)} = R_u^{(L)}\,\delta_{j,u}$ at the output by

$$R_i^{(l-1)} \;=\; \sum_j \left(\alpha \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\varepsilon + \Sigma_j^{(l,+)}} - \beta \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\varepsilon + \Sigma_j^{(l,-)}} \right) R_j^{(l)}, \tag{S.1.85}$$

where $\Sigma_j^{(l,\pm)}$ is (S.1.69): the bias enters the denominators as a terminal bias node, and the relevance routed to it (and to the outside option) is absorbed, as in the implementation. Recursion (S.1.85) is the $\alpha\beta$ rule of Bach et al. (2015) with the stabilization convention adopted throughout this paper: the stabilizer ε of their LRP-ε rule is added separately to each sign denominator. Bach et al. (2015) present the two rules as alternative constructions and do not state this combined form; it is the convention of common LRP implementations (Anders et al., 2023), and Theorem S.1.8 is exact for this form. At $\varepsilon = 0$ it is the original $\alpha\beta$ rule.

Theorem S.1.8 (Recovery of Multi-Path $\alpha\beta$-LRP, `Lean_64`). *For every $\varepsilon \geq 0$ and every input with $f(x) = a_u^{(L)} > 0$ (if $f(x) = 0$ the root player stops and $R^{(l)} = f(x)\Gamma^{(l)} = 0$ trivially), under the stop-at-tie-breaks equilibrium (Remark S.1.2; ties at max-pooling states broken by the smallest index) at the anchor temperature $\tau = 1$ and stabilizer ε, with the cumulative trajectory discount (S.1.84) and the occupation measure $\Gamma_j^{(l)}$ from (S.1.82):*

1) ***Linear Map.*** *The per-neuron difference $\Gamma_j^{(l)}$ of (S.1.82) satisfies*

$$\Gamma_i^{(l-1)} \;=\; \sum_j \left(\alpha \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\varepsilon + \Sigma_j^{(l,+)}} - \beta \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\varepsilon + \Sigma_j^{(l,-)}} \right) \Gamma_j^{(l)}. \tag{S.1.86}$$

   Because (S.1.86) and (S.1.85) share their computational step, $\Gamma_j^{(l)} \cdot R_u^{(L)} = R_j^{(l)}$ at every layer; in particular $R_j^{(l)} = f(x)\,\Gamma_j^{(l)}$ for the seed $R_u^{(L)} = f(x)$. The same formula covers the fused input layer with $a^{(0)} = x$. At $\varepsilon = 0$ this reduces to the unstabilized $\alpha\beta$-LRP redistribution (not conservative: bias nodes absorb relevance); at $\varepsilon > 0$ it is the $\alpha\beta$ rule with the stabilization convention adopted here, i.e. (4.1) with ε added to each sign denominator separately.

2) ***Residual Addition.*** *Let $z = x_{\mathrm{op}} + y_{\mathrm{op}}$ be an addition node with addition state $s_{z,p}^{\mathrm{add}}$ and operand states $s_{x_{\mathrm{op}},p}, s_{y_{\mathrm{op}},p}$ as in Section S.1.3.1. For every player label $p \in \{+,-\}$, with the edge occupation $\Gamma_x(s \to s') := \mathbb{E}_{\zeta\sim\Pi_x}\big[\sum_t d_{t+1}(\zeta)\mathbf{1}\{\zeta_t = s, \zeta_{t+1} = s'\}\big]$ (an operand's total occupation adds the edges from its other parents),*

$$\Gamma_x\big(s_{z,p}^{\mathrm{add}} \to s_{x_{\mathrm{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\mathrm{add}}\big), \qquad \Gamma_x\big(s_{z,p}^{\mathrm{add}} \to s_{y_{\mathrm{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\mathrm{add}}\big), \tag{S.1.87}$$

   so the operand pair carries the full addition-state mass with no duplication.

3) ***Max Pooling.*** *For a pooled output $z = \max\{x_1, \ldots, x_m\}$ with winner $k^\star$ and pooling state $s_{z,p}^{\max}$ (Section S.1.3.1), the deterministic value-maximizing transition concentrates all mass on the*

winner: for every player label $p \in \{+,-\}$,

$$\Gamma_x\big(s^{\max}_{z,p} \to s_{x_{k^\star},p}\big) \;=\; \Gamma_x\big(s^{\max}_{z,p}\big), \qquad \Gamma_x\big(s^{\max}_{z,p} \to s_{x_k,p}\big) \;=\; 0 \text{ for } k \neq k^\star. \tag{S.1.88}$$

Proof. We prove $R^{(L)}_u \cdot \Gamma^{(l)}_j = R^{(l)}_j$ at every layer l by backward induction on l.

Base case ($l = L$): The trajectory law Π_x has unit mass at $s^{(L,\mathrm{act})}_{u,+}$ by definition, so $\Gamma_x(s^{(L,\mathrm{act})}_{u,+}) = 1$ and $\Gamma_x(s^{(L,\mathrm{act})}_{j,p}) = 0$ for every $(j,p) \neq (u,+)$. By (S.1.82),

$$\Gamma^{(L)}_j \;=\; \Gamma_x\big(s^{(L,\mathrm{act})}_{j,+}\big) - \Gamma_x\big(s^{(L,\mathrm{act})}_{j,-}\big) \;=\; \delta_{j,u}, \tag{S.1.89}$$

i.e. the difference is 1 at the chosen output neuron u and 0 elsewhere. Multiplying by the output mass gives $R^{(L)}_u \cdot \Gamma^{(L)}_j = R^{(L)}_u\, \delta_{j,u} = R^{(L)}_j$, matching the LRP root specification of (S.1.85).

Inductive step ($l \Rightarrow l-1$): Assume $R^{(L)}_u \cdot \Gamma^{(l)}_j = R^{(l)}_j$ for every neuron j in layer l. From the SPNE construction in Theorem S.1.7 together with the activation-game value reduction $V_p(s^{(l,\mathrm{act})}_{j,p}) = \ell(a^{(l)}_j)$ (which holds for every $\varepsilon \geq 0$ because the additive ε-shifts on the two sign branches cancel in the continuation advantage), the linear-state Gibbs best response on the alive support is

$$\pi^\star_{j,\kappa}(\perp_{\mathrm{lin}}) = \frac{\varepsilon}{\varepsilon + \Sigma^{(l,\kappa)}_j}, \qquad \pi^\star_{j,\kappa}(\mathrm{bias}) = \frac{[b^{(l)}_j]^\kappa}{\varepsilon + \Sigma^{(l,\kappa)}_j}, \qquad \pi^\star_{j,\kappa}(i) = \frac{a^{(l-1)}_i W^{(l,\kappa)}_{ij}}{\varepsilon + \Sigma^{(l,\kappa)}_j}. \tag{S.1.90}$$

A trajectory reaches the layer-$(l-1)$ activation state $s^{(l-1,\mathrm{act})}_{i,r}$ from a layer-l activation state through one of two routes. By the activation-state continuation kernel of Definition S.1.7, departing from $s^{(l,\mathrm{act})}_{j,p}$ (the player continues: at $l = L$ since $f(x) > 0$; at $l < L$, $\Gamma_x(s^{(l,\mathrm{act})}_{j,\cdot}) = 0$ unless $a^{(l)}_j > 0$ by (S.1.90)) the trajectory takes the $+$ branch to $s^{(l,\mathrm{lin})}_{j,p,+}$ with probability $\frac{1}{2}$ (player p retains the turn), and the $-$ branch to $s^{(l,\mathrm{lin})}_{j,p',-}$ with probability $\frac{1}{2}$ (turn switches to opponent p'); from each linear state the policy (S.1.90) routes to predecessor i with the player label of that linear state. Combining the two branch discount factors $2\alpha, 2\beta$ from (S.1.83) with these $\frac{1}{2}$-probabilities, the trajectory contributions to $\Gamma_x(s^{(l-1,\mathrm{act})}_{i,r})$ are

$$\begin{aligned}\Gamma_x\big(s^{(l-1,\mathrm{act})}_{i,r}\big) &= \sum_j \Big(\tfrac{1}{2}(2\alpha)\,\pi^\star_{j,+}(i)\,\Gamma_x\big(s^{(l,\mathrm{act})}_{j,r}\big) \;+\; \tfrac{1}{2}(2\beta)\,\pi^\star_{j,-}(i)\,\Gamma_x\big(s^{(l,\mathrm{act})}_{j,r'}\big)\Big) \\ &= \sum_j \Big(\alpha\,\pi^\star_{j,+}(i)\,\Gamma_x\big(s^{(l,\mathrm{act})}_{j,r}\big) \;+\; \beta\,\pi^\star_{j,-}(i)\,\Gamma_x\big(s^{(l,\mathrm{act})}_{j,r'}\big)\Big),\end{aligned} \tag{S.1.91}$$

where $r' \in \{+,-\}$ denotes the opposite player label and the absorbing actions bias and $\perp_{\mathrm{lin}}$ contribute no mass to any predecessor.

Differencing (S.1.91) over $r \in \{+,-\}$ (i.e. $r{=}+$ minus $r{=}-$) flips the role of r and r' between the two terms inside the bracket. Using $\Gamma_x(s^{(l,\mathrm{act})}_{j,r}) - \Gamma_x(s^{(l,\mathrm{act})}_{j,r'}) = +\Gamma^{(l)}_j$ when $r{=}+$, $r'{=}-$, and the same

expression with the opposite sign when $r{=}-$, $r'{=}+$, the per-neuron difference at layer $l-1$ satisfies

$$\begin{aligned}\Gamma_i^{(l-1)} &= \Gamma_x\big(s_{i,+}^{(l-1,\mathrm{act})}\big) - \Gamma_x\big(s_{i,-}^{(l-1,\mathrm{act})}\big)\\ &= \sum_j \Big(\alpha\,\pi^\star_{j,+}(i)\,\big[\Gamma_x(s_{j,+}^{(l,\mathrm{act})}) - \Gamma_x(s_{j,-}^{(l,\mathrm{act})})\big] \;+\; \beta\,\pi^\star_{j,-}(i)\,\big[\Gamma_x(s_{j,-}^{(l,\mathrm{act})}) - \Gamma_x(s_{j,+}^{(l,\mathrm{act})})\big]\Big)\\ &= \sum_j \big(\alpha\,\pi^\star_{j,+}(i) - \beta\,\pi^\star_{j,-}(i)\big)\,\Gamma_j^{(l)}\\ &= \sum_j \left(\alpha\frac{[a_i^{(l-1)}W_{ij}^{(l)}]^+}{\varepsilon+\Sigma_j^{(l,+)}} - \beta\frac{[a_i^{(l-1)}W_{ij}^{(l)}]^-}{\varepsilon+\Sigma_j^{(l,-)}}\right)\Gamma_j^{(l)},\end{aligned} \tag{S.1.92}$$

where the third equality regroups the two square brackets as $+\Gamma_j^{(l)}$ and $-\Gamma_j^{(l)}$ respectively, and the fourth substitutes (S.1.90) together with $a_i^{(l-1)}W_{ij}^{(l,\kappa)} = [a_i^{(l-1)}W_{ij}^{(l)}]^\kappa$ for non-negative $a_i^{(l-1)}$. Multiplying both sides by $R_u^{(L)}$ and using the inductive hypothesis $R_u^{(L)}\cdot\Gamma_j^{(l)} = R_j^{(l)}$,

$$R_u^{(L)}\cdot\Gamma_i^{(l-1)} \;=\; \sum_j \left(\alpha\frac{[a_i^{(l-1)}W_{ij}^{(l)}]^+}{\varepsilon+\Sigma_j^{(l,+)}} - \beta\frac{[a_i^{(l-1)}W_{ij}^{(l)}]^-}{\varepsilon+\Sigma_j^{(l,-)}}\right)R_j^{(l)} \;=\; R_i^{(l-1)}. \tag{S.1.93}$$

This closes the induction and proves $R_u^{(L)}\cdot\Gamma_j^{(l)} = R_j^{(l)}$ for every layer l and neuron j.

Input layer ($l=1$): The same calculation applies with the fused first-layer masses $[x_i W_{ij}^{(1)}]^\kappa$ in place of $a_i^{(l-1)}W_{ij}^{(l,\kappa)}$, yielding the identical formula with $a^{(0)} = x$.

Addition node: Let $z = x_{\mathrm{op}} + y_{\mathrm{op}}$ be an addition node. By Section S.1.3.1, from $s_{z,p}^{\mathrm{add}}$ the Oracle transitions to $s_{x_{\mathrm{op}},p}$ and $s_{y_{\mathrm{op}},p}$ with probability $\frac{1}{2}$ each, retaining player label p. Under the trajectory-discount convention (S.1.83)–(S.1.84), addition Oracle transitions carry trajectory-discount factor 1. Hence

$$\Gamma_x\big(s_{z,p}^{\mathrm{add}} \to s_{x_{\mathrm{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\mathrm{add}}\big), \qquad \Gamma_x\big(s_{z,p}^{\mathrm{add}} \to s_{y_{\mathrm{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\mathrm{add}}\big), \tag{S.1.94}$$

proving Item 2 of Theorem S.1.8 for both player labels.

Max-pooling node: Let $z = \max\{x_1,\ldots,x_m\}$ with winner $k^\star$ and pooling state $s_{z,p}^{\max}$. By Section S.1.3.1, the active player has the unique value-maximizing action $x_{k^\star}$ at $s_{z,p}^{\max}$, so the transition to $s_{x_{k^\star},p}$ is deterministic, retains player label p, and (S.1.83) assigns unit trajectory factor. Therefore $\Gamma_x(s_{x_{k^\star},p}) = \Gamma_x(s_{z,p}^{\max})$ and $\Gamma_x(s_{x_k,p}) = 0$ for $k \neq k^\star$, proving Item 3. □

S.1.3.3 Temperature τ as a Controlled Policy Deformation

Theorem S.1.7(i) establishes an SPNE for every $\tau > 0$ when the entropy weight τ is used throughout the backward recursion, but exact $\alpha\beta$-LRP recovery (Theorem S.1.8) holds only at $\tau = 1$. In practice, we deform τ only *locally* at each linear subgame's in-round Gibbs readout,

$$\pi_{j,\kappa}^{(\tau)\star}(i) \propto \exp\big(Q_q(s_{j,q,\kappa}^{(l,lin)}, i)/\tau\big), \tag{S.1.95}$$

while keeping the game values $V(s_{i,q}^{(l-1,act)})$ at their $\tau{=}1$ equilibrium—i.e. at the network activations recovered by Theorem S.1.7(ii). The players are therefore *unaware* of the τ-deformation (a *myopic* policy): their decisions rest on the Bellman equations at temperature 1 (the network forward pass),

and only the local routing readout is perturbed. Under non-negative predecessor activations this gives $\pi_{j,\kappa}^{(\tau)\star}(i) \propto (a_i^{(l-1)} W_{ij}^{(l,\kappa)})^{1/\tau}$, so $\tau < 1$ sharpens and $\tau > 1$ flattens the route distribution. This controlled deformation is used in the experiments; it carries no exactness claim at $\tau \neq 1$. The implementation raises activations, weights and bias-node masses to the power $1/\tau$ but keeps the outside-option weight at ε rather than $\varepsilon^{1/\tau}$. On ResNet-50 it passes relevance through the global average pool uniformly (not in proportion to the activations) and merges batch normalization into the preceding convolution with an ε-stabilized rule, so the ResNet-50 anchor is $\alpha\beta$-LRP up to these two operations; VGG-16 (whose average pool is the identity at 224×224) and ViT-B/16 are not affected.

S.1.4 Analytical Modes

This section collects the formal derivations for the analytical modes that the game framework exposes on top of the $\alpha\beta$-LRP anchor: the local temperature deformation τ on the linear-state Gibbs readout (Section S.1.3.3), the activation-side risk-aversion λ via *Assumed Density Filtering* (ADF) variance propagation, the noisy-observation gate $\tilde{\Phi}$ on the player's stop/continue comparison, and the fan-in entropy bias λ_{ent}. Each mode maps a concept in game space (focus, risk aversion) to an explicit modification of the backward propagation rule; none carries an exactness claim, and each recovers the anchor at its default ($\tau = 1$, $\lambda = 0$, $\tilde{\Phi}$ off, $\lambda_{\text{ent}} = 0$). The mode-by-mode empirical sweeps live in Section S.4.4.3.

S.1.4.1 Defaults, costs and order of use

Table S.1.2 lists every mode with its default, its effect and its measured cost (median seconds per image, one A100 GPU, batch size 1; Section S.4.3.3). At the defaults, SG is the gradient and RG is $\alpha\beta$-LRP. A practical order of use: (i) start at the anchor; (ii) vary τ first, since it is free and moves routing between focused and diffuse maps (Table S.4.17); (iii) add risk aversion (λ on ***C**onvolutional **N**eural **N**etwork*s (CNNs), λ_{sm} on ViTs, with the input variance σ^2) when the maps should discount routes through high-variance units, at the price of one ADF pass; (iv) use equivariant averaging on ViTs only when token-grid artifacts matter, since it multiplies the cost by N. The validation selections of Section S.4.4.1 show which combination the protocol picks per model and criterion.

Tab. S.1.2: Modes: default (anchor) value, effect and measured cost per image.

Mode	Default	Effect	Cost (s/image)
(α,β), ε	(2,1), 0.5	LRP stream weights and stabilizer (exact)	none
τ	1	sharper ($\tau < 1$) or more diffuse ($\tau > 1$) routing	none (VGG-16 0.035 → 0.037, ResNet-50 0.089 → 0.094)
λ, σ^2	λ=0	attenuates routes through high-variance units	one ADF pass (ResNet-50 0.089 → 0.140)
λ_{sm}, σ^2	0	downweights high-variance keys in attention routing	one ADF pass (ViT-B/16 0.097 → 0.172)
$\tilde{\Phi}$	off	smooth stop/continue gate	needs the ADF σ (one ADF pass)
N	off	averages over input transformations (token-grid artifacts)	$\times N$ (ViT-B/16, N=50: 4.93; with λ_{sm}: 8.66)

S.1.4.2 Activation-Side Risk Aversion λ via ADF Variance Propagation

The activation-side risk-aversion mode replaces the deterministic activation $a_i^{(l)} = \text{ReLU}(z_i^{(l)})$ that the player observes by a mean–variance confidence bound $V_\lambda = \mu + \lambda\sigma$ ($\lambda \leq 0$) computed from an ADF

variance propagation pass (Gast and Roth, 2018). We propagate the exact mean $\mathbb{E}[\mathrm{ReLU}(X)]$ and the variance approximation $\sigma^2\Phi(\mu/\sigma)$ of $\mathrm{Var}[\mathrm{ReLU}(X)]$ for $X \sim \mathcal{N}(\mu, \sigma^2)$ (Gast and Roth, 2018).

The propagated variance is initialized at the input by the scale σ^2 at every input neuron. Negative λ shifts the player's observed game value at stopping layers (not linear layers!) by a lower-confidence amount, attenuating routes through high-variance neurons; this is the activation-side analog of confidence-bound exploration in bandit / RL settings (Auer et al., 2002; Fei et al., 2020). The softmax-shift variant λ_{sm} on the attention-logit oracle (ViT) is treated separately in Section S.2.5.

S.1.4.3 Noisy-Observation Gate $\tilde{\Phi}$

The noisy-observation mode replaces the player's pure stop/continue policy at every activation state by a *mixed* policy. At $s_{i,q}^{(l,\mathrm{act})}$, the player continues with probability

$$\tilde{\Phi}(z_i^{(l)}) \;\coloneqq\; \Phi\big(\mu_i^{(l)}/\sigma_i^{(l)} + \lambda\big) \tag{S.1.96}$$

and stops with the complementary probability $1-\tilde{\Phi}(z_i^{(l)})$, where Φ is the standard normal *Cumulative Distribution Function* (CDF) and $\mu_i^{(l)}$, $\sigma_i^{(l)}$ are the ADF mean and standard deviation of $z_i^{(l)}$ (Section S.1.4.2). The analysis below takes $\mu_i^{(l)} = z_i^{(l)}$, $\sigma_i^{(l)} = 1$ and $\lambda = 0$; the general case follows with $z \mapsto \mu + \lambda\sigma$ and $\mathscr{R}_\Phi \mapsto \sigma\mathscr{R}_\Phi$. The mixed policy (S.1.96) arises as the unique solution of a regularized maximization that the player can solve directly. Define the probit regularizer

$$\mathscr{R}_\Phi(\pi) \;\coloneqq\; \phi\big(\Phi^{-1}(\pi)\big), \qquad \pi \in [0,1], \tag{S.1.97}$$

where $\phi(y) = (2\pi)^{-1/2}\, e^{-y^2/2}$ is the standard normal probability density and Φ^{-1} is the quantile function (inverse of the standard normal CDF).

Lemma S.1.9 (Probit dual of the soft gate). *For every $z \in \mathbb{R}$, the regularized maximization*

$$\max_{\pi\in[0,1]} \big\{\, \pi z \;+\; \mathscr{R}_\Phi(\pi) \,\big\} \tag{S.1.98}$$

admits the unique maximizer $\pi^\star = \Phi(z)$ and optimum value

$$z\,\Phi(z) + \phi(z) \;=\; \mathbb{E}_{\eta\sim\mathcal{N}(0,1)}\big[\max(z+\eta, 0)\big]. \tag{S.1.99}$$

Proof. Using $\phi'(y) = -y\,\phi(y)$ and the inverse-function derivative $(\Phi^{-1})'(\pi) = 1/\phi(\Phi^{-1}(\pi))$,

$$\frac{\mathrm{d}\mathscr{R}_\Phi}{\mathrm{d}\pi} \;=\; \phi'\big(\Phi^{-1}(\pi)\big)\cdot\frac{1}{\phi(\Phi^{-1}(\pi))} \;=\; -\Phi^{-1}(\pi). \tag{S.1.100}$$

The first-order condition $z - \Phi^{-1}(\pi) = 0$ yields $\pi^\star = \Phi(z)$; substituting back and using $\phi(\Phi^{-1}(\Phi(z))) = \phi(z)$ gives the left-hand side of (S.1.99). The right-hand identity is the standard Gaussian smoothing $\mathbb{E}[\max(z+\eta,0)] = z\Phi(z) + \phi(z)$. □

Remark S.1.5 (Optimal policy under perceived payoff noise). The optimum $\pi^\star = \Phi(z_i^{(l)})$ admits an equivalent interpretation as the marginal of an optimal pure rule: the player perceives the continuation payoff at $s_{i,q}^{(l)}$ as the noisy quantity $z_i^{(l)} + \eta$ with $\eta \sim \mathcal{N}(0,1)$ independent across states, and applies

the noiseless-optimal pure rule *"continue iff the perceived payoff is strictly positive"* on this perception. Marginalizing the rule over η at fixed $z_i^{(l)}$ recovers the continue probability,

$$\mathbb{P}_{\eta\sim\mathcal{N}(0,1)}\big(z_i^{(l)}+\eta>0\big) \;=\; \Phi(z_i^{(l)}). \tag{S.1.101}$$

The realized continuation payoff is the deterministic $z_i^{(l)}$ from the actual network forward pass: the noise η lives in the player's perception of the payoff while the environment pays out $z_i^{(l)}$ on continuation, so the expected backward Bellman comparison at $s_{i,q}^{(l)}$ is the soft mass $\Phi(z_i^{(l)})\cdot z_i^{(l)}$ in place of the deterministic $\mathbf{1}[z_i^{(l)}>0]\cdot z_i^{(l)}$.

The regularizer $\mathscr{R}_\Phi$ is symmetric around $\pi=\frac{1}{2}$ and peaks there at $\phi(0)=1/\sqrt{2\pi}$ (where $\Phi^{-1}(\frac{1}{2})=0$), tapering to 0 at the boundary $\pi\in\{0,1\}$. It rewards *indecisiveness* — exactly the role Shannon entropy $\mathscr{H}(\pi)$ plays in the Softplus variant of §S.1.5, where the entropy bonus is the active player's surplus from being allowed to mix. The optimum $\pi^\star=\Phi(z)=\mathbb{E}_{\eta\sim\mathcal{N}(0,1)}[\,\mathbf{1}(z+\eta>0)\,]$ is the hard ReLU gate averaged over a Gaussian shift of its threshold, aligning with the original stochastic-gate motivation for GELU (Hendrycks and Gimpel, 2016). The optimal value (S.1.99) is the Gaussian-smoothed ReLU $z\Phi(z)+\phi(z)$, whose derivative is Φ; it is not $\mathrm{GELU}(z)=z\Phi(z)$, which is not convex and hence not the conjugate of any regularizer.

On ReLU networks the forward pass still computes $\mathrm{ReLU}(z_i^{(l)})$ exactly, so the downstream game values inherited from Theorem S.1.3 are unchanged: the player evaluates the soft gate $\tilde{\Phi}$ on the *current* layer and reuses the original network activations for everything below. There, $\tilde{\Phi}$ is an optional smoothing mode (used in the VGG-16 SG row) without an exactness claim.

GELU networks. On a GELU network, $\tilde{\Phi}$ multiplies the occupation-measure recursion by $\Phi(z_i^{(l)})$ at every layer, which is the chain rule of the gate-conditioned map $a_i^{(l)}=\Phi(\bar{z}_i^{(l)})\,z_i^{(l)}$ with the gate argument $\bar{z}_i^{(l)}$ held at its forward value, the conditioning used for ViTs (Section 5.5.3). It is not the GELU gradient, which needs $\mathrm{GELU}'(z)=\Phi(z)+z\phi(z)$; the substitution $\mathbf{1}[z>0]\to\Phi(z)$ is exact only for the Gaussian-smoothed-ReLU network, as the entropy-regularized gate is for Softplus (Section S.1.5).

S.1.4.4 Fan-in entropy bias λ_{ent}

The fan-in entropy mode lowers the bias, i.e. the continuation payoff (SG) or bias-node payoff (RG), of neurons with diffuse fan-in: $b_j^{(l)}\mapsto b_j^{(l)}-\lambda_{\mathrm{ent}}\tilde{\mathscr{H}}_j^{(l)}$ with $\tilde{\mathscr{H}}_j^{(l)}=-\sum_i p_{ij}\log p_{ij}/\log n_j$, $p_{ij}=|W_{ij}^{(l)}|/\gamma_j^{(l)}$ and n_j the fan-in of neuron j. It depends on the weights only (negligible cost) and changes the forward pass of the explained network.

S.1.5 Softplus Variant of the Stopping Game

We introduce the entropy regularization that Gaubert & Vlassopoulos (2025) apply at activation states to model networks with Softplus instead of ReLU activations and, similarly to the ReLU case, adapt it to the Stopping Decomposition of Section S.1.2.1: the player-specific game values agree with the activations of a Softplus Decomposition, and the gradient of the Softplus network is recovered by a linear combination of occupation measures.

Modified rule: entropy-regularized activation-state policy. The technical tool throughout this section, and re-used by the Routing Game in Section S.1.3 and the attention block of Section 5.5, is the standard Legendre–Fenchel identity for Shannon entropy: for every $\theta > 0$, $n \in \mathbb{N}$, and payoff vector $r \in \mathbb{R}^n$,

$$\max_{\pi \in \Delta_n} \left\{ \sum_{i=1}^{n} \pi(i)\, r_i + \theta\, \mathscr{H}(\pi) \right\} = \theta\, \log \sum_{i=1}^{n} \exp\!\Big(\frac{r_i}{\theta}\Big), \qquad \pi^\star(i) \propto \exp\!\Big(\frac{r_i}{\theta}\Big), \tag{S.1.102}$$

with $\mathscr{H}(\pi) := -\sum_i \pi(i) \log \pi(i)$; the unique maximizer is the Gibbs measure at inverse temperature $1/\theta$. Identity (S.1.102) is the Fenchel conjugate of the (scaled) negative entropy (Boyd and Vandenberghe, 2004).

Only the active player's stop/continue choice changes. At every activation state $s = s_{j,q}^{(l,\mathrm{act})}$, replace the binary maximization (S.1.25) by an entropy-regularized maximization over mixed policies $\pi_q(s) \in \Delta(\{\text{stop,cont}\})$ at temperature θ. With $\tilde{Q}_q^\pi(s,\text{stop}) = \tilde{Q}_{q'}^\pi(s,\text{stop}) = 0$ (cf. (S.1.21)) and $\tilde{Q}_p^\pi(s,\text{cont})$ as in (S.1.22), the soft Bellman recursion for the active-player advantage is

$$V_q^\pi(s) := \max_{\nu \in \Delta(\{\text{stop,cont}\})} \Big\{ \textstyle\sum_a \nu(a) \big(\tilde{Q}_q^\pi(s,a) - \tilde{Q}_{q'}^\pi(s,a) \big) + \theta\, \mathscr{H}(\nu) \Big\}. \tag{S.1.103}$$

Applying (S.1.102) to the binary action set {stop,cont} with payoffs 0 and $\tilde{Q}_q^\pi(s,\text{cont}) - \tilde{Q}_{q'}^\pi(s,\text{cont})$ immediately yields the unique optimizer as the Gibbs policy at inverse temperature $1/\theta$ and the closed-form soft advantage,

$$\pi_q^\star\big(\text{cont} \,\big|\, s\big) = \mathrm{sp}_\theta'\big(\tilde{Q}_q^\pi(s,\text{cont}) - \tilde{Q}_{q'}^\pi(s,\text{cont})\big), \qquad V_q^\pi(s) = \mathrm{sp}_\theta\big(\tilde{Q}_q^\pi(s,\text{cont}) - \tilde{Q}_{q'}^\pi(s,\text{cont})\big), \tag{S.1.104}$$

with $\mathrm{sp}_\theta(z) = \theta \log(1 + e^{z/\theta})$ and $\mathrm{sp}_\theta'(z) = 1/(1 + e^{-z/\theta})$. The transition kernel (S.1.17)–(S.1.18), the payoffs r_p, the Bellman equations (S.1.20)–(S.1.22) for $\tilde{Q}_p^\pi$ and the value recursion (S.1.25) for $\tilde{V}_p^\pi$ are unchanged, except that π is no longer pure and the entropy bonus $\theta \mathscr{H}(\pi_q(s))$ is credited to the active player q.

Theorem S.1.10 (Softplus forward equivalence, `Lean_65`). *The entropy-regularized Stopping Game of (S.1.103) admits an SPNE whose activation-state policy is the Gibbs policy (S.1.104); it is unique on networks without max-pooling nodes, since (S.1.103) is strictly concave (ties at max-pooling states are resolved as in Section S.1.1). Under this equilibrium, at every activation state $s_{j,q}^{(l,\mathrm{act})}$,*

$$\tilde{V}_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) \;=\; a_{\mathrm{soft},j}^{(l,p\oplus q,\,\theta)}, \qquad V_q^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) \;=\; \mathrm{sp}_\theta\big(z_j^{(l)}\big), \tag{S.1.105}$$

where $a_{\mathrm{soft},j}^{(l,\pm,\theta)}$ are the Softplus-Decomposition activations of the Softplus network with activation sp_θ, propagated by (S.1.2)–(S.1.3) from $a_{\mathrm{soft},k}^{(0,\pm,\theta)} = x_k^\pm$ with the decomposition function

$$f_{\mathrm{soft}}^\theta(z^+, z^-) \;:=\; \big(\mathrm{sp}_\theta'(z^+ - z^-)\, z^+ + \theta\, \mathscr{H}_\theta(z^+ - z^-),\; \mathrm{sp}_\theta'(z^+ - z^-)\, z^- \big), \tag{S.1.106}$$

and $\mathscr{H}_\theta(z) := \mathscr{H}(\mathrm{sp}_\theta'(z))$ the binary entropy of the Gibbs weight.

Proof. By backward induction in l, mirroring the proof of Theorem S.1.3, with the same base case $\tilde{V}_p^{\pi^\star}(s_{k,q}^{(0)}) = x_k^{p\oplus q} = a_{\mathrm{soft},k}^{(0,p\oplus q,\theta)}$. For $l \geq 1$, assume (S.1.105) holds at layer $l-1$, so the decomposed activations obey $a_{\mathrm{soft},i}^{(l-1,+,\theta)} - a_{\mathrm{soft},i}^{(l-1,-,\theta)} = a_i^{(l-1)}$. Substituting into (S.1.22), the Q-difference of (S.1.104)

computes to $b_j^{(l)} + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\,\xi_{q,r}\,a_i^{(l-1)} = z_j^{(l)}$, using $\sum_r W_{ij}^{(l,q\oplus r)}\xi_{q,r} = W_{ij}^{(l)}$ exactly as in the proof of Theorem S.1.3. Hence, the Gibbs policy is $\pi_q^\star(\text{cont} \mid \cdot) = \mathrm{sp}'_\theta(z_j^{(l)})$, and (S.1.104) gives $V_q^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}) = \mathrm{sp}_\theta(z_j^{(l)})$, the second identity of (S.1.105).

For the first identity, for the active player ($p = q$, so $p \oplus q = +$), the entropy-regularized optimization (S.1.103) attributes both the routed expectation and the entropy bonus to player q: $\tilde{V}_q^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}) = \mathrm{sp}'_\theta(z_j^{(l)})\,z_j^{(l,+)} + \theta\,\mathscr{H}_\theta(z_j^{(l)}) = a_{\mathrm{soft},j}^{(l,+,\theta)}$, where $\tilde{Q}_q^{\pi^\star}(\cdot, \text{cont}) = z_j^{(l,+)}$ follows from the inductive hypothesis via (S.1.2). For the opponent ($p = q'$, so $p \oplus q = -$), the entropy bonus is absent (it is the active player's surplus from being allowed to mix) and only the routed expectation contributes: $\tilde{V}_{q'}^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}) = \mathrm{sp}'_\theta(z_j^{(l)})\,z_j^{(l,-)} = a_{\mathrm{soft},j}^{(l,-,\theta)}$ by (S.1.3). The two identities together close the induction. □

Gradient recovery on Softplus networks. The discounted occupation measure of Definition S.1.6 is unchanged in form, but the trajectory law Π_x^θ is now generated by the Gibbs policies (S.1.104) in place of the deterministic stop/continue rule.

Theorem S.1.11 (Softplus gradient recovery, `Lean_66`). *Under the SPNE of Theorem S.1.10 on the Softplus network with activation* sp_θ*, the discounted occupation difference recovers the Softplus-network gradient exactly as in Theorem S.1.6:*

$$\frac{\partial a_j^{(m)}}{\partial a_i^{(l)}} \;=\; \xi_{q,+}\Big(\Gamma_x\big(s_{i,+}^{(l,\mathrm{act})} \mid s\big) - \Gamma_x\big(s_{i,-}^{(l,\mathrm{act})} \mid s\big)\Big), \qquad s = s_{j,q}^{(m,\mathrm{act})}, \tag{S.1.107}$$

with Γ_x *now driven by* Π_x^θ.

Proof. Write $D(s_{h,r}^{(m',\mathrm{act})}) := \Gamma_x(s_{i,r}^{(l,\mathrm{act})} \mid s_{h,r}^{(m',\mathrm{act})}) - \Gamma_x(s_{i,r'}^{(l,\mathrm{act})} \mid s_{h,r}^{(m',\mathrm{act})})$, the right-hand side above. The one-step occupation recursion in the proof of Theorem S.1.6 holds with the gate $\mathbf{1}[z_j^{(m)} > 0]$ replaced by the Gibbs continuation probability $\mathrm{sp}'_\theta(z_j^{(m)})$; taking its difference over the two targets gives $D(s_{j,q}^{(m,\mathrm{act})}) = \mathrm{sp}'_\theta(z_j^{(m)})\sum_h \big(W_{hj}^{(m,+)} D(s_{h,q}^{(m-1,\mathrm{act})}) - W_{hj}^{(m,-)} D(s_{h,q'}^{(m-1,\mathrm{act})})\big)$. By induction on $m-l$, $D(s_{h,r}^{(m-1,\mathrm{act})}) = \partial a_h^{(m-1)}/\partial a_i^{(l)}$ for both r, so the right-hand side is $\mathrm{sp}'_\theta(z_j^{(m)})\sum_h W_{hj}^{(m)}\,\partial a_h^{(m-1)}/\partial a_i^{(l)}$, the chain rule at a Softplus gate; the base case $D(s_{i,r}^{(l,\mathrm{act})}) = 1$ and the addition and max-pooling steps are unchanged. Unlike Part 1 of Theorem S.1.6, the individual occupation measures are not derivatives of the decomposed activations (S.1.106), because the soft gate depends on them; only their difference is. □

Probit analog and GELU. Replacing $\theta\mathscr{H}$ in (S.1.103) by the probit regularizer $\mathscr{R}_\Phi(\pi) = \phi(\Phi^{-1}(\pi))$ gives, by Lemma S.1.9, the continuation probability $\Phi(z)$ and the value

$$z\Phi(z) + \phi(z) = \mathbb{E}_{\eta\sim\mathcal{N}(0,1)}[\mathrm{ReLU}(z+\eta)],$$

so the proofs above carry over verbatim with sp_θ replaced by this Gaussian-smoothed ReLU, sp'_θ by its derivative Φ and $\theta\mathscr{H}_\theta(z)$ by $\phi(z)$: the probit-regularized game recovers Gaussian-smoothed-ReLU networks and their exact gradient. GELU, $z\Phi(z)$, is *not* covered: it is not convex, so it is not the value $\max_\pi\{\pi z + \mathscr{R}(\pi)\}$ (a Fenchel conjugate) of any regularized stop/continue choice. Substituting

the gate $\Phi(z)$ into Theorem S.1.6 on a GELU network yields the gradient of the gate-conditioned map $a = \Phi(z^{\text{fwd}})\, z$, not the GELU gradient $\Phi(z) + z\phi(z)$ (Section S.1.4.3).

Relation to the curvature-based defense of Dombrowski et al. Dombrowski et al. (2022) train Softplus networks from scratch as a defense against gradient-explanation manipulation. Their argument bounds the Frobenius norm of the network Hessian: replacing ReLU, whose distributional second derivative $\delta(z)$ leaves $\|\nabla^2 f\|_F$ unbounded, by sp_θ with $|\mathrm{sp}''_\theta(z)| \le 1/(4\theta)$ yields a uniform bound on $\|\nabla^2 f\|_F$, and the gradient theorem then bounds the maximal change of the gradient explanation under input perturbations. Theorems S.1.10–S.1.11 place this Softplus replacement inside our framework as a single change in the game: entropy-regularizing the activation-state stop/continue policy at temperature θ. Whether the Hellinger response of the resulting trajectory law (Section 8.3) yields bounds sharper than, or differently targeted from, the Lipschitz bounds of Dombrowski et al. (2022) is left for future work.

S.2 Extended Attention-Routing Derivations

The target rule, stream masses and zero-mass convention are defined in Section 5.5.1 of the thesis. We retain that notation throughout.

S.2.1 State space

We embed the attention block into the routing-graph state space of §S.1.3 using the tuple-superscript notation $s_{\bullet}^{(\text{att},\,\text{Side},\,\text{type})}$ throughout, where Side $\in \{\text{O}, \text{val}, \text{X}\}$ identifies the activation slot inside the attention block and type $\in \{\text{act}, \text{lin}\}$ matches state types of §S.1.3. The state indices encode the tensor structure of the AV mixing and the Value projection, with $t,k \in \{1,\ldots,S\}$ ranging over output/key tokens (sequence axis), $d \in \{1,\ldots,d_h\}$ over the within-head feature axis of the focused head (i.e. the head index h is fixed and d runs over that head's d_h feature dimensions), and $e \in \{1,\ldots,D\}$ over the per-token input embedding dimensions.

States for the composite $Av{+}W_v$ block. The explanatory walk through the attention block visits three state types beyond the surrounding ReLU-network's; all are parametrized by the player label in turn.

- **Output activation states** $s_{t,d,p}^{(\text{att},\,\text{O},\,\text{act})}$ at the head's output entry $O_{t,d}$, indexed by output token $t \in \{1,\ldots,S\}$, within-head feature dim $d \in \{1,\ldots,d_h\}$, and player $p \in \{+,-\}$. These are the entry points of the explanatory walk: the surrounding linear-layer recursion of §S.1.3.2 delivers occupation mass to $\{s_{t,d,p}^{(\text{att},\,\text{O},\,\text{act})}\}_{t,d,p}$ from the layer behind the head.
- **Output sign-branch (linear) states** $s_{(t,d),p,\kappa}^{(\text{att},\,\text{O},\,\text{lin})}$, indexed by (t,d,p,κ). These mirror the linear routing states $s_{j,q,\kappa}^{(l,\text{lin})}$ of Definition S.1.7 for the AV mixing at output entry (t,d): *this is where the QK logits enter the routing* via the reference $\bar{\pi}_t = A_{t,\cdot}$. The active player picks a key token k by a mixed action $\pi^{\star}_{t,d,\kappa} \in \Delta(\{1,\ldots,S\})$ that combines the AV reference $\bar{\pi}_t$ with the κ-stream Value mass $\tilde{v}^{\kappa}_{k,d}$.
- **Value-projection linear states** $s_{(k,d),p,\kappa}^{(\text{att},\,\text{val},\,\text{lin})}$, indexed by key token k, within-head feature dim d, player p, and sign κ. These are the linear-routing states of the per-token Value projection: the player picks an input dimension e (or the bias node) and the game value satisfies $V_p\big(s_{(k,d),p,\kappa}^{(\text{att},\,\text{val},\,\text{lin})}\big) = \ell(\tilde{v}^{\kappa}_{k,d})$ with $\tilde{v}^{\kappa}_{k,d}$ from (5.17).
- **Input activation states** $s_{(k,e),p}^{(\text{att},\,\text{X},\,\text{act})}$ at the input embedding entry $X_{k,e}$, indexed by token–embedding-dim pair (k,e) and player p. X lives upstream of the Value projection and carries the input-embedding axis e; the within-head feature axis d enters first at the Value projection's output downstream. They are the exit points of the attention block: the surrounding network's recursion resumes from the per-player occupation on $\{s_{(k,e),p}^{(\text{att},\,\text{X},\,\text{act})}\}_{(k,e),p}$.

Transitions. The trajectory through the attention block has three steps; all transitions inside the block carry trajectory discount 1 except the sign-oracle split, which uses the same $\{2\alpha, 2\beta\}$ as a ReLU activation state of Definition S.1.7. There is only *one* sign-oracle split per output entry (t,d), since $A_{t,k} \geq 0$ absorbs the AV mixing into the W_v sign split.

1) *Sign-oracle split (output activation → output sign-branch, fixed (t,d)).* At $s^{(\mathrm{att},\mathrm{O},\mathrm{act})}_{t,d,p}$ an unobserved Oracle transitions uniformly to $s^{(\mathrm{att},\mathrm{O},\mathrm{lin})}_{(t,d),p,+}$ (player p retains the turn, trajectory discount 2α) or to $s^{(\mathrm{att},\mathrm{O},\mathrm{lin})}_{(t,d),p',-}$ (turn switches to opponent p', discount 2β), each with probability $\frac{1}{2}$. The feature index d is preserved.
2) *Value-routing policy (output sign-branch → Value-projection linear).* At $s^{(\mathrm{att},\mathrm{O},\mathrm{lin})}_{(t,d),p,\kappa}$ the active player picks a key token k by the mixed action

$$\pi^{\star}_{t,d,\kappa}(k) \;=\; \frac{A_{t,k}\,\tilde{v}^{\kappa}_{k,d}}{Z^{\kappa}_{t,d}}, \tag{S.2.1}$$

derived in §S.2.3 as the equilibrium of a KL-regularized log-payoff problem against the reference $\bar{\pi}_t = A_{t,\cdot}$. The trajectory transitions to $s^{(\mathrm{att},\mathrm{val},\mathrm{lin})}_{(k,d),p,\kappa}$ *with the player label, the sign κ, and the within-head feature dim d preserved*; the only thing changing is the sequence-axis index from output token t to key token k. The trajectory-discount on this transition is 1.
3) *Value-projection routing (Value-projection linear → input activation).* At $s^{(\mathrm{att},\mathrm{val},\mathrm{lin})}_{(k,d),p,\kappa}$ the active player picks an input dim e by the standard linear-state Gibbs policy of Definition S.1.7 on the κ-stream masses $[X_{k,e}W_{v,e,d}]^{\kappa}$ (the bias node receives $[b_{v,d}]^{\kappa}/\tilde{v}^{\kappa}_{k,d}$ and is absorbed):

$$\pi^{\star}_{\mathrm{val},k,d,\kappa}(e) \;=\; \frac{[X_{k,e}W_{v,e,d}]^{\kappa}}{\tilde{v}^{\kappa}_{k,d}}. \tag{S.2.2}$$

The trajectory transitions to $s^{(\mathrm{att},\mathrm{X},\mathrm{act})}_{(k,e),p}$ with the player label preserved and trajectory-discount factor 1.

The discounted occupation measure $\Gamma_x(\,\cdot\,)$ of §S.1.3.2 is evaluated at all four state types along the explanatory walk. The recovery theorem of §S.2.4 then compares the per-input-neuron player-difference of Γ_x at the input activation states with the target rule (5.21).

S.2.2 Legendre–Fenchel transform of the KL divergence

The Value-routing equilibrium below is a direct corollary of the convex-conjugate identity for the KL divergence; we record it as a stand-alone Lemma to make the inheritance explicit.

Lemma S.2.1 (Legendre–Fenchel transform of KL divergence, `Lean_78`**).** *Let $\mathscr{X}$ be a finite set, $\bar{\pi} \in \Delta(\mathscr{X})$ a reference distribution with full support on $\mathscr{X}$, and $r : \mathscr{X} \to \mathbb{R}$ a real-valued payoff. The KL-regularized linear maximization*

$$F(r;\bar{\pi}) \;:=\; \max_{\pi\in\Delta(\mathscr{X})} \Big\{ \mathbb{E}_{\pi}[r] \;-\; \mathrm{KL}(\pi \,\|\, \bar{\pi}) \Big\} \tag{S.2.3}$$

has the following optimizer and optimum:

$$\pi^{\star}_{r}(x) \;=\; \frac{\bar{\pi}(x)\,\exp\big(r(x)\big)}{Z(r;\bar{\pi})}, \qquad F(r;\bar{\pi}) \;=\; \log Z(r;\bar{\pi}) \;=\; \log \sum_{x\in\mathscr{X}} \bar{\pi}(x)\,\exp\big(r(x)\big). \tag{S.2.4}$$

Two specializations are central below. When $r(x) = \ell(u(x))$ for some non-negative $u : \mathscr{X} \to \mathbb{R}_{\geq 0}$ (so $\exp(r(x)) = u(x)$ with the convention $\exp(-\infty) := 0$), the value collapses to the $\bar{\pi}$-weighted linear

sum and the optimizer becomes the $\bar{\pi}$-tilted normalization of u:

$$F\big(\ell(u);\,\bar{\pi}\big) \;=\; \ell\Big(\sum_{x}\bar{\pi}(x)\,u(x)\Big), \qquad \pi^{\star}_{\ell(u)}(x) \;=\; \frac{\bar{\pi}(x)\,u(x)}{\sum_{x'}\bar{\pi}(x')\,u(x')}. \tag{S.2.5}$$

When $\bar{\pi}$ is uniform on $\mathscr{X}$, $\mathrm{KL}(\pi\,\|\,\mathrm{Unif}_{\mathscr{X}}) = \log|\mathscr{X}| - \mathscr{H}(\pi)$, so subtracting the KL penalty in (S.2.3) flips into a $+\mathscr{H}$ entropy bonus:

$$\mathbb{E}_{\pi}[r] \;-\; \mathrm{KL}\big(\pi\,\|\,\mathrm{Unif}_{\mathscr{X}}\big) \;=\; \mathbb{E}_{\pi}[r] \;+\; \mathscr{H}(\pi) \;-\; \log|\mathscr{X}|. \tag{S.2.6}$$

Up to the additive constant $-\log|\mathscr{X}|$, (S.2.3) therefore reduces to the entropy-regularized maximization of (S.1.56), with value $\log\sum_x \exp(r(x))$ and optimizer $\pi^{\star}(x) \propto \exp(r(x))$.

Proof. This is the Gibbs variational principle (Boyd and Vandenberghe, 2004): for every π, $\mathbb{E}_{\pi}[r] - \mathrm{KL}(\pi\,\|\,\bar{\pi}) = \log Z(r;\bar{\pi}) - \mathrm{KL}(\pi\,\|\,\pi^{\star}_r)$, which is maximal, with value $\log Z(r;\bar{\pi})$, exactly at $\pi = \pi^{\star}_r$. □

S.2.3 Value-routing equilibrium

Fix an output entry (t,d), a sign $\kappa \in \{+,-\}$, and the reference $\bar{\pi}_t = A_{t,\cdot}$ from §S.2.1 (the *same* $\bar{\pi}_t$ for every d, since the AV mixing reuses one row across all feature dims). At the output sign-branch state $s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}$ the active player p solves the KL-regularized log-payoff maximization

$$V_p\big(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}\big) \;\coloneqq\; \max_{\pi\in\Delta(\{1,\dots,S\})}\left\{\sum_{k=1}^{S}\pi(k)\,\ell\big(\tilde{v}^{\kappa}_{k,d}\big) \;-\; \mathrm{KL}\big(\pi\,\|\,\bar{\pi}_t\big)\right\}, \tag{S.2.7}$$

where $\tilde{v}^{\kappa}_{k,d} \geq 0$ is the κ-stream split of the Value entry $v_{k,d}$ from (5.17) and ℓ is the extended logarithm of §S.1.3. The payoff $\ell(\tilde{v}^{\kappa}_{k,d})$ on action k is the active-player value of the predecessor Value-projection linear state, $V_p(s^{(\mathrm{att},\,\mathrm{val},\,\mathrm{lin})}_{(k,d),p,\kappa}) = \ell(\tilde{v}^{\kappa}_{k,d})$, so combining with the AV log-weight $\ell(A_{t,k})$ gives composite payoff $\ell(A_{t,k}\,\tilde{v}^{\kappa}_{k,d})$ at the (t,d)-output. Define the per-output-entry $\bar{\pi}_t$-weighted Value mass

$$Z^{\kappa}_{t,d} \;\coloneqq\; \sum_{m=1}^{S}\bar{\pi}_t(m)\,\tilde{v}^{\kappa}_{m,d}. \tag{S.2.8}$$

The dependence on d enters only through $\tilde{v}^{\kappa}_{m,d}$; the AV reference $\bar{\pi}_t$ is shared across all feature dims.

Proposition S.2.2 (Value-side optimal policy, `Lean_79`). *If $Z^{\kappa}_{t,d} > 0$, the unique optimizer of (S.2.7) is*

$$\pi^{\star}_{t,d,\kappa}(k) \;=\; \frac{\bar{\pi}_t(k)\,\tilde{v}^{\kappa}_{k,d}}{Z^{\kappa}_{t,d}}, \tag{S.2.9}$$

with active-player value $V_p(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}) = \ell(Z^{\kappa}_{t,d})$ and inactive-player value $V_{p'}(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}) = -\infty$ as in Definition S.1.7. If $Z^{\kappa}_{t,d} = 0$, no key contributes mass on stream κ and $V_p(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}) = -\infty$. All key policies then have value $-\infty$; there is no unique maximizing key distribution. We use the cemetery completion stated above.

Proof. Apply the logit specialization (S.2.5) of Lemma S.2.1 with $\mathscr{X} = \{1,\dots,S\}$, $\bar{\pi} = \bar{\pi}_t$, and $u(k) = \tilde{v}^{\kappa}_{k,d}$, so that the payoff $r(k) = \ell(\tilde{v}^{\kappa}_{k,d})$ has $\exp r(k) = \tilde{v}^{\kappa}_{k,d}$. Equation (S.2.5) immediately gives the optimizer $\pi^{\star}_{t,d,\kappa}$ of (S.2.9) and value $\ell(Z^{\kappa}_{t,d})$ when $Z^{\kappa}_{t,d} > 0$. The inactive-player value $-\infty$ is the Routing-Game convention of Definition S.1.7, inherited from the $r = q'$ branch of (S.1.59). If $Z^{\kappa}_{t,d} = 0$ then $\bar{\pi}_t(k)\,\tilde{v}^{\kappa}_{k,d} = 0$ for every k and the supremum of (S.2.7) over the simplex is $-\infty$, consistent with $\ell(0) = -\infty$. □

Specialization to the softmax reference. Setting $\bar{\pi}_t = A_{t,\cdot}$ in (S.2.9) yields the closed-form per-(t,d,κ) policies used in the RG's backward walk through the attention block,

$$\pi^{\star}_{t,d,+}(k) \;=\; \frac{A_{t,k}\,\tilde{v}^{+}_{k,d}}{Z^{+}_{t,d}}, \qquad \pi^{\star}_{t,d,-}(k) \;=\; \frac{A_{t,k}\,\tilde{v}^{-}_{k,d}}{Z^{-}_{t,d}}, \tag{S.2.10}$$

when $Z^{\kappa}_{t,d} > 0$. When $Z^{\kappa}_{t,d} = 0$, set all these key probabilities to zero and the cemetery probability to one. Here $Z^{\kappa}_{t,d} = \sum_m A_{t,m}\,\tilde{v}^{\kappa}_{m,d}$ is defined in (S.2.8). The numerical specification $\bar{\pi}_t = A_{t,\cdot}$ does not require the oracle interpretation of §5.5.2; it is enough that the cached softmax row is treated as a fixed input to (S.2.7).

S.2.4 $\alpha\beta$-LRP recovery for the attention block

We now show that, under the trajectory through the four state types of §S.2.1 and the trajectory discount of (S.1.83), the player-difference of Γ_x at the *input activation states* (the predecessors of the Value-projection linear states, where the W_v weights actually fire) recovers exactly the target rule (5.21) of §5.5.1. We keep the game side and the attribution side strictly separate:

- the *game-side* object is the discounted occupation measure $\Gamma_x(\,\cdot\,)$ of §S.1.3.2 evaluated at the four attention-block state types of §S.2.1. Following the per-neuron difference convention (S.1.82) of the surrounding Routing-Game recursion, we form the per-output-entry difference $\Gamma^{O}_{t,d}$ at the entry point (defined in (S.2.13) below) and the per-input-neuron difference at the exit

$$\Gamma^{X}_{k,e} \;\coloneqq\; \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}_{(k,e),+}\big) - \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}_{(k,e),-}\big). \tag{S.2.11}$$

- the *attribution-side* object is the standard $\alpha\beta$-LRP relevance vector $R^{X}_{k,e}$ given by the input-level target rule (5.21), seeded by $R^{O}_{t,d}$ from the surrounding linear-layer recursion above the head.

Theorem S.2.3 ($\alpha\beta$-LRP recovery for attention, `Lean_80`). *For signed inputs X, under the SPNE of §S.2.1 with reference $\bar{\pi}_t = A_{t,\cdot}$ and the trajectory discount of (S.1.83) restricted to the (sole) sign-oracle split at $s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}$, the per-input-neuron player-difference $\Gamma^{X}_{k,e}$ from (S.2.11) satisfies*

$$\Gamma^{X}_{k,e} \;=\; \sum_{t,d}\left(\alpha\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{+}}{Z^{+}_{t,d}} \;-\; \beta\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{-}}{Z^{-}_{t,d}}\right)\Gamma^{O}_{t,d}, \tag{S.2.12}$$

where the per-output-entry input difference is

$$\Gamma^{O}_{t,d} \;\coloneqq\; \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}_{t,d,+}\big) \;-\; \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}_{t,d,-}\big), \tag{S.2.13}$$

with $\Gamma^{O}_{t,d} = R^{O}_{t,d}$ by inductive hypothesis from the surrounding recursion, for the unit seed used by the implementation. Any real seed r_u multiplies the occupation difference by r_u, including the signed

affine-logit seed of Section 4.5. The right-hand side is exactly the target rule (5.21), hence $\Gamma^X_{k,e} = R^X_{k,e}$. *Aggregating over* e *and the bias node at fixed* (k,d) *collects the relevance routed through the Value entry* $v_{k,d}$*:*

$$\Gamma^{\mathrm{val}}_{k,d} \;=\; \sum_t \left(\alpha \, \frac{A_{t,k}\, \tilde{v}^+_{k,d}}{Z^+_{t,d}} \;-\; \beta \, \frac{A_{t,k}\, \tilde{v}^-_{k,d}}{Z^-_{t,d}} \right) \Gamma^O_{t,d} \;=\; R^{\mathrm{val}}_{k,d}, \tag{S.2.14}$$

recovering the per-Value-entry formula (5.19).

Proof. The statement is block-local: by hypothesis the surrounding recursion delivers the output-side per-entry differences $\Gamma^O_{t,d} = R^O_{t,d}$ at every output position (t,d).

Per-state recursion at the input activation states. Fix an input embedding (k,e) and a player label $r \in \{+,-\}$. A trajectory reaches $s^{(\mathrm{att,\,X,\,act})}_{(k,e),r}$ from an output activation $s^{(\mathrm{att,\,O,\,act})}_{t,d,p}$ in three stages, with all transitions inside the attention block carrying their discounts and player-label switches as in the kernel of §S.2.1. The path goes through one specific within-head feature dim $d \in \{1,\dots,d_h\}$ (an index along the focused head's d_h-wide feature axis): the sign-oracle is per (t,d), the Value-routing policy $\pi^\star_{t,d,\kappa}$ depends on d via $\tilde{v}^\kappa_{k,d}$, and the Value-projection routing $\pi^\star_{\mathrm{val},k,d,\kappa}$ also depends on d. The d index is dropped at the final transition into X, where the input embedding $X_{k,e}$ has indices (k,e) and the within-head feature axis d enters downstream at the Value projection's output.

– $\kappa = +$ *branch (per* (t,d)*).* Sign-oracle takes $s^{(\mathrm{O,act})}_{t,d,p} \to s^{(\mathrm{O,lin})}_{(t,d),p,+}$ with probability $\frac{1}{2}$ and trajectory discount 2α (player retained). The Value-routing policy $\pi^\star_{t,d,+}(k)$ takes the trajectory to $s^{(\mathrm{val,lin})}_{(k,d),p,+}$ (player, sign, d preserved, discount 1). The Value-projection routing $\pi^\star_{\mathrm{val},k,d,+}(e)$ takes the trajectory to $s^{(\mathrm{X,act})}_{(k,e),p}$ (player preserved, d dropped, discount 1). The composite per-(t,d) contribution to $\Gamma_x(s^{(\mathrm{X,act})}_{(k,e),r})$ is

$$\tfrac{1}{2}(2\alpha)\, \pi^\star_{t,d,+}(k)\, \pi^\star_{\mathrm{val},k,d,+}(e)\, \Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big) \;=\; \alpha \, \frac{A_{t,k}\, \tilde{v}^+_{k,d}}{Z^+_{t,d}} \, \frac{[X_{k,e} W_{v,e,d}]^+}{\tilde{v}^+_{k,d}} \, \Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big) \tag{S.2.15}$$

$$\;=\; \alpha \, \frac{A_{t,k}\, [X_{k,e} W_{v,e,d}]^+}{Z^+_{t,d}} \, \Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big), \tag{S.2.16}$$

where cancellation applies when $\tilde{v}^+_{k,d} > 0$. If this mass or $Z^+_{t,d}$ is zero, the ordinary-successor contribution on both sides is zero by (5.18).

– $\kappa = -$ *branch (per* (t,d)*).* Sign-oracle takes $s^{(\mathrm{O,act})}_{t,d,p} \to s^{(\mathrm{O,lin})}_{(t,d),p',-}$ with probability $\frac{1}{2}$, discount 2β, turn switched to opponent p'; the rest of the path preserves p', ending at $s^{(\mathrm{X,act})}_{(k,e),p'}$. The contribution to $\Gamma_x(s^{(\mathrm{X,act})}_{(k,e),r})$ comes from sources with player $p = r'$ (so the switch yields r):

$$\tfrac{1}{2}(2\beta)\, \pi^\star_{t,d,-}(k)\, \pi^\star_{\mathrm{val},k,d,-}(e)\, \Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r'}\big) \;=\; \beta \, \frac{A_{t,k}\, [X_{k,e} W_{v,e,d}]^-}{Z^-_{t,d}} \, \Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r'}\big). \tag{S.2.17}$$

Summing over (t,d) and over the two branches,

$$\Gamma_x\big(s^{(\mathrm{att,\,X,\,act})}_{(k,e),r}\big) \;=\; \sum_{t,d} \Big[\alpha \, \frac{A_{t,k}\, [X_{k,e} W_{v,e,d}]^+}{Z^+_{t,d}} \, \Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big) \;+\; \beta \, \frac{A_{t,k}\, [X_{k,e} W_{v,e,d}]^-}{Z^-_{t,d}} \, \Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r'}\big) \Big], \tag{S.2.18}$$

where the $\frac{1}{2}$ from the sign-oracle splits cancels against the 2 in $2\alpha, 2\beta$.

Player difference. Differencing (S.2.18) for $r{=}+$ minus $r{=}-$ uses $\Gamma_x(s^{(\mathrm{O,act})}_{t,d,+}) - \Gamma_x(s^{(\mathrm{O,act})}_{t,d,-}) = \Gamma^O_{t,d}$ in the $+$ branch (player kept) and the opposite $-\Gamma^O_{t,d}$ in the $-$ branch (player switched):

$$\Gamma^X_{k,e} \;=\; \alpha \sum_{t,d} \frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^+}{Z^+_{t,d}}\,\Gamma^O_{t,d} \;-\; \beta \sum_{t,d} \frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^-}{Z^-_{t,d}}\,\Gamma^O_{t,d}, \tag{S.2.19}$$

which is (S.2.12).

Target-rule identification. The right-hand side matches (5.21) verbatim. Since $\Gamma^O_{t,d} = R^O_{t,d}$, $\Gamma^X_{k,e} = R^X_{k,e}$. Aggregating over e and the bias node at fixed (k,d) uses $\sum_e [X_{k,e}W_{v,e,d}]^\kappa + [b_{v,d}]^\kappa = \tilde{v}^\kappa_{k,d}$, giving (S.2.14), which matches (5.19). □

S.2.5 Risk-Averse Attention via ADF

When ADF variance propagation is active, let s_{tk} denote the uncertainty score attached to attending from query token t to key token k (in the current implementation, $s_{tk} = \hat{\sigma}_k$, the square root of the feature-averaged ADF variance of token k's query projection; it depends only on the key token and is a proxy for, not the standard deviation of, e_{tk}). Risk-averse attention shifts the QK-logits in the direction of reduced uncertainty,

$$\tilde{e}_{tk} = e_{tk} + \lambda_{\mathrm{sm}}\, s_{tk}, \qquad \lambda_{\mathrm{sm}} \le 0, \tag{S.2.20}$$

yielding the risk-modified policy

$$\tilde{\pi}_t(k) = \frac{\exp(\tilde{e}_{tk})}{\sum_{k'} \exp(\tilde{e}_{tk'})}. \tag{S.2.21}$$

The sign convention $\lambda_{\mathrm{sm}} \le 0$ is used throughout the paper and in all tables: more-negative values penalize high-uncertainty keys more strongly, so the shift always redirects relevance toward tokens whose QK-interactions are stable under input noise. The shift is computed in the AV-backward hook from cached pre-softmax logits, so the forward pass and model predictions remain unchanged.

The naming *softmax-shift risk aversion* is a deliberate analog of the activation-side risk aversion λ but acts in a different direction: λ is risk-averse on the value side (it routes through stable activations by lowering mean$-|\lambda|\sigma$), whereas $\lambda_{\mathrm{sm}} < 0$ exponentially tilts the routing policy, $\tilde{\pi}_t(k) \propto A_{t,k} \exp(\lambda_{\mathrm{sm}} s_{tk})$, toward *low*-uncertainty keys: $\sum_k \tilde{\pi}_t(k)\, s_{tk}$ is non-decreasing in λ_{sm}, while the entropy of $\tilde{\pi}_t$ may rise or fall. We retain the "risk-averse" label because what is being avoided is uncertainty in the routed pathway, but the reader should note that the shift acts on the routing policy, not on the player's utility curvature.

S.3 Extended Trajectory Derivations

This chapter provides the formal foundations and proofs for Chapter 8.

S.3.1 Layered Markov Process Foundations

We collect here the layered-Markov-process abstraction that underlies every Hellinger statement in this section: a finite layered graph augmented with a cemetery †, and the act-state *Markov Process*s (MPs) that the two games of this chapter instantiate on it. The natural RG carries an extra linear-routing state per neuron between consecutive activation states. We retain its gate, branch, and routing substeps for full-law diagnostics; only ordinary surviving paths admit exact act-only elimination, as detailed in Remark S.3.1. For full routing-process dynamic programming below, the layer index includes these refined substeps; activation-boundary readouts form a subset of those layers.

S.3.1.1 Layered Markov Process with Cemetery

State graph. Fix an integer $L \geq 1$. For each $l \in \{0, 1, \ldots, L\}$ let $\mathscr{S}^{(l)}$ be a finite set of *ordinary states* at layer l. Adjoin a single absorbing *cemetery state* † shared across all layers and write $\mathscr{S}_+^{(l)} := \mathscr{S}^{(l)} \cup \{\dagger\}$. The full state graph is the disjoint union $\bigsqcup_l \mathscr{S}_+^{(l)}$, and transitions are only permitted from layer l to layer $l-1$ (or into the cemetery, which then absorbs the trajectory forever). The states at layer 0, the input layer, are referred to as *pixels* below; the per-pixel decomposition will sum exactly over $\mathscr{S}_+^{(0)}$.

Transition kernel. For each layer $l \geq 1$, a *backward transition kernel* $T^{(l)} : \mathscr{S}_+^{(l)} \to \Delta\big(\mathscr{S}_+^{(l-1)}\big)$ assigns to each state $s \in \mathscr{S}_+^{(l)}$ a probability distribution $T^{(l)}(s \to \cdot)$ over $\mathscr{S}_+^{(l-1)}$, with the constraint that the cemetery is absorbing:

$$T^{(l)}(\dagger \to \dagger) = 1, \qquad T^{(l)}(\dagger \to s') = 0 \ \text{ for } \ s' \in \mathscr{S}^{(l-1)}. \tag{S.3.1}$$

Ordinary states may transition either to the next layer's ordinary states or into the cemetery (early termination). We write $T^{(l)}(s \to \dagger)$ for the probability that an ordinary state at layer l jumps to the cemetery rather than to a layer-$(l-1)$ ordinary state.

Initial distribution and trajectories. Fix a deterministic starting state $s_0 \in \mathscr{S}^{(L)}$. A *trajectory* (indexed by layer, $\zeta_l \in \mathscr{S}_+^{(l)}$) is a sequence

$$\zeta = (\zeta_L, \zeta_{L-1}, \ldots, \zeta_0) \ \in \ \mathscr{S}_+^{(L)} \times \mathscr{S}_+^{(L-1)} \times \cdots \times \mathscr{S}_+^{(0)}, \tag{S.3.2}$$

and the kernel $T^{(\cdot)}$ together with δ_{s_0} at layer L induces a probability distribution on trajectories via

$$\Pi(\zeta_L, \zeta_{L-1}, \ldots, \zeta_0) \ = \ \mathbf{1}\{\zeta_L = s_0\} \prod_{k=1}^{L} T^{(k)}(\zeta_k \to \zeta_{k-1}). \tag{S.3.3}$$

For any $l \in \{0, \ldots, L\}$ the prefix distribution $\Pi^{(l)}$ over $(\zeta_L, \ldots, \zeta_l)$ is the marginal of Π over the trailing coordinates.

Two parallel processes. We compare two such processes that share the state graph and the start state s_0, but use different kernels: $M = A$ with $T_A^{(l)}, \Pi_A^{(l)}$, and $M = B$ with $T_B^{(l)}, \Pi_B^{(l)}$. The kernels may differ either because the underlying network weights differ (e.g. randomization), the activations differ (e.g. different inputs), or both.

S.3.1.2 How the Two Games Fit This Abstraction

We instantiate the layered MP of Section S.3.1.1 for both games in this subsection. State labels $s_{i,p,\kappa}^{(l,\text{type})}$ follow Appendices S.1.2 and S.1.3. We evaluate the SPNE policies of Theorems S.1.3 (SG) and S.1.7 (RG) at $\tau = 1$ (so activation-state decisions are deterministic, continue iff $z_j^{(l)} > 0$); for the RG we further fix $\varepsilon = 0$.

SG MP. Ordinary states at layers $l \in \{1, \dots, L\}$ are activation states $s_{i,p}^{(l,\text{act})}$, and layer-0 ordinary states are pixel-player terminal pairs $s_{j,q}^{(0)}$ (each pixel splits into two terminals, mirroring the player tag the trajectory carries to it). The deterministic start state is $s_0 = s_{u,+}^{(L,\text{act})}$. The SPNE backward kernel keeps the player on positive weights and flips it on negative weights:

$$
\begin{aligned}
T_M^{(l)}\big(s_{j,p}^{(l,\text{act})} \to s_{i,p}^{(l-1,\text{act})}\big) &= \frac{W_M^{(l,+)}[i,j]}{\gamma_{M,j}^{(l)}}\,\mathbf{1}\{z_j^{(l)} > 0\},\\
T_M^{(l)}\big(s_{j,p}^{(l,\text{act})} \to s_{i,p'}^{(l-1,\text{act})}\big) &= \frac{W_M^{(l,-)}[i,j]}{\gamma_{M,j}^{(l)}}\,\mathbf{1}\{z_j^{(l)} > 0\}, \qquad 2 \le l \le L,
\end{aligned}
\tag{S.3.4}
$$

with $\gamma_{M,j}^{(l)} = \sum_i |W_M^{(l)}[i,j]|$, $p' = -p$, cemetery row $T_M^{(l)}(s_{j,p}^{(l,\text{act})} \to \dagger) = 1 - \mathbf{1}\{z_j^{(l)} > 0\}\mathbf{1}\{\gamma_{M,j}^{(l)} > 0\}$, with $0/0 \coloneqq 0$ in the ordinary transitions, and the analogous form at $l = 1$ landing on the pixel-player terminals $s_{i,p}^{(0)}, s_{i,p'}^{(0)}$. This is the natural SG MP of Section S.1.2 verbatim.

RG MP. Ordinary states at layers $l \in \{1, \dots, L\}$ are activation states $s_{i,p}^{(l,\text{act})}$, and layer-0 ordinary states are pixel-stream terminal pairs $s_{j,\kappa}^{(0)}$ (each pixel splits into two terminals tagged by the layer-1 routing-edge stream). The deterministic start state is $s_0 = s_{u,+}^{(L,\text{act})}$. The kernel composes the activation-state stop/continue decision and the subsequent κ-branched Gibbs routing into one act-to-act transition:

$$
\begin{aligned}
T_M^{(l)}\big(s_{j,p}^{(l,\text{act})} \to s_{i,p}^{(l-1,\text{act})}\big) &= \tfrac{1}{2}\,\pi_{M,+}^{(l)}(i \mid j)\,\mathbf{1}\{z_j^{(l)} > 0\},\\
T_M^{(l)}\big(s_{j,p}^{(l,\text{act})} \to s_{i,p'}^{(l-1,\text{act})}\big) &= \tfrac{1}{2}\,\pi_{M,-}^{(l)}(i \mid j)\,\mathbf{1}\{z_j^{(l)} > 0\}, \qquad 2 \le l \le L,
\end{aligned}
\tag{S.3.5}
$$

and at $l = 1$ the destinations are pixel-stream terminals carrying the routing-edge κ,

$$
T_M^{(1)}\big(s_{j,p}^{(1,\text{act})} \to s_{i,\kappa}^{(0)}\big) \;=\; \tfrac{1}{2}\,\pi_{M,\kappa}^{(1)}(i \mid j)\,\mathbf{1}\{z_j^{(1)} > 0\}, \qquad \kappa \in \{+,-\},
\tag{S.3.6}
$$

For this act-only marginal, the cemetery probability is the residual

$$
T_M^{(l)}(s_{j,p} \to \dagger) = 1 - \sum_{s' \in \mathscr{S}^{(l-1)}} T_M^{(l)}(s_{j,p} \to s').
$$

An empty routing stream has zero ordinary policy mass and its 1/2 branch share is absorbed, even when the source neuron is alive. The Gibbs policy on a nonempty stream is the temperature-τ distribution

$$\pi^{(l)}_{M,\kappa}(i \mid j) \;\propto\; \big[\, a^{(l-1)}_{M,i}\, W^{(l,\kappa)}_{M,ij} \,\big]^{1/\tau}, \qquad i \in \mathrm{Pred}^{(l)}_j, \tag{S.3.7}$$

(at $l = 1$, $a^{(0)}_{M,i}\, W^{(1,\kappa)}_{M,ij}$ is replaced by $[x_i\, W^{(1)}_{M,ij}]^{\kappa}$; when $\varepsilon > 0$, each continuing κ branch sends its $\frac{1}{2}\varepsilon^{1/\tau}/Z^{(l)}_{j,\kappa}$ share to the cemetery; as in the implementation, the bias node of Section S.1.3 is omitted). If all scores in a stream are $-\infty$, its ordinary policy is zero and the whole branch is absorbed, in agreement with the outside-option convention. Equations (S.3.5)–(S.3.6) are the act-to-act marginal of the *diagnostic* routing process: it uses the original activation gates, omits the bias action, and normalizes each remaining routing branch. When the original bias mass is nonzero, this differs from the bias-inclusive game of Section S.1.3. For example, $a = W = b = 1$, $\varepsilon = 0$, gives probability 1/4 on the ordinary positive route in that game, but 1/2 after omitting the bias action. Remark S.3.1 concerns state elimination within the same chosen process; it does not identify these two different kernels. The reported full-trajectory diagnostics use the refined gate, branch, and routing substeps: absorption at different substeps is retained in the trajectory history. The ordinary surviving-path overlap can be computed from the displayed act-to-act kernels, but their collapsed cemetery row must not be substituted for the refined process when computing the full overlap.

S.3.1.3 Marginal Occupation Measures

For each model $M \in \{A,B\}$, the marginal probability that the chain visits state s at layer l is

$$m^{(l)}_M(s) \;=\; \sum_{\zeta:\zeta_l=s} \Pi^{(l)}_M(\zeta), \qquad s \in \mathscr{S}^{(l)}_+. \tag{S.3.8}$$

This satisfies the deterministic boundary condition

$$m^{(L)}_M(s_0) = 1, \qquad m^{(L)}_M(s) = 0 \;\text{ for }\; s \neq s_0, \tag{S.3.9}$$

and the one-step recursion

$$m^{(l-1)}_M(s') \;=\; \sum_{s \in \mathscr{S}^{(l)}_+} m^{(l)}_M(s)\, T^{(l)}_M(s \to s'), \qquad s' \in \mathscr{S}^{(l-1)}_+. \tag{S.3.10}$$

We refer to the single backward propagation (S.3.10) as the *Marginal Pass*. At the input layer, $m^{(0)}_M(s)$ is the probability that a backward trajectory from s_0 terminates at pixel s under model M, and $m^{(0)}_M(\dagger)$ is the total mass absorbed by the cemetery along the way.

S.3.2 Hellinger Distance from Bhattacharyya Coefficients

Properties. $\mathrm{BC}^{(l)}$ and $\mathrm{H}^{(l)}$ lie in [0,1] for every l, even under support mismatch. If a state is alive in one model and dead in the other, its geometric-mean row vanishes and its mass contributes directly to the Hellinger increment. Since H is a metric and satisfies $\mathrm{H}^2 \le d_{\mathrm{TV}} \le \sqrt{2}\,\mathrm{H}$ and $\mathrm{H}^2 \le \frac{1}{2}\mathrm{KL}$, small Hellinger distance controls TV and small KL controls H as well. *Normalization conventions.* Here $d_{\mathrm{TV}}(\Pi_A, \Pi_B) = \frac{1}{2}\sum_\zeta |\Pi_A(\zeta) - \Pi_B(\zeta)| \in [0,1]$ is the probability-metric (half-L^1) total-variation distance, and KL is measured in nats; the displayed inequalities follow this convention. Readers using the unnormalized L^1 total variation $\|\Pi_A - \Pi_B\|_1 = 2d_{\mathrm{TV}}$ should absorb the factor of 2 accordingly.

Alternative divergences. KL and Jeffreys divergence have an exact Markov-chain chain rule but are unbounded and become infinite under support mismatch, e.g. when randomization opens or closes routes. Jensen–Shannon is bounded and finite, but $\frac{1}{2}(\Pi_A + \Pi_B)$ is not Markov on the layered graph, so no exact linear-time dynamic program is available. Wasserstein distances need a ground cost between neuron-labeled trajectories (across layers, signs and permuted neurons), for which there is no canonical choice, and the resulting transport problem has no exact local recursion. Hellinger is bounded, finite, and exactly tractable: the overlap factorizes along the layered law (Lemma S.3.1), which gives a dynamic program linear in the number of edges (Algorithm 8.1) and the non-negative per-state decomposition of Theorem 8.1. Note that H compares the *undiscounted* law Π: the discount d_t does not enter the kernels, so H is blind to the path weights (α,β) and to the SG weight magnitudes $\gamma_j^{(l)}$, which enter only through d_t; it compares *where* trajectories go, not how much weight they carry.

Computation. Compute the marginal occupations $m_M^{(l)}$, run the Hellinger pass (Algorithm 8.1), and read off $\mathrm{H}^2(s)$ from (8.5); the total cost is linear in the number of edges.

S.3.3 Hellinger Identities and Proofs

S.3.3.1 Multiplicative Chain Rule

Lemma S.3.1 (Chain rule, `Lean_81`). *For every $l \in \{0,1,\dots,L\}$, the Bhattacharyya coefficient of the prefix distributions factorizes along trajectories:*

$$\mathrm{BC}\big(\Pi_A^{(l)}, \Pi_B^{(l)}\big) \;=\; \sum_{(\zeta_L,\dots,\zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1}). \tag{S.3.11}$$

Proof. Substitute the Markov factorization $\Pi_M^{(l)}(\zeta) = \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} T_M^{(k)}(\zeta_k, \zeta_{k-1})$ for $M \in \{A,B\}$ into the definition of BC:

$$\mathrm{BC}\big(\Pi_A^{(l)}, \Pi_B^{(l)}\big) \;=\; \sum_{(\zeta_L,\dots,\zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \sqrt{\prod_{k=l+1}^{L} T_A^{(k)}(\zeta_k, \zeta_{k-1})\, T_B^{(k)}(\zeta_k, \zeta_{k-1})} \tag{S.3.12}$$

$$= \sum_{(\zeta_L,\dots,\zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1}), \tag{S.3.13}$$

using $\sqrt{\prod_k a_k b_k} = \prod_k \sqrt{a_k b_k}$ for non-negative factors. □

Lemma S.3.2 (Monotonicity, `Lean_82`). *Let $\mathrm{BC}^{(l)} := \mathrm{BC}(\Pi_A^{(l)}, \Pi_B^{(l)})$. Then $\mathrm{BC}^{(l)}$ is monotonically non-increasing as l decreases from L to 0:*

$$1 \;=\; \mathrm{BC}^{(L)} \;\geq\; \mathrm{BC}^{(L-1)} \;\geq\; \cdots \;\geq\; \mathrm{BC}^{(0)} \;\geq\; 0. \tag{S.3.14}$$

Equivalently, the squared Hellinger distance $\mathrm{H}^2_{(l)} = 1 - \mathrm{BC}^{(l)}$ is monotonically non-decreasing.

Proof. Define

$$g^{(l)}(s) \;:=\; \sum_{(\zeta_L,\dots,\zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k,\zeta_{k-1})\,\mathbf{1}\{\zeta_l = s\}. \tag{S.3.15}$$

From Lemma S.3.1, $\mathrm{BC}^{(l)} = \sum_{s\in\mathscr{S}_+^{(l)}} g^{(l)}(s)$. Grouping the layer-$(l-1)$ sum by $\zeta_l = s$,

$$\mathrm{BC}^{(l-1)} \;=\; \sum_{s\in\mathscr{S}_+^{(l)}} g^{(l)}(s)\,\widetilde{R}^{(l)}(s). \tag{S.3.16}$$

Since $\widetilde{R}^{(l)}(s) \in [0,1]$ and $g^{(l)}(s) \geq 0$, we get $\mathrm{BC}^{(l-1)} \leq \mathrm{BC}^{(l)}$. The base case $\mathrm{BC}^{(L)} = 1$ holds because $\Pi_A^{(L)} = \Pi_B^{(L)} = \delta_{s_0}$ and $\sqrt{1\cdot 1} = 1$. □

Lemma S.3.3 (Backward pass correctness, `Lean_83`). *Algorithm 8.1 computes $g^{(l)}$, hence $\mathrm{BC}^{(l)}$, $\mathrm{H}^{(l)}$, and $\mathrm{H}^2(s)$ correctly for every l and s. In particular,*

$$\sum_{s\in\mathscr{S}_+^{(0)}} \mathrm{H}^2(s) \;=\; \mathrm{H}^2_{(0)}. \tag{S.3.17}$$

Proof. Correctness of $g^{(l)}$ by downward induction on l. At $l = L$, the initialization $g^{(L)}(s_0) = 1$ and $g^{(L)}(s) = 0$ for $s \neq s_0$ matches the definition $g^{(L)}(s) = \mathbf{1}\{s = s_0\}$ in Lemma S.3.2 (the empty product equals 1).

Assume $g^{(l)}$ stored by the algorithm equals the $g^{(l)}$ defined in Lemma S.3.2. In iteration l, the algorithm computes

$$g^{(l-1)}(s') \;=\; \sum_{s\in\mathscr{S}_+^{(l)}} g^{(l)}(s)\,\widetilde{T}^{(l)}(s,s'), \tag{S.3.18}$$

which by direct expansion equals

$$\begin{aligned}
&\sum_{s}\Big(\sum_{(\zeta_L,\dots,\zeta_l)} \mathbf{1}\{\zeta_L = s_0\}\prod_{k=l+1}^{L}\widetilde{T}^{(k)}(\zeta_k,\zeta_{k-1})\,\mathbf{1}\{\zeta_l = s\}\Big)\widetilde{T}^{(l)}(s,s') \\
&\qquad= \sum_{(\zeta_L,\dots,\zeta_{l-1})} \mathbf{1}\{\zeta_L = s_0\}\prod_{k=l}^{L}\widetilde{T}^{(k)}(\zeta_k,\zeta_{k-1})\,\mathbf{1}\{\zeta_{l-1} = s'\}.
\end{aligned} \tag{S.3.19}$$

This equals $g^{(l-1)}(s')$ from Lemma S.3.2, closing the induction.

Correctness of $\mathrm{BC}^{(l)}$. After the loop, the algorithm sets $\mathrm{BC}^{(l)} = \sum_{s'} g^{(l)}(s')$, which by the proof of Lemma S.3.2 equals $\mathrm{BC}\big(\Pi_A^{(l)},\Pi_B^{(l)}\big)$. Then $\mathrm{H}^{(l)} = \sqrt{1-\mathrm{BC}^{(l)}}$ by definition (8.2).

Correctness of $\mathrm{H}^2(\cdot)$. At $l = 0$, the stored value satisfies

$$g^{(0)}(s) = \sum_{\zeta:\zeta_0=s} \sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)}, \tag{S.3.20}$$

which is exactly the quantity used in the proof of Theorem 8.1. Substituting into (8.5) gives the per-pixel disagreement values from Theorem 8.1, and (8.6) guarantees they sum to $\mathrm{H}^2_{(0)}$. □

Remark S.3.1 (Routing-state elimination and absorption histories, `Lean_84`*).* The refined diagnostic routing process just defined alternates activation and linear-routing states. A dead activation stops at the gate substep; an alive activation chooses its κ branch uniformly, after which an empty branch or outside option can absorb at the routing substep. The refined layered process retains these substeps and pads the cemetery state after absorption. Algorithm 8.1 applies to this expanded state graph; the diagnostics are read at the activation boundaries. This is the process implemented by the separate gate and branch absorption updates.

For a trajectory that remains ordinary, deleting each intermediate routing state is a bijection: its neuron and player are determined by the adjacent activation states, and its branch by whether the player is kept or switched (or by the pixel-stream tag at the input). The act-to-act formulas therefore preserve ordinary surviving-path probabilities and their Bhattacharyya overlap. The same applies after conditioning both laws on survival.

For absorbing trajectories, deletion need not be injective: a gate stop and a later empty-branch stop can become the same act-only trajectory. Thus full-law Bhattacharyya and Hellinger equality is asserted only when absorption histories are recoverable, for example when no alive routing branch leaks to the cemetery. In general the finite-fibre Cauchy–Schwarz inequality gives

$$\mathrm{BC}_{\text{refined}} \le \mathrm{BC}_{\text{act-only}}, \qquad \mathrm{H}_{\text{refined}} \ge \mathrm{H}_{\text{act-only}}.$$

For example, take one input $x = 1$ and weight $W = 1$. Model A has bias 0 and continues, with its empty negative branch absorbing with probability $1/2$; model B has bias -2 and stops at the gate. Their refined path supports are disjoint, so $\mathrm{BC}_{\text{refined}} = 0$. Collapsing the two absorption substeps instead gives $\mathrm{BC}_{\text{act-only}} = 1/\sqrt{2}$.

S.3.4 Proof of the Per-Pixel Decomposition

The following proves Theorem 8.1.

Proof. Apply the algebraic identity $(\sqrt{a} - \sqrt{b})^2 = a + b - 2\sqrt{ab}$ to each trajectory:

$$\mathrm{H}^2 \;=\; \tfrac{1}{2}\sum_{\zeta}\big(\sqrt{\Pi_A^{(0)}(\zeta)} - \sqrt{\Pi_B^{(0)}(\zeta)}\big)^2 \;=\; \tfrac{1}{2}\sum_{\zeta}\big[\Pi_A^{(0)}(\zeta) + \Pi_B^{(0)}(\zeta) - 2\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)}\big]. \tag{S.3.21}$$

Group the trajectory sum by the terminal pixel $\zeta_0 = s$:

$$\mathrm{H}^2 \;=\; \tfrac{1}{2}\sum_{s\in\mathscr{S}_+^{(0)}}\Big[\sum_{\zeta:\zeta_0=s}\Pi_A^{(0)}(\zeta) + \sum_{\zeta:\zeta_0=s}\Pi_B^{(0)}(\zeta) - 2\sum_{\zeta:\zeta_0=s}\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)}\Big]. \tag{S.3.22}$$

By definition, $\sum_{\zeta:\zeta_0=s}\Pi_M^{(0)}(\zeta) = m_M^{(0)}(s)$ for $M \in \{A,B\}$. By Lemma S.3.1 restricted to trajectories terminating at s (i.e. tracing through Lemma S.3.2's definition of $g^{(l)}$), $\sum_{\zeta:\zeta_0=s}\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)} =$

$g^{(0)}(s)$. Substituting,

$$\mathrm{H}^2 \;=\; \tfrac{1}{2}\sum_{s}\big[m_A^{(0)}(s)+m_B^{(0)}(s)-2g^{(0)}(s)\big] \;=\; \sum_{s\in\mathscr{S}_+^{(0)}}\mathrm{H}^2(s), \tag{S.3.23}$$

proving (ii).

For (i), apply Cauchy–Schwarz to the sum defining $g^{(0)}(s)$:

$$g^{(0)}(s)=\sum_{\zeta:\zeta_0=s}\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)}\le\sqrt{\Big(\sum_{\zeta:\zeta_0=s}\Pi_A^{(0)}(\zeta)\Big)\Big(\sum_{\zeta:\zeta_0=s}\Pi_B^{(0)}(\zeta)\Big)}=\sqrt{m_A^{(0)}(s)\,m_B^{(0)}(s)}. \tag{S.3.24}$$

Combined with the AM–GM inequality $\sqrt{m_A^{(0)}(s)m_B^{(0)}(s)}\le\frac{1}{2}(m_A^{(0)}(s)+m_B^{(0)}(s))$, this gives $g^{(0)}(s)\le\frac{1}{2}(m_A^{(0)}(s)+m_B^{(0)}(s))$, so $\mathrm{H}^2(s)\ge 0$. Equality holds iff $\Pi_A^{(0)}(\cdot\mid\zeta_0=s)=\Pi_B^{(0)}(\cdot\mid\zeta_0=s)$ *and* $m_A^{(0)}(s)=m_B^{(0)}(s)$.

Item (iii) is immediate from (ii) by separating the cemetery from the ordinary pixels. □

S.3.5 Conditioned-Survival Trajectory Laws

Theorem S.3.4 (Conditioned-survival trajectory law and conditioned Hellinger distance, `Lean_86`). *Let* $\mathsf{Surv}:=\{(\zeta_L,\dots,\zeta_0):\zeta_0\in\mathscr{S}^{(0)}\}$ *be the event, in the trajectory space* $\prod_{l=0}^{L}\mathscr{S}_+^{(l)}$, *that the backward trajectory ends in an ordinary input-layer state — equivalently, the event that the trajectory never enters the cemetery* †. *For each model* $M\in\{A,B\}$ *define the survival function*

$$\psi_M^{(l)}(s)\;:=\;\mathbb{P}_M(\mathsf{Surv}\mid\zeta_l=s),\qquad s\in\mathscr{S}_+^{(l)},\;\; l\in\{0,\dots,L\}. \tag{S.3.25}$$

Then:

(i) The boundary and recursion are

$$\psi_M^{(0)}(s)=1\;\text{ for }s\in\mathscr{S}^{(0)},\qquad \psi_M^{(0)}(\dagger)=0, \tag{S.3.26}$$

and for every $l\ge 1$,

$$\psi_M^{(l)}(s)\;=\;\sum_{s'\in\mathscr{S}_+^{(l-1)}}T_M^{(l)}(s\to s')\,\psi_M^{(l-1)}(s')\;=\;\sum_{s'\in\mathscr{S}^{(l-1)}}T_M^{(l)}(s\to s')\,\psi_M^{(l-1)}(s'). \tag{S.3.27}$$

In particular,

$$Z_M\;:=\;\mathbb{P}_M(\mathsf{Surv})\;=\;\psi_M^{(L)}(s_0)\;=\;\sum_{s\in\mathscr{S}^{(0)}}m_M^{(0)}(s)\;=\;1-m_M^{(0)}(\dagger). \tag{S.3.28}$$

(ii) Assume $Z_M>0$. *For every ordinary state* $s\in\mathscr{S}^{(l)}$ *with* $\psi_M^{(l)}(s)>0$, *define the conditioned kernel on ordinary states by*

$$T_{M,\mathrm{surv}}^{(l)}(s\to s')\;:=\;\frac{T_M^{(l)}(s\to s')\,\psi_M^{(l-1)}(s')}{\psi_M^{(l)}(s)},\qquad s'\in\mathscr{S}^{(l-1)}. \tag{S.3.29}$$

Then $T^{(l)}_{M,\mathrm{surv}}(s \to \cdot)$ *is row-stochastic on* $\mathscr{S}^{(l-1)}$*, and the conditional trajectory distribution* $\Pi_{M,\mathrm{surv}} \coloneqq \Pi_M(\,\cdot \mid \mathsf{Surv}\,)$ *factorizes as the layered Markov law*

$$\Pi_{M,\mathrm{surv}}(\zeta_L,\dots,\zeta_0) = \mathbf{1}\{\zeta_L = s_0\} \prod_{k=1}^{L} T^{(k)}_{M,\mathrm{surv}}(\zeta_k \to \zeta_{k-1}), \qquad (\zeta_L,\dots,\zeta_0) \in \mathsf{Surv}. \tag{S.3.30}$$

Its layer-l *marginals satisfy*

$$m^{(l)}_{M,\mathrm{surv}}(s) \;=\; \frac{m^{(l)}_M(s)\,\psi^{(l)}_M(s)}{Z_M}, \qquad s \in \mathscr{S}^{(l)}. \tag{S.3.31}$$

(iii) Define the conditioned geometric-mean kernel

$$\widetilde{T}^{(l)}_{\mathrm{surv}}(s,s') \coloneqq \sqrt{T^{(l)}_{A,\mathrm{surv}}(s \to s')\,T^{(l)}_{B,\mathrm{surv}}(s \to s')} = \frac{\widetilde{T}^{(l)}(s,s')\,\sqrt{\psi^{(l-1)}_A(s')\,\psi^{(l-1)}_B(s')}}{\sqrt{\psi^{(l)}_A(s)\,\psi^{(l)}_B(s)}}. \tag{S.3.32}$$

The Bhattacharyya coefficient of the conditioned trajectory laws is

$$\mathrm{BC}\big(\Pi_{A,\mathrm{surv}},\Pi_{B,\mathrm{surv}}\big) \;=\; \sum_{(\zeta_L,\dots,\zeta_0)\in\mathsf{Surv}} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=1}^{L} \widetilde{T}^{(k)}_{\mathrm{surv}}(\zeta_k,\zeta_{k-1}), \tag{S.3.33}$$

so it is computed by the same Hellinger pass (Algorithm 8.1), but on the ordinary-state graph with the conditioned kernel.

(iv) Let

$$\mathrm{BC}^{(0)}_{\mathrm{ord}} \;\coloneqq\; \sum_{s\in\mathscr{S}^{(0)}} g^{(0)}(s) \;=\; \sum_{(\zeta_L,\dots,\zeta_0)\in\mathsf{Surv}} \sqrt{\Pi_A(\zeta_L,\dots,\zeta_0)\,\Pi_B(\zeta_L,\dots,\zeta_0)}. \tag{S.3.34}$$

Then

$$\mathrm{BC}\big(\Pi_{A,\mathrm{surv}},\Pi_{B,\mathrm{surv}}\big) \;=\; \frac{\mathrm{BC}^{(0)}_{\mathrm{ord}}}{\sqrt{Z_A Z_B}}, \tag{S.3.35}$$

and the conditioned-survival Hellinger distance is

$$\mathrm{H}^2_{\mathrm{surv}} \;\coloneqq\; \mathrm{H}^2\big(\Pi_{A,\mathrm{surv}},\Pi_{B,\mathrm{surv}}\big) \;=\; 1 - \frac{\mathrm{BC}^{(0)}_{\mathrm{ord}}}{\sqrt{Z_A Z_B}}. \tag{S.3.36}$$

Proof. For (i), (S.3.26) is immediate from the definition of Surv: at layer 0 every ordinary state survives and the cemetery does not. For $l \ge 1$, conditioning on the next step gives

$$\psi^{(l)}_M(s) = \sum_{s'\in\mathscr{S}^{(l-1)}_+} T^{(l)}_M(s \to s')\,\mathbb{P}_M(\mathsf{Surv} \mid \zeta_{l-1} = s') = \sum_{s'\in\mathscr{S}^{(l-1)}_+} T^{(l)}_M(s \to s')\,\psi^{(l-1)}_M(s'). \tag{S.3.37}$$

Since $\psi^{(l-1)}_M(\dagger) = 0$, the cemetery term drops out and (S.3.27) follows. Evaluating at s_0 gives $Z_M = \psi^{(L)}_M(s_0)$. The identities with $m^{(0)}_M$ are just the layer-0 marginal of the event Surv.

For (ii), row-stochasticity follows from (S.3.27):

$$\sum_{s' \in \mathscr{S}^{(l-1)}} T^{(l)}_{M,\mathrm{surv}}(s \to s') = \frac{1}{\psi^{(l)}_M(s)} \sum_{s' \in \mathscr{S}^{(l-1)}} T^{(l)}_M(s \to s')\, \psi^{(l-1)}_M(s') = 1. \tag{S.3.38}$$

Now fix $(\zeta_L, \dots, \zeta_0) \in \mathsf{Surv}$. By definition of conditional probability,

$$\Pi_{M,\mathrm{surv}}(\zeta_L, \dots, \zeta_0) = \frac{\Pi_M(\zeta_L, \dots, \zeta_0)}{Z_M} = \frac{\mathbf{1}\{\zeta_L = s_0\}}{Z_M} \prod_{k=1}^{L} T^{(k)}_M(\zeta_k \to \zeta_{k-1}). \tag{S.3.39}$$

Multiplying the conditioned kernels and telescoping the survival factors gives

$$\prod_{k=1}^{L} T^{(k)}_{M,\mathrm{surv}}(\zeta_k \to \zeta_{k-1}) = \prod_{k=1}^{L} T^{(k)}_M(\zeta_k \to \zeta_{k-1}) \frac{\psi^{(k-1)}_M(\zeta_{k-1})}{\psi^{(k)}_M(\zeta_k)} \tag{S.3.40}$$

$$= \frac{\psi^{(0)}_M(\zeta_0)}{\psi^{(L)}_M(s_0)} \prod_{k=1}^{L} T^{(k)}_M(\zeta_k \to \zeta_{k-1}) \tag{S.3.41}$$

$$= \frac{1}{Z_M} \prod_{k=1}^{L} T^{(k)}_M(\zeta_k \to \zeta_{k-1}), \tag{S.3.42}$$

because $\zeta_0 \in \mathscr{S}^{(0)}$ implies $\psi^{(0)}_M(\zeta_0) = 1$. This is (S.3.30). For the marginals,

$$m^{(l)}_{M,\mathrm{surv}}(s) = \mathbb{P}_M(\zeta_l = s \mid \mathsf{Surv}) = \frac{\mathbb{P}_M(\zeta_l = s, \mathsf{Surv})}{Z_M} = \frac{m^{(l)}_M(s)\, \psi^{(l)}_M(s)}{Z_M}, \tag{S.3.43}$$

which is (S.3.31).

For (iii), substitute (S.3.29) into the square root to obtain (S.3.32). Then apply Lemma S.3.1 to the conditioned trajectory laws (S.3.30) on the ordinary-state graph. This gives (S.3.33) directly.

For (iv), expand the Bhattacharyya coefficient of the conditioned laws:

$$\mathrm{BC}\big(\Pi_{A,\mathrm{surv}}, \Pi_{B,\mathrm{surv}}\big) = \sum_{(\zeta_L,\dots,\zeta_0) \in \mathsf{Surv}} \sqrt{\frac{\Pi_A(\zeta_L, \dots, \zeta_0)}{Z_A} \frac{\Pi_B(\zeta_L, \dots, \zeta_0)}{Z_B}} \tag{S.3.44}$$

$$= \frac{1}{\sqrt{Z_A Z_B}} \sum_{(\zeta_L,\dots,\zeta_0) \in \mathsf{Surv}} \sqrt{\Pi_A(\zeta_L, \dots, \zeta_0)\, \Pi_B(\zeta_L, \dots, \zeta_0)}. \tag{S.3.45}$$

Grouping the last sum by the ordinary terminal state $\zeta_0 = s$ and using the definition of $g^{(0)}(s)$ exactly as in the proof of Theorem 8.1 yields (S.3.34), hence (S.3.35). Equation (S.3.36) is then the identity $\mathrm{H}^2 = 1 - \mathrm{BC}$ applied to the conditioned trajectory laws. □

Implementation consequence. Theorem S.3.4 gives two exactly equivalent computations of the conditioned scalar:

(a) *Kernel-side conditioning:* replace each row by the reweighted kernel (S.3.29), form the geometric mean (S.3.32), and run the ordinary Hellinger backward pass on $\mathscr{S}^{(L)} \times \cdots \times \mathscr{S}^{(0)}$.

(b) *Post-hoc normalization:* run the original pass once, extract the ordinary overlap $\mathrm{BC}^{(0)}_{\mathrm{ord}} = \sum_{s\in\mathscr{S}^{(0)}} g^{(0)}(s)$ and the survival masses Z_A, Z_B, and apply (S.3.36).

The first route makes the conditioned trajectory law explicit; the second is cheaper when only the scalar is needed, but for deep networks it becomes numerically infeasible when the survival probability Z_M of either model $M \in \{A,B\}$ gets too small.

Conditioned disagreement map. Applying Theorem 8.1 to the conditioned kernels $T^{(l)}_{M,\mathrm{surv}}$ yields the conditioned disagreement map

$$\mathrm{H}^2_{\mathrm{surv}}(s) \;\coloneqq\; \tfrac{1}{2}\big(m^{(0)}_{A,\mathrm{surv}}(s) + m^{(0)}_{B,\mathrm{surv}}(s)\big) - g^{(0)}_{\mathrm{surv}}(s), \qquad s \in \mathscr{S}^{(0)}, \tag{S.3.46}$$

which is non-negative and satisfies $\sum_{s\in\mathscr{S}^{(0)}} \mathrm{H}^2_{\mathrm{surv}}(s) = \mathrm{H}^2_{\mathrm{surv}}$.

S.3.6 Randomization Bounds

Lemma S.3.5 (Alive-set mismatch, `Lean_87`). *Suppose a dead state sends all its mass to the cemetery. Every state in the alive symmetric difference then has zero geometric-mean mass to ordinary successors: $\sum_{s'\in\mathscr{S}^{(l-1)}} \widetilde{T}^{(l)}(s_i,s') = 0$. If alive states have no cemetery leakage, as for the stated nondegenerate Stopping-Game kernel, its full geometric-mean row is also zero.*

Proof. If i is alive in A and dead in B, then $T^{(l)}_B(s_i,s') = 0$ for every ordinary successor, so each ordinary geometric mean is zero. The same argument applies with A and B exchanged. If the alive row also has zero cemetery mass, the cemetery geometric mean vanishes. With alive-state leakage this last conclusion does not follow; the ordinary-state conclusion still holds. □

Fix a ReLU MLP with hidden widths $n_1,\dots,n_L$, an input x, and start state s_0 at the target output. Let B replace $W^{(l^*)}$ by an i.i.d. matrix from a continuous distribution symmetric about zero, and assume that for each $l \ge l^*$ and every neuron i, $\mathbb{P}_B\big(i \in \mathrm{alive}^{(l)}_B \mid \text{kernels above layer } l\big) \le 1-\delta$ for some $\delta \in (0,\tfrac{1}{2}]$. Then $T^{(l)}_A = T^{(l)}_B$ for $l < l^*$; at $l = l^*$ the row weights differ and the alive sets may differ; for $l > l^*$ the perturbation propagates through the activations. The same reduction applies to convolutional and transformer blocks interpreted as single *Layered Markov Process* (LMP) layers. Write $\eta \coloneqq 1-\delta \in \big[\tfrac{1}{2},1\big)$ and $N \coloneqq L - l^* + 1$.

Theorem S.3.6 (Randomization in the Stopping Game, `Lean_88`). $\mathbb{E}_B\big[\mathrm{BC}^{(0)}_{\mathrm{ord}}(\Pi_A,\Pi_B)\big] \le \eta^N$. *Hence, for every $z > 0$, the conditioned-survival distance of Theorem S.3.4 satisfies $\mathbb{E}_B\big[\mathrm{H}^2_{\mathrm{surv}}\,\mathbf{1}\{Z_AZ_B \ge z\}\big] \ge \mathbb{P}_B(Z_AZ_B \ge z) - \eta^N/\sqrt{z}$, while the plain distance is $\mathrm{H}^2 = 1 - \mathrm{BC}^{(0)}_{\mathrm{ord}} - g^{(0)}(\dagger)$.*

Proof. At a layer $l \ge l^*$, ordinary overlap mass at a state dead in both models moves into the common cemetery, mass on the alive symmetric difference vanishes (Lemma S.3.5), and on states alive in both models the row mass is at most 1 by Cauchy–Schwarz. Hence $\mathrm{BC}^{(l-1)}_{\mathrm{ord}} \le \sum_{s\in\mathscr{S}^{(l)}} g^{(l)}(s)\,\mathbf{1}\{s \in \mathrm{alive}^{(l)}_B\}$, and, since $g^{(l)}$ is fixed by the kernels above layer l, the conditional bound gives $\mathbb{E}_B[\mathrm{BC}^{(l-1)}_{\mathrm{ord}}] \le \eta\,\mathbb{E}_B[\mathrm{BC}^{(l)}_{\mathrm{ord}}]$. Iterating from $\mathrm{BC}^{(L)}_{\mathrm{ord}} = 1$ over $l = L,\dots,l^*$ and using that $\mathrm{BC}^{(l)}_{\mathrm{ord}}$ is non-increasing below l^* yields the bound. The $\mathrm{H}_{\mathrm{surv}}$ statement follows from $\mathrm{H}^2_{\mathrm{surv}} = 1 - \mathrm{BC}^{(0)}_{\mathrm{ord}}/\sqrt{Z_AZ_B} \ge 1 - \mathrm{BC}^{(0)}_{\mathrm{ord}}/\sqrt{z}$ on $\{Z_AZ_B \ge z\}$ (Theorem S.3.4), the plain one from $\mathrm{BC}^{(0)} = \mathrm{BC}^{(0)}_{\mathrm{ord}} + g^{(0)}(\dagger)$. □

Remark S.3.2 (Cemetery floor). Theorem S.3.6 bounds only the ordinary-state overlap. Overlap mass that reaches a neuron dead in both models enters the cemetery in both,

$$g^{(l-1)}(\dagger) = g^{(l)}(\dagger) + \sum_{s \in \mathrm{dead}_A^{(l)} \cap \mathrm{dead}_B^{(l)}} g^{(l)}(s),$$

and is never thinned again, so plain H saturates below one. In a mean-field approximation with uniform dead fraction q under A and alive probability p under B, the ordinary overlap shrinks by $(1-q)p$ per layer while a fraction $q(1-p)$ is absorbed, so $g^{(0)}(\dagger) \to q(1-p)/(1-(1-q)p)$ as $N \to \infty$; for illustrative values $q = 0.2$, $p = \frac{1}{2}$ this gives H ≈ 0.91, the same order as the observed 0.94–0.97 (Table 8.3). The conditioned-survival distance removes the cemetery floor exactly, and $\mathbb{E}_B[\mathrm{H}_{\mathrm{surv}}^2 \mid Z_A Z_B \geq z] \to 1$ for every fixed $z > 0$ with $\mathbb{P}_B(Z_A Z_B \geq z)$ bounded away from zero. For the Routing Game the activation-weighted kernel concentrates $g^{(l)}$ on neurons with largest $|aW|$, which rarely coincide with $\mathrm{dead}_A \cap \mathrm{dead}_B$, so H is nearly identical to $\mathrm{H}_{\mathrm{surv}}$ in practice.

Theorem S.3.7 (Randomization in the Routing Game, `Lean_89`). *Under the same setup without the alive-set assumption, take the RG kernel at temperature $\tau > 0$ and assume that for every $l \geq l^*$ each state alive in both models has row mass $\sum_{s'} \widetilde{T}^{(l)}(s, s') \leq \bar{b}$ for some $0 \leq \bar{b} < 1$. Then* $\mathrm{BC}_{\mathrm{ord}}^{(0)}(\Pi_A, \Pi_B) \leq \bar{b}^N$.

Proof. Since $a_{M,i}^{(l-1)} = 0$ for $i \notin \mathrm{alive}_M^{(l-1)}$, the Gibbs policy (S.3.7) sends no mass to dead neurons, so for $1 \leq l < L$ the overlap $g^{(l)}$ vanishes outside $\mathrm{alive}_A^{(l)} \cap \mathrm{alive}_B^{(l)}$, and the start state is alive in both, in the symmetric difference or dead in both. In the latter two cases its ordinary overlap contribution is zero by the ordinary-state part of Lemma S.3.5. Hence $\mathrm{BC}_{\mathrm{ord}}^{(l-1)} \leq \bar{b}\,\mathrm{BC}_{\mathrm{ord}}^{(l)}$ for every $l \geq l^*$, with no alive-set factor (the alive mismatch at layer $l-1$ is already part of the row masses at layer l); iterate as in Theorem S.3.6. The assumption reflects the structure of the RG: each act-to-act row factors as the source-deadness indicator $\mathbf{1}\{z_j^{(l)} > 0\}$, the constant $\frac{1}{2}$ uniform κ-branch, and the Gibbs policy $\pi_{M,\kappa}^{(l)}(\cdot \mid j) \propto [a_M^{(l-1)}[\cdot]\, W_M^{(l,\kappa)}[\cdot, j]]^{1/\tau}$. At $l = l^*$ the two policies share $a^{(l^*-1)}$, while their weights may differ; above that layer their activation factors may also differ. These changes can reduce overlap, but a continuous randomization law alone does not imply distinct normalized rows almost surely. For example, if each sign branch has just one positive-mass ordinary route, changing its weight without changing its sign leaves that branch's normalized policy unchanged. The strict conditional row-overlap bound $\bar{b} < 1$ is therefore an explicit hypothesis of this theorem; its applicability requires a separate argument for the architecture and randomization law, or an empirical assessment. The equality case of Cauchy–Schwarz explains strict contraction when the completed normalized rows actually differ. □

S.4 Extended Experimental Results and Protocol

This chapter collects extended results for both experimental campaigns. The split-method sections retain the hidden-monotonicity evaluation; the game-method sections use the revised game-paper protocol, confidence intervals, validation selections, ablations and cross-dataset results.

S.4.1 Full Evaluation Results for the Split Methods

This section provides the comprehensive evaluation results for all interpretability methods tested in the split-method setup of Section 7.1.

S.4.1.1 VGG-16 Qualitative Saliency Map Comparison

Complementing Figure 7.1, Figure S.4.1 gives more saliency examples. Again, SplitCAM in layer 26 focuses more on the whole classified objects compared to LayerCAM in the same layer. SplitGrad in layer 0 detects too many edges of the original image to be a useful explanation of the model's decision; especially in images with many background details, e.g. the rubble in the wood rabbit image, it fails to focus on the important image parts. This aligns with the observed performance peaks of SplitGrad and SplitCAM in deeper layers.

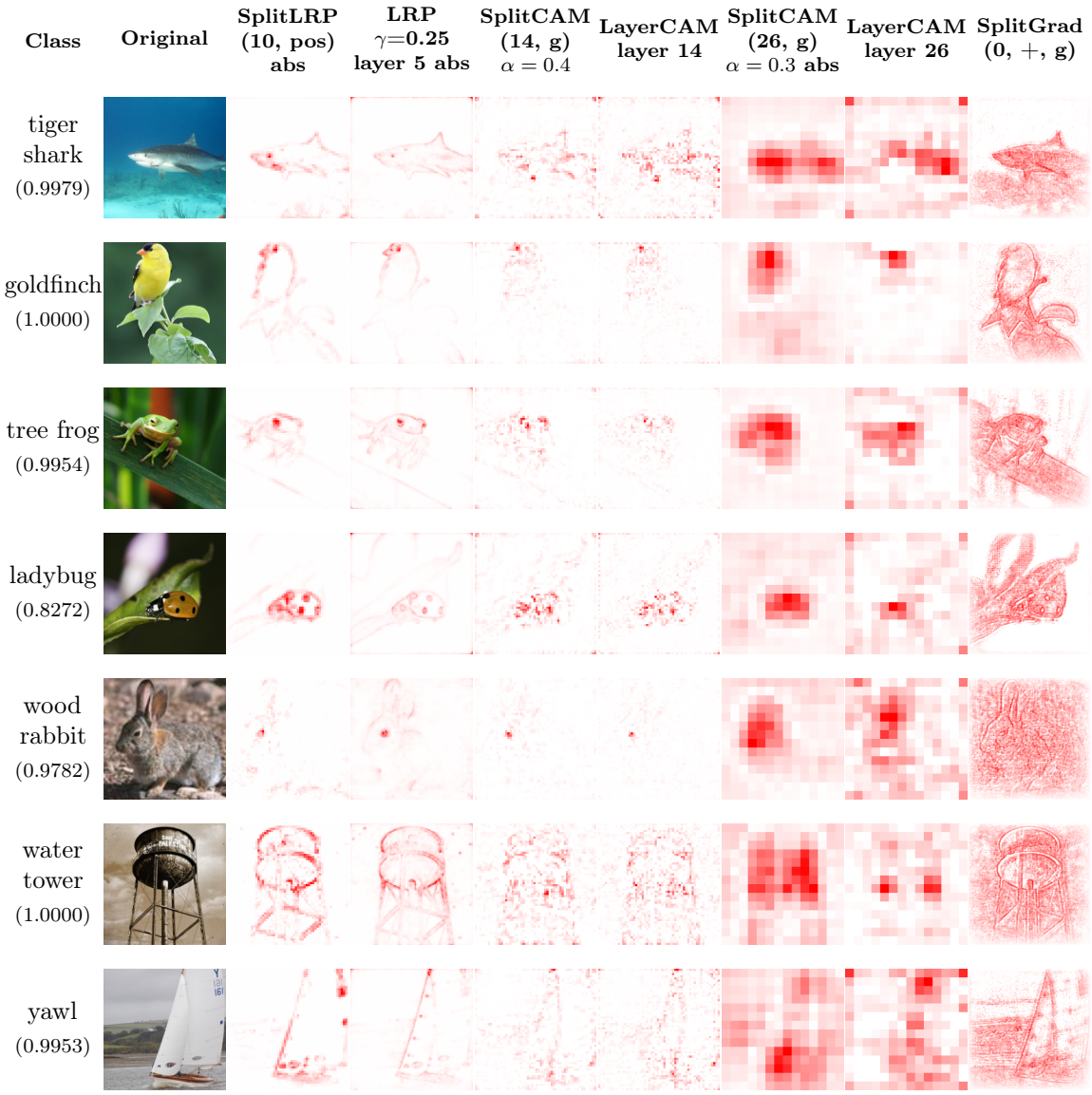


Fig. S.4.1: Saliency map comparison for VGG-16 on images from the custom ImageNet-S test subset. Each row shows the predicted class (with confidence), original image, and saliency maps from different methods. Methods include SplitLRP, classical LRP, SplitCAM variants, LayerCAM, and SplitGrad.

S.4.1.2 Quantitative Analysis on VGG-16

Table S.4.1 presents the detailed quantitative metrics across all split configurations, baselines, and classical methods for VGG-16. SplitCAM's best reported localization scores exceed those of the listed LayerCAM rows, and its reported Maximum Sensitivity values are lower, while LayerCAM at layer 14 has higher Pixel-Flipping score-drop AUC. Comparisons across the selected rows also vary the layer or absolute-value convention. The controlled α sweeps in Section S.4.2 expose the resulting trade-offs more directly; neither comparison establishes a uniform improvement across metrics.

Tab. S.4.1: Comprehensive interpretability evaluation results (VGG-16; representative configurations, best per method). Abbreviations as in Table 7.1. For each method configuration the best metric values across the considered layer configurations are highlighted in bold with dark blue shading. PF^{drop} is score-drop AUC; n/r denotes a result not reported in the paper, and n/a an inapplicable entry.

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitCAM Methods								
SplitCAM (sc, α 0.4)	(12, g)	n	4.720	**0.651**	**0.912**	0.213	1.794	**0.389**
SplitCAM (sc, α 0.4)	(14, g)	n	**2.727**	0.563	0.836	**0.300**	**2.404**	1.012
SplitCAM (sc, α 0.4, wta)	(14, g)	n	**2.761**	0.543	0.827	**0.303**	**2.414**	1.056
SplitCAM (sc, α 0.4, wta)	(26, +, g)	y	4.711	**0.635**	**0.938**	0.165	1.590	**0.456**
SplitCAM (sc, α 0.5, wta)	(14, g)	n	**2.766**	0.536	0.823	**0.303**	**2.416**	1.056
SplitCAM (sc, α 0.5, wta)	(26, +, g)	n	4.608	**0.607**	**0.903**	0.185	1.684	**0.566**
SplitGrad Methods								
SplitGrad (α 0.4)	(12, +, g)	n	5.633	**0.613**	**0.873**	0.148	1.461	**0.364**
SplitGrad (α 0.4)	(14, +, g)	y	**3.429**	0.533	0.820	**0.235**	**2.119**	0.851
SplitGrad (α 0.4, wta)	(12, +, g)	n	4.735	**0.635**	**0.883**	0.186	1.756	0.651
SplitGrad (α 0.4, wta)	(14, +, g)	y	**3.287**	0.540	0.818	**0.249**	**2.186**	0.857
SplitGrad (α 0.5, wta)	(12, +, g)	y	**3.379**	**0.551**	0.859	**0.261**	**2.220**	0.869
SplitGrad (α 0.5, wta)	(26, +, g)	y	4.948	0.543	**0.899**	0.162	1.527	**0.617**
SplitLRP Methods								
SplitLRP (sc)	(7, pos)	n	**3.062**	0.092	**0.857**	**0.296**	**2.290**	0.865
SplitLRP (sc)	(19, pos)	n	4.393	**0.600**	0.823	0.166	1.656	**0.291**
SplitLRP (sc, wta)	(7, pos)	n	**3.235**	-0.036	0.836	**0.296**	**2.291**	0.866
SplitLRP (sc, wta)	(26, pos)	y	5.168	**0.588**	**0.871**	0.151	1.473	**0.282**
Classical Methods (No Layer Optimization)								
Deconvolution	0	y	5.435	0.468	0.629	0.204	1.715	0.956
DeepLIFT	0	y	4.447	0.551	0.726	0.281	1.945	1.053
Feature Ablation	0	y	6.029	0.479	0.624	0.123	1.250	0.859
GradientSHAP	0	y	4.808	0.552	0.783	0.261	1.862	1.142
IntGrad	0	y	4.838	0.554	0.797	0.262	1.851	1.069
Occlusion	0	y	5.214	0.499	0.716	0.130	1.217	0.699
GCAM++	30	n	5.955	0.564	0.827	0.130	1.245	0.401
Classical Methods (Layer Optimization)								
Guided Backprop	2	n	**3.276**	**0.632**	0.883	**0.281**	**2.302**	0.571
Guided Backprop	5	n	3.448	0.614	**0.887**	0.240	2.091	**0.375**
Guided GCAM	0	n	**4.301**	**-0.053**	**0.857**	0.161	**1.459**	1.047
Guided GCAM	26	n	5.130	-2.635	0.843	**0.179**	1.415	**0.947**
GCAM	0	n	6.006	**0.356**	**0.638**	0.060	0.899	1.123
GCAM	14	n	**5.555**	0.306	0.580	**0.136**	**1.525**	**0.903**
LCAM	14	n/a	2.792	0.576	0.798	**0.317**	**2.474**	**2.656**
LCAM	2	n/a	**3.128**	**0.580**	**0.855**	0.300	2.191	5.508
LRP $\gamma = 0.25$	0	n	**3.189**	**0.615**	**0.680**	0.218	1.918	0.519
LRP $\gamma = 0.25$	14	n	3.593	0.577	0.611	**0.279**	**2.255**	**0.491**
Baseline Methods								
Baseline: Binary Mask	0	y	6.011	1.000	1.000	0.091	1.054	n/r
Baseline: Original Image	0	y	8.770	-1.699	0.572	0.061	0.619	n/r
Random Baseline	0	y	3.570	0.425	0.458	0.203	1.585	n/r

S.4.1.3 Quantitative Analysis on ResNet-18

Table S.4.2 presents the detailed quantitative metrics across all split configurations, baselines, and classical methods for ResNet-18.

Tab. S.4.2: Comprehensive interpretability evaluation results (ResNet-18; representative configurations, best per method). Abbreviations as in Table 7.2. For each method configuration the best metric values across the considered layer configurations are highlighted in bold with dark blue shading. PF^{drop} is score-drop AUC; n/r denotes a result not reported in the paper, and n/a an inapplicable entry.

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitCAM Methods								
SplitCAM (sc, α 0.3, wta)	(1.0.2, g)	n	**3.503**	0.487	0.747	**0.168**	**1.395**	1.277
SplitCAM (sc, α 0.3, wta)	(4.1.2, g)	n	5.654	**0.673**	**0.972**	0.081	0.880	**0.233**
SplitCAM (sc, α 0.5, wta)	(2.0.1, g)	n	**3.682**	-1.370	0.887	**0.158**	**1.447**	1.078
SplitCAM (sc, α 0.5, wta)	(4.1.1, +, g)	y	6.526	**0.611**	**0.956**	0.066	0.737	**0.187**
SplitGrad Methods								
SplitGrad (α 0.3, wta)	(1.0.2, g)	y	**4.449**	0.513	0.777	**0.149**	**1.266**	0.813
SplitGrad (α 0.3, wta)	(4.1.2, g)	y	5.663	**0.649**	**0.972**	0.082	0.882	**0.219**
SplitGrad (α 0.5, wta)	(3.1.1, g)	y	**3.588**	0.559	0.880	**0.130**	**1.366**	0.859
SplitGrad (α 0.5, wta)	(4.1.2, g)	y	5.690	**0.654**	**0.972**	0.083	0.886	**0.230**
SplitLRP Methods								
SplitLRP (sc, wta)	(conv1, pos)	y	**6.313**	**0.506**	**0.587**	**0.056**	**0.697**	0.755
SplitLRP (sc, wta)	(1.0.2, pos)	y	6.785	0.483	0.567	0.050	0.686	**0.696**
Classical Methods (No Layer Optimization)								
Deconvolution	conv1	y	5.474	0.478	0.666	0.151	1.177	0.618
GradientSHAP	conv1	y	5.269	0.527	0.779	0.166	1.227	1.074
GCAM++	4.1.2	n	6.054	0.556	0.945	0.077	0.838	0.140
IntGrad	conv1	y	5.265	0.528	0.777	0.167	1.226	0.994
Feature Ablation	conv1	n	4.655	0.136	0.684	0.188	1.388	1.077
Occlusion	conv1	n	5.300	0.179	0.726	0.130	1.030	0.785
Deconvolution	conv1	n	3.934	0.484	0.666	0.147	1.203	0.751
Classical Methods (Layer Optimization)								
GCAM	2.0.1	y	**4.003**	0.450	0.678	0.052	**1.024**	0.738
GCAM	4.1.2	n	6.070	**0.554**	**0.945**	**0.076**	0.829	**0.131**
Guided GCAM	4.0.2	n	4.929	**0.742**	0.866	0.119	1.008	0.687
Guided GCAM	4.1.2	n	**3.843**	0.717	**0.883**	**0.129**	**1.096**	**0.612**
LCAM	2.0.2	n	**3.690**	0.549	0.865	**0.144**	**1.428**	0.913
LCAM	4.1.1	n	5.661	**0.664**	**0.971**	0.083	0.890	**0.227**
LRP $\gamma = 0.25$	conv1	y	**3.550**	**0.656**	0.852	**0.213**	**1.737**	0.521
LRP $\gamma = 0.25$	4.1.1	n	5.649	0.630	**0.965**	0.081	0.877	**0.200**
Guided Backprop	conv1	n	**3.517**	**0.621**	0.867	**0.212**	**1.757**	0.459
Guided Backprop	3.1.2	n	3.874	0.582	**0.938**	0.121	1.307	**0.213**
Baseline Methods								
Baseline: Original Image	conv1	y	8.351	-1.699	0.572	0.049	0.435	n/r
Baseline: Binary Mask	conv1	y	5.964	1.000	1.000	0.067	0.739	n/r
Random Baseline	n/a	y	4.071	0.425	0.440	0.147	1.067	n/r

S.4.1.4 Randomization Sanity Check

The model-randomization sanity check (Adebayo et al., 2018) tests whether a saliency method truly depends on the model's learned parameters or merely reflects input-dependent patterns that survive when the model is corrupted. For each attribution method, saliency maps on the trained VGG-16 are compared with maps obtained after randomly reinitializing the weights of the last three feedforward layers with Gaussian noise; a method genuinely sensitive to the trained parameters should show a substantial similarity drop. While Binder et al. (2023) discuss important shortcomings of top-down randomization-based sanity checks, the test is applied here as a complementary diagnostic (the trajectory-space randomization diagnostic of Section 8.4 is the thesis' primary tool). Similarity is quantified by (i) HOG similarity (Pearson correlation of the histograms of oriented gradients), (ii) signed Spearman rank correlation, and (iii) SSIM; lower similarity indicates stronger parameter dependence.

In Table S.4.3, every reported SplitCAM and SplitGrad configuration has lower SSIM than every listed baseline, whereas the HOG and Spearman comparisons are mixed. These results show that the maps respond to the tested parameter randomization, but do not isolate the effect of stabilization on numerical gradient growth. SplitLRP retains high HOG and Spearman similarity (approximately 0.99 and 1.00), while SSIM decreases to 0.45–0.56. Attenuation by the ϵ-LRP rule is a possible explanation for this behavior, but the randomization comparison alone does not identify its cause.

S.4.1.5 Computational Efficiency

Table S.4.4 reports mean attribution times across the custom 50-image validation subset of ImageNet-S for ResNet-18 and VGG-16. Model splitting is reported separately, averaged over 20 setup procedures: 0.033 s for ResNet-18 and 0.030 s for VGG-16. SplitCAM and SplitGrad take 0.33–0.44 s per image, compared with 0.28–0.31 s for the row labeled Guided Grad-CAM. SplitCAM takes approximately 14–29 times as long as the corresponding ordinary Grad-CAM and LayerCAM rows, while the split methods are much faster than the reported Feature Ablation and Occlusion configurations.

The released evaluator initializes the split model once per method configuration and reuses it across images. Its per-image timer covers attribution generation and the helper's postprocessing, copying and cleanup; image and model loading, initial input transfer to the device, target prediction and model splitting occur outside that timer. The split helper computes maps for all spatial layer chunks with cached activations and gradients before selecting the configured output, so the timing includes this additional map computation. The shifting parameter α changes the backward propagation rule, but runtime variation with α was not separately measured.

The available artifacts do not identify the hardware, numerical precision or exact configuration IDs of the historical timing run. In particular, the released VGG-16 timing configuration named `guided_grad_cam` dispatches Guided LayerCAM, while the source table labels its row Guided Grad-CAM. The table retains the reported label and numbers; this mismatch limits identification of that measured baseline. The reported timings describe the source implementation and do not establish an upper bound on the overhead of other implementations.

Tab. S.4.3: Randomization sanity check for the split methods on VGG-16: mean similarity between attribution maps of the trained model and of a copy whose last three feedforward layers are randomly reinitialized, across the custom test subset of ImageNet-S. Lower similarity indicates stronger dependence on the learned parameters. Abbreviations as in Table 7.1.

Method	HOG ↓	Spearman ↓	SSIM ↓
Baselines			
Deconvolution	0.985	0.999	0.481
DeepLIFT	0.365	0.274	0.716
GradientSHAP	0.286	0.081	0.571
Input × Gradient	0.313	0.082	0.621
IntGrad	0.303	0.102	0.584
LCAM (layer 14)	0.688	0.123	0.417
LCAM (layer 2)	0.542	0.114	0.475
Saliency	0.188	0.416	0.353
SplitCAM			
SplitCAM (sc, α 0.4) (12, g)	0.736	0.729	0.202
SplitCAM (sc, α 0.4) (14, g)	0.724	0.194	0.174
SplitCAM (sc, α 0.4, wta) (14, g)	0.724	0.389	0.303
SplitCAM (sc, α 0.4, wta) (26, +, g) abs	0.728	0.342	0.319
SplitCAM (sc, α 0.5, wta) (14, g)	0.728	0.096	0.292
SplitCAM (sc, α 0.5, wta) (26, +, g)	0.804	0.186	0.148
SplitGrad			
SplitGrad (α 0.4) (12, +, g)	0.677	0.711	0.174
SplitGrad (α 0.4) (14, +, g) abs	0.704	0.197	0.091
SplitGrad (α 0.4, wta) (12, +, g)	0.706	0.454	0.174
SplitGrad (α 0.4, wta) (14, +, g) abs	0.721	0.211	0.199
SplitGrad (α 0.5, wta) (12, +, g) abs	0.725	0.238	0.206
SplitGrad (α 0.5, wta) (26, +, g) abs	0.795	0.149	0.306
SplitLRP			
SplitLRP (7, pos)	0.997	1.000	0.471
SplitLRP (19, pos)	0.998	1.000	0.562
SplitLRP (wta) (7, pos)	0.998	1.000	0.456
SplitLRP (wta) (26, pos) abs	0.994	0.999	0.446

Tab. S.4.4: Mean attribution time in seconds per image across the custom 50-image validation subset of ImageNet-S. Model Splitting Time is instead seconds per setup, averaged over 20 procedures and excluded from the attribution times. Timer scope and limits of the historical run metadata are described in Section S.4.1.5.

Method	**ResNet-18**	**VGG-16**
Model Splitting Time	0.0331	0.0295
Split Methods		
SplitCAM	0.4368	0.3348
SplitGrad	0.4284	0.3309
SplitLRP	0.5711	0.3204
Classical Methods		
GCAM	0.0216	0.0232
GCAM++	0.0157	0.0155
LCAM	0.0149	0.0124
Guided GCAM	0.3102	0.2808
LRP $\gamma = 0.25$	0.0599	0.0507
DeepLIFT	0.0203	0.0210
IntGrad	0.0070	0.0121
GradientSHAP	0.0084	0.0082
Guided Backprop	0.0253	0.0258
Deconvolution	0.0087	0.0074
Saliency	0.0060	0.0047
Input $\times$ Gradient	0.0055	0.0046
Feature Ablation	231.6631	192.2602
Occlusion	238.2302	201.0051

S.4.2 Stabilization Ablations for the Split Methods

This section presents the ablation studies examining the effect of the proposed stabilization techniques for the forward and backward pass, as well as the influence of the selected max-pooling implementation (Section 3.6).

S.4.2.1 Backward-Pass Stabilization

Table S.4.5 reports the full α-ablation for SplitCAM and SplitGrad across the selected configurations, extending the main-text Table 7.3. At layer 26, changing α from 0.5 to 0.4 improves both localization scores for each listed configuration, but the Pixel-Flipping score-drop AUC increases for SplitGrad and decreases for SplitCAM. The layer-14 configurations have smaller or adverse changes in some metrics. Low α often reduces Maximum Sensitivity while also reducing localization or Pixel-Flipping performance. Since α changes the backward attribution rule, these sweeps do not isolate numerical stabilization from the choice of explanation. The table also compares convex and winner-takes-it-all max-pooling: their differences are small near $\alpha = 0.5$ and become more pronounced at lower values, with their ranking depending on the configuration and metric.

Unresolved source-table discrepancy. For SplitCAM with scaling, $\alpha = 0.4$, winner-takes-it-all pooling and layer configuration $(26, +, g)$ with absolute values, Table S.4.1 reports $\mathrm{PF}_{20}^{\mathrm{drop}} = 1.590$ and MaxS= 0.456, whereas the corresponding row of Table S.4.5 reports 1.5849 and 0.4573. These values are inherited from the source tables. The released artifacts do not provide the underlying per-image results needed to resolve the discrepancy, so the values are retained without attributing the difference to a particular cause.

S.4.2.2 Forward-Pass Stabilization

This subsection investigates the influence of the forward stabilization on SplitLRP for the framework-selected layer configurations, and on SplitCAM for manually selected layer configurations. As a baseline, a shifting mode is introduced that shifts the activations of positive neurons to half of the respective original activation and those of negative neurons to the negative half. A SplitCAM variant without any forward stabilization exhibits an activation explosion in late VGG-16 layers on the order of 10^{18}, resulting in numerical errors on the order of 10^{9} despite per-layer correction; for SplitLRP, the absence of forward stabilization produced nearly entirely white saliency maps with only small, random, semantically meaningless artifacts, so no quantitative metrics were computed for that setting.

Table S.4.7 summarizes all SplitCAM configurations *without* forward-pass stabilization, and Table S.4.8 the results *with* the shifting-based stabilization; both mirror the configurations of Table S.4.5. Table S.4.6 directly compares SplitLRP with shifting versus scaling stabilization. Scaling has better values for all reported metrics in the two layer-7 configurations. At the later layers, shifting gives higher Attribution Localization and Pixel-Flipping score-drop AUC, while scaling gives lower Maximum Sensitivity; the Pointing Game comparison changes direction between the two configurations. The evidence therefore supports a configuration-dependent trade-off between the two forward rules.

Tab. S.4.5: Alpha ablation as performed in Section 7.4 across four SplitGrad and SplitCAM configurations of layers 14 and 26, respectively. The notation is similar to Table 7.1.

Method	Metric	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
	PG↑	0.8975	0.9064	0.9470	**0.9558**	0.8887	0.4011
SplitGrad (wta)	AL↑	0.5434	0.5476	0.6051	**0.6169**	0.5346	0.4486
(26, g), abs	MaxS↓	0.6160	0.6071	0.4399	0.2382	0.3344	**0.0283**
	PF_{20}^{drop} ↑	1.5270	1.5337	**1.5808**	1.4476	1.3171	1.2471
	PG↑	0.8975	0.9064	**0.9470**	0.9099	0.2827	0.6413
SplitGrad	AL↑	0.5434	0.5476	**0.6037**	0.5216	0.4245	0.4563
(26, g), abs	MaxS↓	0.6171	0.6102	0.4426	0.3864	0.0622	**0.0079**
	PF_{20}^{drop} ↑	1.5272	1.5333	**1.5876**	1.3702	0.2431	0.5755
	PG↑	0.8198	0.8198	0.8198	**0.8322**	0.8127	0.7898
SplitGrad (wta)	AL↑	**0.5403**	0.5403	0.5402	0.5006	0.4936	0.4857
(14, g), abs	MaxS↓	0.8579	0.8597	0.8584	0.0875	0.0677	**0.0481**
	PF_{20}^{drop} ↑	2.1887	**2.1894**	2.1869	1.8740	1.8304	1.7684
	PG↑	0.8198	**0.8198**	0.8180	0.7951	0.8004	0.7968
SplitGrad	AL↑	0.5403	**0.5403**	0.5330	0.5054	0.4926	0.4847
(14, g), abs	MaxS↓	0.8618	0.8584	0.8478	0.1103	0.0662	**0.0301**
	PF_{20}^{drop} ↑	**2.1894**	2.1891	2.1185	1.3425	1.2959	1.1927
	PG↑	0.8675	0.8675	**0.9382**	0.9329	0.6131	0.8551
SplitCAM (sc, wta)	AL↑	0.5501	0.5535	0.6347	**0.6627**	0.4033	0.5133
(26, +, g), abs	MaxS↓	0.6906	0.6820	0.4573	0.2645	0.2511	**0.1196**
	PF_{20}^{drop} ↑	1.6067	**1.6085**	1.5849	1.4989	0.8534	1.5071
	PG↑	0.8940	0.8905	**0.9346**	0.6466	0.8251	0.8604
SplitCAM (sc)	AL↑	0.5595	0.5636	**0.6363**	0.4015	0.4977	0.5114
(26, +, g), abs	MaxS↓	0.6738	0.6640	0.4520	0.3729	0.1102	**0.1062**
	PF_{20}^{drop} ↑	1.5902	**1.5978**	1.5906	0.9076	1.2784	1.3605
	PG↑	0.8251	0.8251	**0.8269**	0.7880	0.7809	0.7703
SplitCAM (sc, wta)	AL↑	0.5363	0.5363	**0.5435**	0.5338	0.5286	0.5225
(14, g)	MaxS↓	1.0566	1.0569	1.0489	**0.1620**	0.1625	0.1630
	PF_{20}^{drop} ↑	2.4128	2.4130	**2.4135**	1.9014	1.8613	1.8731
	PG↑	0.8251	0.8251	**0.8428**	0.7756	0.7686	0.7756
SplitCAM (sc)	AL↑	0.5363	0.5363	**0.5626**	0.5363	0.5271	0.5208
(14, g)	MaxS↓	1.0552	1.0593	1.0136	0.1629	0.1585	**0.1555**
	PF_{20}^{drop} ↑	2.4171	**2.4172**	2.4065	1.5032	1.5082	1.5139

Tab. S.4.6: Scale (sc) vs. shift (sh) forward-pass stabilization for SplitLRP methods on VGG-16.

Method	Type	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitLRP (wta), 7	sc	**3.235**	**-0.036**	**0.836**	**0.296**	**2.291**	**0.866**
	sh	3.619	-0.610	0.698	0.260	2.070	0.914
SplitLRP (wta), 26, abs	sc	**5.168**	0.588	0.871	0.151	1.473	**0.282**
	sh	5.168	**0.628**	**0.877**	**0.161**	**1.478**	0.468
SplitLRP, 7	sc	**3.235**	**-0.036**	**0.836**	**0.296**	**2.291**	**0.866**
	sh	3.619	-0.695	0.698	0.260	2.070	0.914
SplitLRP, 19	sc	5.168	0.588	**0.871**	0.151	1.473	**0.282**
	sh	**4.955**	**0.629**	0.853	**0.197**	**1.692**	0.507

Tab. S.4.7: Alpha ablation for scale normalization across four SplitCAM configurations of layers 14 and 26. Here, in the forward pass *no* stabilization is applied. Notation and structure are similar to Table S.4.5. Light blue entries marked with * indicate better performance of the method compared to the default variant with scale stabilization in the forward pass.

Method	**Metric**	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
SplitCAM (sc, wta) (26, +, g), abs	PG↑	0.7686	0.7686	0.8632	**0.8712**	0.3441	0.5453
	AL↑	0.5374	0.5412	0.5943	**0.6118**	0.3595	0.4569
	MaxS↓	0.6578*	0.6507*	0.4841	0.2873	0.1918*	**0.0135***
	PF_{20}^{drop} ↑	1.2744	1.2859	**1.3209**	1.1792	0.4430	0.6527
SplitCAM (sc) (26, +, g), abs	PG↑	0.7565	0.7565	**0.8451**	0.3783	0.4507	0.4970
	AL↑	0.5346	0.5384	**0.5897**	0.3663	0.4420	0.4545
	MaxS↓	0.6604*	0.6512*	0.4985	0.3077*	0.0279*	**0.0043***
	PF_{20}^{drop} ↑	1.2680	1.2856	**1.3295**	0.5912	0.4906	0.5563
SplitCAM (sc, wta) (14, g)	PG↑	**0.8491***	0.8491*	0.8491*	0.7867	0.7847*	0.7686
	AL↑	0.5324	0.5321	**0.5406**	0.5306	0.5261	0.5200
	MaxS↓	1.0834	1.0814	1.0885	**0.2271**	0.2309	0.2384
	PF_{20}^{drop} ↑	2.4454*	2.4482*	**2.4486***	1.9588*	1.9193*	1.9349*
SplitCAM (sc) (14, g)	PG↑	0.7746	0.7767	**0.7847**	0.5855	0.5594	0.5573
	AL↑	0.4003	0.4144	**0.5517**	0.5300	0.5209	0.5137
	MaxS↓	1.5733	1.5627	1.5443	**1.2661**	1.3939	1.4272
	PF_{20}^{drop} ↑	2.1298	**2.1384**	2.1371	1.6669*	1.6793*	1.6740*

Tab. S.4.8: Alpha ablation for scaling normalization across four SplitCAM configurations of layers 14 and 26. Here, in the forward pass shift normalization is applied. Notation and structure are similar to Table S.4.5. Light blue entries marked with * indicate better performance of the method compared to the default variant with scale stabilization in the forward pass.

Method	**Metric**	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
	PG↑	0.8330	0.8149	0.7968	**0.9235**	0.6036	0.8592*
SplitCAM (sc, wta)	AL↑	0.5242	0.5203	0.5147	**0.6549**	0.3968	0.5093
(26, +, g), abs	MaxS↓	0.6180*	0.6182*	0.6509	0.3104	0.2791	**0.1665**
	PF_{20}^{drop} ↑	**1.6804***	1.6646*	1.5065	1.4962	0.8896*	1.5405*
	PG↑	0.8330	0.8149	0.7988	0.5654	0.8431*	**0.8652***
SplitCAM (sc)	AL↑	**0.5242**	0.5203	0.5076	0.3736	0.4949	0.5094
(26, +, g), abs	MaxS↓	0.6129*	0.6168*	0.6494	0.3820	0.1686	**0.1664**
	PF_{20}^{drop} ↑	**1.6804***	1.6646*	1.5105	0.8714	1.4289*	1.4765*
	PG↑	**0.8310***	0.8310*	0.8310*	0.7887*	0.7827*	0.7726*
SplitCAM (sc, wta)	AL↑	0.5312	0.5312	**0.5376**	0.5315	0.5264	0.5203
(14, g)	MaxS↓	1.0562*	1.0521*	1.0649	**0.1633**	0.1641	0.1642
	PF_{20}^{drop} ↑	**2.4412***	2.4412*	2.4400*	1.9306*	1.8953*	1.9059*
	PG↑	0.8310*	0.8310*	**0.8431***	0.7746	0.7706*	0.7807*
SplitCAM (sc)	AL↑	0.5311	0.5312	**0.5580**	0.5337	0.5245	0.5183
(14, g)	MaxS↓	1.0554	1.0568*	1.0198	0.1641	0.1587	**0.1570**
	PF_{20}^{drop} ↑	**2.4412***	2.4411*	2.4318*	1.5187*	1.5222*	1.5329*

S.4.3 Implementation and Evaluation Details

Data splits, explanation targets, metric definitions, configuration selection and statistical tests are specified in the thesis, Section A.2.

Backbone and dataset provenance. All game-based experiments use the pretrained torchvision checkpoints shipped with the library release used for evaluation. Specifically, VGG-16 uses `VGG16_Weights.IMAGENET1K_V1` (https://download.pytorch.org/models/vgg16-397923af.pth; recipe https://github.com/pytorch/vision/tree/main/references/classification#alexnet-and-vgg), ResNet-50 uses `ResNet50_Weights.IMAGENET1K_V2` (https://download.pytorch.org/models/resnet50-11ad3fa6.pth; recipe https://github.com/pytorch/vision/issues/3995#issuecomment-1013906621), and ViT-B/16 uses `ViT_B_16_Weights.IMAGENET1K_V1` (https://download.pytorch.org/models/vit_b_16-c867db91.pth; recipe https://github.com/pytorch/vision/tree/main/references/classification#vit_b_16). These are ImageNet-1K classifiers trained on the ImageNet / ILSVRC benchmark (Deng et al., 2009; Russakovsky et al., 2015). Quantitative scoring is performed with Quantus (Hedström et al., 2023).

Licenses. ImageNet images are used under the ImageNet terms of access (non-commercial research and educational use) (Deng et al., 2009). The ImageNet-S masks (Gao et al., 2023) are released by their authors without an explicit license; their images are ImageNet images and fall under the same terms. The Pascal VOC 2012 images (Everingham et al., 2009) are subject to the Flickr terms of use, as stated by the dataset providers. The checkpoints are distributed with torchvision (BSD-3-Clause) and were trained on ImageNet-1K. Libraries: PyTorch and torchvision (BSD-3-Clause), Quantus (GPL-3.0), zennit (GPL-3.0/LGPL-3.0), lxt, captum and scikit-image (BSD-3-Clause), pytorch-grad-cam (MIT); the reference code of TIBAV and GAE (MIT) is included with its license notice.

S.4.3.1 Methods

Gradient-based block. **Gradient** is the raw input-gradient saliency map (Simonyan et al., 2014), the maximum over color channels of the absolute gradient. **SmGrad** is SmoothGrad, i.e. an average over noisy gradient maps (Smilkov et al., 2017). **IntGrad** is Integrated Gradients (Sundararajan et al., 2017) with a black-image baseline. **DeepLIFT** propagates activation differences relative to a reference input with the Rescale rule (Shrikumar et al., 2017) (CNNs only). **DAVE** is the distribution-aware ViT gradient decomposition of Wróbel et al. (2026). **SG** is our SG, included in this block because it recovers the gradient as a Stopping-Game occupation measure (Theorem S.1.6).

LRP / relevance-propagation block. **LRP-ε** denotes the stabilized Layer-wise Relevance Propagation rule (Bach et al., 2015). **LRP-γ** is the positive-weight-amplified LRP variant used in best-practice comparisons (Kohlbrenner et al., 2020). **AttnLRP** is transformer LRP that propagates relevance through the softmax and the bilinear attention product as well as the Value path (Achtibat et al., 2024). **RG** is our RG. **RG+Eq** is the same method with equivariant Reynolds averaging, used mainly on ViT to suppress tokenization-grid artifacts. **TIBAV** (Chefer et al., 2021b), **GAE** (Chefer et al., 2021a) and Attention Rollout (**AR**) are the transformer-specific attention-map baselines.

CAM block. **GCAM** is Grad-CAM (Selvaraju et al., 2017), **GCAM++** is Grad-CAM++ (Chattopadhay et al., 2018), and **LCAM** is LayerCAM, which weights class-activation evidence per location (Jiang et al., 2021). All three are computed at the last convolutional block of the CNNs and upsampled bilinearly.

Perturbation block. **LIME** is Local Interpretable Model-agnostic Explanations (Ribeiro et al., 2016). **KernelSHAP** is the model-agnostic weighted-regression estimator of SHAP values (Lundberg and Lee, 2017). Both perturb SLIC superpixels (Achanta et al., 2012). These methods explain predictions by perturbing the input and fitting a local surrogate, so they are separated from the backward-propagation methods above.

S.4.3.2 Declared validation grids

Table S.4.9 lists the grid of every method. Functionally duplicate rows (e.g. different σ^2 when $\lambda = \lambda_{\text{sm}} = 0$) are evaluated once; in total 2,021 validation configurations are evaluated. On ViT-B/16 the activation-side λ is fixed to 0, which keeps GELU gates conditioned (Section 5.4). At attention–Value products, RG applies $\alpha\beta$-LRP-ε to the composite map of the attention product and the Value projection (Theorem S.2.3; Section 5.5.3). Baselines without a declared grid are run at the settings of their reference implementations.

Tab. S.4.9: Per-method validation grids (all architectures unless stated). Symbols: Section S.4.4.

Method	Grid
Gradient	single config
SmGrad	N=50, noise level $\sigma/(x_{\max}-x_{\min}) \in \{0.1, 0.15, 0.2\}$, mean aggregation (3 configs)
IntGrad	black-image baseline, $N_{\text{steps}} \in \{20, 50, 100, 300\}$ (4 configs)
DeepLIFT	Rescale rule, single config (CNNs only)
DAVE	$N \in \{25, 50\} \times \theta_{\max} \in \{0, 10, 20\} \times s_{\max} \in \{0, 0.05, 0.1\} \times t_{\max} \in \{0, 0.25, 0.5\}$, horizontal flips with probability 0.5, averaged effective map multiplied by the input (54 configs; ViT-B/16 only)
SG	$\lambda \in \{-5, -2, -1, -0.3, 0\} \times \sigma^2 \in \{1, 2, 5, 10\}\times$ noisy gate $\tilde{\Phi} \in \{\text{off}, \text{on}\}$ (40 configs; VGG-16 only)
LRP-ε	$\varepsilon \in \{0.01, 0.05, 0.1, 0.25, 0.5, 1, 2\}$ (7 configs)
LRP-γ	$(\varepsilon, \gamma) \in \{0.1, 0.25, 0.5\} \times \{0.1, 0.25, 0.5, 1, 2\}$; ViT-B/16: γ on all Linear layers (15 configs); CNNs: both zennit's EpsilonGammaBox composite ($z^{\mathscr{B}}$ input layer, γ on convolutions, ε on dense layers) and γ on all layers (30 configs)
AttnLRP	γ-composite of Achtibat et al. (2024): reference setting $(\gamma_{\text{conv}}, \gamma_{\text{lin}}, \varepsilon_{\text{attn}})=(0.25, 0.05, 10^{-6})$ + univariate sweeps $\gamma_{\text{lin}} \in \{0.01, 0.025, 0.1, 0.25\}$, $\gamma_{\text{conv}} \in \{0.1, 0.5, 1\}$, $\varepsilon_{\text{attn}} \in \{0.01, 0.05, 0.25\}$ (11 configs; ViT-B/16 only)
TIBAV	single config (ViT-B/16 only)
GAE	single config (ViT-B/16 only)
AR	residual weight $w_r \in \{0, 0.25, 0.5, 0.75\}$ (4 configs; ViT-B/16 only)
RG	$\tau \in \{0.3, 0.4, \ldots, 1.0\}$, $(\alpha,\beta)=(2,1)$, $\sigma^2 \in \{1, 2, 5, 10\}$, $\varepsilon \in \{0.1, 0.25, 0.5, 1\}$; ViT-B/16: λ=0, $\lambda_{\text{sm}} \in \{-50, -10, -5, -2, -1, 0\}$, composite Value rule of Theorem S.2.3 (672 configs); CNNs: $\lambda \in \{-5, -2, -1, -0.3, 0\}$ (544 configs)
RG+Eq	$\tau \in \{0.3, 0.4, \ldots, 1.0\} \times \lambda_{\text{sm}} \in \{0, -10\}$, σ^2=2, ε=0.5, N=50, composite Value rule (16 configs; ViT-B/16 only)
GCAM, GCAM++, LCAM	last convolutional block (VGG-16: `features.29`, ResNet-50: `layer4`), single config (CNNs only)
LIME	80 SLIC superpixels, N=1000, ridge surrogate, exponential kernel of width 0.25 on the cosine distance of the binary superpixel vectors, removed superpixels filled with their mean color; fixed reference row
KernelSHAP	80 SLIC superpixels, N=2M+2048 for M superpixels, removed superpixels filled with the image mean; fixed reference row
Anchor	$\alpha\beta$-LRP anchor $(\alpha,\beta,\varepsilon,\tau)$=(2,1,0.5,1) of RG and RG+Eq, reported as the theorem-driven reference row (not selected)

S.4.3.3 Compute resources

The attribution evaluations are inference-only. The main cost is the repeated backward attribution pass over the validation and test images across the method grids, including the equivariant-Reynolds and noise-averaged configurations. Runtime measurements were taken on a single NVIDIA A100-SXM4-40GB with batch size 1; evaluation sweeps were distributed over several GPUs of the same type, with up to two evaluation processes per GPU. The full pipeline (validation grids, test evaluation, robustness, Pascal VOC, ablations and randomization diagnostics) required about 450 GPU-hours.

S.4.3.4 Code and correctness tests

The code (https://github.com/JayPiZimmermann/Playing_The_Network_Backward_Official_Repository) contains unit tests that pin the implemented backward rules to the statements of the thesis and supplement. Two of them concern the attention block.

`tests/test_attention_ab_recovery.py` runs the RG hooks at $\tau = 1$ on a small attention block with signed inputs, a Value bias and $\varepsilon > 0$, in float64, and compares the result with an independent implementation of $\alpha\beta$-LRP-ε on the composite map $X \mapsto A(XW_v^\top + b_v)$, i.e. the target rule (5.21) of Theorem S.2.3, for a single head and for a multi-head block in the layout of `nn.MultiheadAttention`; it also asserts that the result differs from the sequential variant of §5.5.1, which the released code does not offer (`av_rule` accepts only `'merged'`). `tests/test_hellinger_merged_attention.py` enumerates every backward trajectory of a small attention block under two models, at $\tau \in \{0.5, 1, 2\}$, and checks the per-input masses, the Bhattacharyya mass and the cemetery masses of the Hellinger pass (Section 8.3) against this brute-force reference; a pre-norm residual block built from `nn.MultiheadAttention` must give the same distance as the same block with its attention matrix supplied as a constant.

S.4.4 Additional Results

The validation, test and VOC tables of this section are generated by one pipeline from the declared grids and the selection rules of Section A.2.

Configuration symbols. The configuration columns of all tables use the following parameters. RG, RG+Eq and SG (Table 4.1): temperature τ, activation-side risk aversion λ, attention-logit risk aversion λ_{sm}, ADF input variance σ^2, LRP stabilizer ε, stream weights (α,β) (default (2,1)), noisy-observation gate $\tilde{\Phi}$ ("gate on/off"), and the number N of transformed inputs averaged by RG+Eq. SmoothGrad: N noisy samples with noise standard deviation σ relative to the input range (σ/range). Integrated Gradients: N_{steps} integration steps from a black-image baseline. DAVE: N transformed inputs with rotations up to $\theta_{\max}$ degrees, shifts up to $s_{\max}$ of the image extent, and variance-preserving Gaussian noise of interpolation weight up to $t_{\max}$. LRP-ε and LRP-γ: stabilizer ε and positive-weight boost γ; on the CNNs, LRP-γ is zennit's composite with the $z^{\mathscr{B}}$ rule at the input layer. AttnLRP: γ-rule parameters γ_{conv} and γ_{lin} of the convolutional and linear layers and the stabilizer $\varepsilon_{\text{attn}}$ of the attention layers. AR (attention rollout): residual weight w_r. LIME and KernelSHAP: N perturbation samples ($2M$+2048 for M superpixels) on 80 SLIC superpixels; LIME fits a ridge surrogate. DeepLIFT: Rescale rule. Grad-CAM, Grad-CAM++ and LayerCAM: the target layer (torchvision module name). Row labels in parentheses give the selection criterion (`loc`, `faith`); single-configuration methods have no criterion.

S.4.4.1 Validation selections

Tables S.4.10–S.4.12 list, for every method, the configurations selected on the 50-image validation split under the `loc` and `faith` criteria of Section A.2.3, together with both validation rank-sums. For fixed configurations there is no selection comparison, so the rank-sums are marked n/a.

Tab. S.4.10: ViT-B/16: validation-selected configurations per method and criterion (`loc`: minimal rank-sum over AL, PG, TK; `faith`: over PF_5, PF_{20}, Sel; rank-sums over the configurations of one method, lower is better; the number of configurations is given after the method name). Symbols: Section S.4.4.

Method	Criterion	Selected configuration	Loc. rank-sum	Faith. rank-sum
Gradient (1)	fixed	fixed	n/a	n/a
SmGrad (3)	loc	N=50,σ/range=0.2	3.5	3
	faith	N=50,σ/range=0.2	3.5	3
IntGrad (4)	loc	N_{steps}=100, black baseline	3.5	4
	faith	N_{steps}=100, black baseline	3.5	4
LRP-ε (7)	loc	ε=0.25	5.5	7
	faith	ε=0.1	6.5	3
LRP-γ (15)	loc	ε=0.1,γ=2	6	25
	faith	ε=0.1,γ=0.1	32	12
AttnLRP (11)	loc	γ_{conv}=0.25,γ_{lin}=0.05,$\varepsilon_{\text{attn}}$=0.25	3	29
	faith	γ_{conv}=0.25,γ_{lin}=0.05,$\varepsilon_{\text{attn}}$=0.01	14	7
DAVE (54)	loc	N=50,$\theta_{\max}$=0,$s_{\max}$=0,$t_{\max}$=0.5	14	6
	faith	N=50,$\theta_{\max}$=0,$s_{\max}$=0,$t_{\max}$=0.5	14	6
TIBAV (1)	fixed	fixed	n/a	n/a
GAE (1)	fixed	fixed	n/a	n/a
AR (4)	loc	w_r=0.75	3.5	3
	faith	w_r=0.75	3.5	3
LIME (1)	fixed	N=1000, 80 superpixels, ridge	n/a	n/a
KernelSHAP (1)	fixed	N=2M+2048, 80 superpixels	n/a	n/a
RG (672)	loc	τ=0.4,λ_{sm}=$-$10,σ^2=5,ε=0.1	47	788
	faith	τ=0.7,λ_{sm}=$-$5,σ^2=5,ε=0.1	1156	22
	anchor	τ=1	1420	344
RG+Eq (16)	loc	τ=0.5,λ_{sm}=$-$10,σ^2=2,N=50	6.5	29
	faith	τ=0.7,λ_{sm}=$-$10,σ^2=2,N=50	14	11
	anchor	τ=1,N=50	47	31

Tab. S.4.11: VGG-16: validation-selected configurations per method and criterion, as in Table S.4.10.

Method	Criterion	Selected configuration	Loc. rank-sum	Faith. rank-sum
Gradient (1)	fixed	fixed	n/a	n/a
SmGrad (3)	loc	N=50,σ/range=0.1	3	3
	faith	N=50,σ/range=0.1	3	3
IntGrad (4)	loc	N_{steps}=20, black baseline	6	3
	faith	N_{steps}=20, black baseline	6	3
DeepLIFT (1)	fixed	Rescale	n/a	n/a
LRP-ε (7)	loc	ε=0.5	6	9
	faith	ε=1	6	4
LRP-γ (30)	loc	ε=0.5,γ=0.25, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	13	43
	faith	ε=0.25,γ=0.1	36	5
GCAM (1)	fixed	`features.29`	n/a	n/a
GCAM++ (1)	fixed	`features.29`	n/a	n/a
LCAM (1)	fixed	`features.29`	n/a	n/a
LIME (1)	fixed	N=1000, 80 superpixels, ridge	n/a	n/a
KernelSHAP (1)	fixed	N=2M+2048, 80 superpixels	n/a	n/a
SG (40)	loc	λ=-1,σ^2=5, gate on	37	10
	faith	λ=-0.3,σ^2=1, gate off	49.5	6
RG (544)	loc	τ=0.3,ε=0.1	78	194
	faith	τ=0.8,ε=0.1	173.5	5
	anchor	τ=1	339	60

Tab. S.4.12: ResNet-50: validation-selected configurations per method and criterion, as in Table S.4.10.

Method	Criterion	Selected configuration	Loc. rank-sum	Faith. rank-sum
Gradient (1)	fixed	fixed	n/a	n/a
SmGrad (3)	loc	N=50,σ/range=0.15	4	5
	faith	N=50,σ/range=0.1	8	4
IntGrad (4)	loc	N_{steps}=100, black baseline	3	6
	faith	N_{steps}=50, black baseline	7.5	6
DeepLIFT (1)	fixed	Rescale	n/a	n/a
LRP-ε (7)	loc	ε=0.5	5.5	9
	faith	ε=2	8	4
LRP-γ (30)	loc	ε=0.5,γ=2, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	12.5	33
	faith	ε=0.5,γ=0.5, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	36	22.5
GCAM (1)	fixed	`layer4`	n/a	n/a
GCAM++ (1)	fixed	`layer4`	n/a	n/a
LCAM (1)	fixed	`layer4`	n/a	n/a
LIME (1)	fixed	N=1000, 80 superpixels, ridge	n/a	n/a
KernelSHAP (1)	fixed	N=2M+2048, 80 superpixels	n/a	n/a
RG (544)	loc	τ=0.9,ε=1	19	224
	faith	τ=1,λ=−1,σ^2=1,ε=0.25	466	38
	anchor	τ=1	212.5	140

S.4.4.2 Test results with confidence intervals

Tables S.4.13–S.4.15 list every row evaluated on the 565-image test split (the `loc`- and `faith`-selected configuration of every method, the anchors and the single-configuration methods) with 95% bootstrap confidence intervals and the tests of Section A.2.4 (bold and dark shading: best row of a column; ≈ and light shading: not significantly different from it). Per-image runtimes come from the single-GPU benchmark of Section S.4.3.3.

Tab. S.4.13: ViT-B/16, ImageNet-S test split: all rows with 95% bootstrap confidence intervals. Metrics as in Table 7.4; configuration symbols and row labels: Section S.4.4.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.411	0.361	0.386	8.45	8.08	6.033	1.290	0.922
		[0.389,0.432]	[0.322,0.402]	[0.359,0.413]	[8.33,8.56]	[7.94,8.22]	[5.905,6.154]	[1.233,1.351]	[0.893,0.953]
SmGrad (loc/faith)	N=50,σ/range=0.2	0.517	0.706	0.675	8.10	7.02	5.029	0.597	0.468
		[0.498,0.536]	[0.667,0.743]	[0.654,0.694]	[7.96,8.24]	[6.83,7.20]	[4.894,5.163]	[0.569,0.630]	[0.451,0.485]
IntGrad (loc/faith)	N_{steps}=100,black baseline	0.458	0.513	0.498	8.37	7.93	5.712	1.090	0.825
		[0.437,0.479]	[0.473,0.554]	[0.472,0.524]	[8.24,8.48]	[7.78,8.08]	[5.577,5.842]	[1.041,1.140]	[0.796,0.855]
LRP-ε (loc)	ε=0.25	0.592	0.834	0.746	7.97	6.88	4.859	0.486	0.384
		[0.575,0.610]	[0.804,0.864]	[0.728,0.764]	[7.81,8.11]	[6.69,7.07]	[4.733,4.983]	[0.468,0.505]	[0.372,0.395]
LRP-ε (faith)	ε=0.1	0.582	0.834	0.748	7.84	6.76	4.890	0.623	0.498
		[0.564,0.599]	[0.804,0.864]	[0.730,0.765]	[7.68,7.99]	[6.57,6.96]	[4.763,5.015]	[0.597,0.650]	[0.482,0.515]
LRP-γ (loc)	ε=0.1,γ=2	0.644	0.873	0.816≈	7.93	6.64	4.451≈	0.300	0.218
		[0.628,0.660]	[0.844,0.899]	[0.800,0.832]	[7.78,8.08]	[6.44,6.83]	[4.324,4.572]	[0.287,0.314]	[0.210,0.226]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
LRP-γ (faith)	ε=0.1,γ=0.1	0.614 [0.597,0.630]	0.844 [0.814,0.873]	0.779 [0.763,0.795]	7.73 [7.56,7.89]	6.40 [6.20,6.60]	4.603 [4.478,4.724]	0.394 [0.378,0.410]	0.309 [0.299,0.319]
AttnLRP (loc)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.25	0.644 [0.628,0.660]	0.855 [0.825,0.883]	0.809 [0.794,0.825]	7.77 [7.61,7.93]	6.43 [6.23,6.62]	4.582 [4.452,4.706]	0.456 [0.430,0.484]	0.342 [0.324,0.364]
AttnLRP (faith)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.01	0.629 [0.613,0.645]	0.850 [0.819,0.878]	0.798 [0.783,0.814]	**7.64** [7.47,7.81]	**6.12** [5.92,6.32]	**4.441** [4.314,4.563]	0.416 [0.395,0.438]	0.317 [0.304,0.331]
DAVE (loc/faith)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.5	0.491 [0.472,0.510]	0.717 [0.678,0.752]	0.612 [0.591,0.633]	8.12 [7.98,8.25]	7.37 [7.19,7.54]	5.386 [5.256,5.514]	0.781 [0.755,0.810]	0.643 [0.626,0.661]
TIBAV	fixed	0.615 [0.597,0.634]	0.878 [0.851,0.904]	0.782 [0.759,0.805]	8.13 [8.01,8.24]	7.60 [7.46,7.74]	4.906 [4.774,5.033]	1.062 [1.001,1.126]	0.512 [0.486,0.540]
GAE	fixed	0.533 [0.514,0.551]	0.743 [0.708,0.779]	0.649 [0.623,0.677]	8.13 [8.01,8.25]	7.62 [7.47,7.76]	5.098 [4.961,5.231]	0.525 [0.494,0.558]	0.368 [0.346,0.391]
AR (loc/faith)	w_r=0.75	0.447 [0.428,0.467]	0.565 [0.522,0.605]	0.537 [0.510,0.564]	8.44 [8.34,8.53]	8.06 [7.94,8.17]	5.254 [5.128,5.374]	0.208 [0.195,0.222]	0.122≈ [0.116,0.128]
LIME	N=1000,80 superpixels,ridge	0.535 [0.515,0.553]	0.897 [0.871,0.922]	0.686 [0.656,0.716]	8.36 [8.25,8.46]	7.68 [7.54,7.82]	5.376 [5.242,5.505]	0.937 [0.905,0.972]	0.668 [0.647,0.689]
KernelSHAP	N=2M+2048,80 superpixels	0.565 [0.548,0.583]	**0.961** [0.945,0.977]	0.823≈ [0.799,0.848]	8.29 [8.18,8.40]	7.50 [7.35,7.65]	5.140 [5.012,5.261]	0.897 [0.868,0.928]	0.650 [0.632,0.667]
RG anchor	τ=1	0.509 [0.490,0.528]	0.724 [0.685,0.759]	0.699 [0.678,0.720]	8.19 [8.05,8.31]	6.97 [6.78,7.15]	4.854 [4.732,4.972]	**0.166** [0.161,0.172]	0.120≈ [0.117,0.124]
RG (loc)	τ=0.4,λ_{sm}=−10, σ^2=5,ε=0.1	**0.711** [0.692,0.730]	0.825 [0.793,0.855]	0.788 [0.768,0.808]	8.27 [8.14,8.39]	7.48 [7.31,7.64]	5.109 [4.987,5.226]	0.729 [0.685,0.777]	0.494 [0.471,0.518]
RG (faith)	τ=0.7,λ_{sm}=−5, σ^2=5,ε=0.1	0.549 [0.531,0.567]	0.770 [0.736,0.804]	0.741 [0.721,0.760]	8.11 [7.97,8.24]	6.78 [6.59,6.97]	4.808 [4.686,4.925]	0.257 [0.245,0.268]	0.174 [0.168,0.179]
RG+Eq anchor	τ=1,N=50	0.520 [0.501,0.539]	0.761 [0.726,0.796]	0.752 [0.729,0.775]	8.41 [8.29,8.52]	7.72 [7.56,7.87]	4.864 [4.742,4.981]	0.170≈ [0.165,0.175]	**0.115** [0.112,0.118]
RG+Eq (loc)	τ=0.5,λ_{sm}=−10, σ^2=2,N=50	0.698 [0.680,0.717]	0.855 [0.827,0.883]	**0.836** [0.815,0.857]	8.39 [8.28,8.50]	7.80 [7.65,7.94]	5.010 [4.889,5.127]	0.441 [0.421,0.463]	0.301 [0.290,0.313]
RG+Eq (faith)	τ=0.7,λ_{sm}=−10, σ^2=2,N=50	0.654 [0.636,0.672]	0.812 [0.781,0.844]	0.820 [0.799,0.841]	8.39 [8.28,8.51]	7.71 [7.56,7.86]	4.856 [4.735,4.974]	0.310 [0.297,0.325]	0.199 [0.194,0.206]

Tab. S.4.14: VGG-16, ImageNet-S test split: all rows with 95% bootstrap confidence intervals, as in Table S.4.13.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.522 [0.501,0.542]	0.738 [0.701,0.773]	0.688 [0.665,0.711]	14.47 [13.95,14.99]	10.96 [10.55,11.37]	6.470 [6.212,6.731]	1.212 [1.162,1.269]	0.947 [0.917,0.980]
SmGrad (loc/faith)	N=50,σ/range=0.1	0.533 [0.515,0.552]	0.782 [0.747,0.816]	0.723 [0.705,0.741]	14.12 [13.60,14.65]	9.24 [8.85,9.63]	5.768 [5.526,6.007]	0.641 [0.625,0.659]	0.523 [0.515,0.531]
IntGrad (loc/faith)	N_{steps}=20,black baseline	0.550 [0.530,0.571]	0.770 [0.735,0.804]	0.706 [0.683,0.730]	15.39 [14.86,15.95]	11.89 [11.46,12.32]	6.438 [6.174,6.694]	1.132 [1.089,1.179]	0.917 [0.890,0.945]
DeepLIFT	Rescale	0.542 [0.523,0.562]	0.701 [0.664,0.738]	0.654 [0.631,0.677]	13.05 [12.59,13.52]	9.32 [8.96,9.69]	6.049 [5.806,6.291]	1.193 [1.145,1.249]	0.948 [0.919,0.985]
LRP-ε (loc)	ε=0.5	0.613 [0.595,0.632]	0.832 [0.800,0.862]	0.745 [0.725,0.764]	12.63 [12.18,13.10]	8.61 [8.26,8.96]	5.685 [5.458,5.908]	1.173 [1.127,1.223]	0.918 [0.893,0.947]
LRP-ε (faith)	ε=1	0.639 [0.621,0.657]	0.809 [0.777,0.841]	0.744 [0.725,0.762]	**12.43** [11.98,12.89]	8.17 [7.83,8.50]	5.489 [5.270,5.704]	1.100 [1.059,1.143]	0.840 [0.820,0.861]
LRP-γ (loc)	ε=0.5,γ=0.25, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.601 [0.584,0.617]	0.692 [0.655,0.729]	0.704 [0.687,0.721]	14.83 [14.28,15.39]	10.07 [9.68,10.46]	**5.010** [4.799,5.219]	0.515 [0.489,0.542]	0.368 [0.353,0.384]
LRP-γ (faith)	ε=0.25,γ=0.1	0.615 [0.598,0.633]	0.793 [0.759,0.827]	0.698 [0.681,0.716]	12.66 [12.18,13.13]	**7.89** [7.56,8.23]	5.078≈ [4.867,5.285]	0.702 [0.675,0.733]	0.561 [0.543,0.580]
GCAM	`features.29`	0.557 [0.538,0.576]	0.830 [0.798,0.860]	**0.821** [0.798,0.844]	16.36 [15.79,16.93]	12.47 [12.01,12.93]	5.640 [5.407,5.870]	0.430 [0.412,0.449]	0.334 [0.320,0.349]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
GCAM++	`features.29`	0.551 [0.533,0.569]	0.823 [0.791,0.855]	0.814≈ [0.792,0.835]	16.68 [16.09,17.27]	12.87 [12.40,13.35]	5.700 [5.462,5.938]	0.314 [0.301,0.327]	0.243 [0.233,0.253]
LCAM	`features.29`	0.544 [0.526,0.562]	0.720 [0.683,0.756]	0.747 [0.725,0.768]	15.98 [15.42,16.54]	11.48 [11.05,11.91]	5.341 [5.113,5.565]	0.434 [0.416,0.452]	0.330 [0.319,0.341]
LIME	N=1000,80 superpixels,ridge	0.603 [0.585,0.622]	**0.972** [0.958,0.984]	0.814≈ [0.790,0.837]	16.66 [16.09,17.25]	12.62 [12.17,13.06]	5.785 [5.551,6.015]	0.848 [0.816,0.882]	0.621 [0.599,0.644]
KernelSHAP	N=2M+2048,80 superpixels	0.559 [0.541,0.577]	0.954≈ [0.936,0.970]	0.820≈ [0.795,0.844]	16.68 [16.10,17.27]	12.82 [12.37,13.26]	6.140 [5.912,6.366]	0.994 [0.955,1.035]	0.736 [0.712,0.761]
SG (loc)	λ=−1,σ^2=5,gate on	0.597 [0.577,0.616]	0.770 [0.735,0.804]	0.770 [0.747,0.793]	17.28 [16.69,17.87]	13.32 [12.85,13.79]	6.217 [5.969,6.459]	**0.278** [0.267,0.290]	**0.196** [0.188,0.204]
SG (faith)	λ=−0.3,σ^2=1,gate off	0.598 [0.577,0.618]	0.821 [0.789,0.851]	0.788≈ [0.764,0.812]	17.08 [16.50,17.67]	12.98 [12.53,13.44]	6.278 [6.025,6.524]	0.543 [0.516,0.571]	0.423 [0.401,0.446]
RG anchor	τ=1	0.619 [0.601,0.637]	0.759 [0.724,0.795]	0.691 [0.673,0.709]	14.27 [13.74,14.80]	9.49 [9.10,9.89]	5.179 [4.968,5.388]	0.551 [0.526,0.578]	0.398 [0.385,0.412]
RG (loc)	τ=0.3,ε=0.1	**0.685** [0.665,0.705]	0.736 [0.699,0.772]	0.673 [0.654,0.692]	14.48 [13.95,15.02]	9.64 [9.25,10.03]	5.632 [5.411,5.845]	1.349 [1.273,1.430]	0.879 [0.842,0.918]
RG (faith)	τ=0.8,ε=0.1	0.640 [0.623,0.658]	0.712 [0.673,0.749]	0.688 [0.670,0.706]	14.05 [13.53,14.58]	9.07 [8.70,9.46]	5.117 [4.906,5.325]	0.655 [0.621,0.690]	0.466 [0.448,0.485]

Tab. S.4.15: ResNet-50, ImageNet-S test split: all rows with 95% bootstrap confidence intervals, as in Table S.4.13.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.481 [0.459,0.504]	0.556 [0.515,0.596]	0.568 [0.540,0.595]	6.33 [6.22,6.44]	5.91 [5.78,6.04]	5.027 [4.900,5.151]	1.448 [1.365,1.535]	1.072 [1.019,1.130]
SmGrad (loc)	N=50, σ/range=0.15	0.549 [0.530,0.569]	0.789 [0.756,0.821]	0.738 [0.719,0.755]	6.05 [5.93,6.17]	5.17 [5.02,5.31]	4.429 [4.303,4.554]	0.735 [0.718,0.753]	0.600 [0.593,0.609]
SmGrad (faith)	N=50,σ/range=0.1	0.544 [0.525,0.564]	0.773 [0.740,0.807]	0.735 [0.715,0.753]	6.06 [5.94,6.19]	5.19 [5.04,5.34]	4.489 [4.361,4.612]	0.726 [0.706,0.748]	0.586 [0.577,0.597]
IntGrad (loc)	N_{steps}=100,black baseline	0.510 [0.488,0.532]	0.635 [0.596,0.676]	0.600 [0.574,0.626]	6.27 [6.16,6.38]	5.83 [5.69,5.96]	4.949 [4.820,5.072]	1.203 [1.152,1.257]	0.947 [0.917,0.979]
IntGrad (faith)	N_{steps}=50,black baseline	0.507 [0.484,0.529]	0.604 [0.563,0.642]	0.595 [0.568,0.622]	6.28 [6.16,6.39]	5.82 [5.68,5.95]	4.952 [4.822,5.075]	1.207 [1.157,1.261]	0.951 [0.922,0.983]
DeepLIFT	Rescale	0.543 [0.523,0.564]	0.727 [0.690,0.765]	0.653 [0.629,0.677]	5.97 [5.85,6.10]	5.29 [5.15,5.43]	4.515 [4.390,4.637]	1.273 [1.203,1.351]	1.000 [0.955,1.052]
LRP-ε (loc)	ε=0.5	0.603 [0.584,0.622]	0.775 [0.740,0.809]	0.714 [0.694,0.734]	5.94 [5.81,6.06]	5.14 [4.99,5.28]	4.318 [4.200,4.433]	1.246 [1.203,1.292]	0.982 [0.960,1.006]
LRP-ε (faith)	ε=2	0.637 [0.618,0.656]	0.699 [0.662,0.736]	0.683 [0.663,0.702]	5.94 [5.81,6.06]	4.91 [4.76,5.05]	4.280 [4.164,4.394]	1.262 [1.224,1.301]	0.967 [0.948,0.986]
LRP-γ (loc)	ε=0.5,γ=2,$z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.711≈ [0.695,0.726]	0.842 [0.812,0.873]	0.822≈ [0.807,0.836]	5.69≈ [5.55,5.83]	4.53 [4.38,4.68]	3.935≈ [3.814,4.053]	0.633 [0.601,0.666]	0.486 [0.464,0.507]
LRP-γ (faith)	ε=0.5,γ=0.5, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.700 [0.684,0.716]	0.834 [0.802,0.864]	0.796 [0.781,0.812]	**5.65** [5.51,5.79]	**4.43** [4.28,4.59]	**3.931** [3.809,4.050]	0.745 [0.714,0.777]	0.584 [0.562,0.605]
GCAM	`layer4`	0.664 [0.645,0.683]	0.873 [0.844,0.899]	0.850≈ [0.830,0.870]	6.42 [6.32,6.52]	6.15 [6.03,6.26]	4.344 [4.216,4.469]	0.292 [0.274,0.312]	0.207 [0.195,0.220]
GCAM++	`layer4`	0.627 [0.607,0.646]	0.834 [0.804,0.864]	0.836 [0.815,0.857]	6.43 [6.34,6.52]	6.18 [6.07,6.29]	4.483 [4.356,4.609]	0.294 [0.278,0.310]	0.207 [0.197,0.219]
LCAM	`layer4`	0.649 [0.630,0.668]	0.869 [0.841,0.897]	**0.851** [0.831,0.871]	6.44 [6.34,6.53]	6.18 [6.07,6.29]	4.437 [4.311,4.559]	**0.274** [0.260,0.288]	**0.194** [0.184,0.204]
LIME	N=1000,80 superpixels,ridge	0.553 [0.533,0.572]	0.926≈ [0.903,0.947]	0.716 [0.687,0.744]	6.37 [6.26,6.46]	5.99 [5.88,6.11]	4.759 [4.630,4.881]	0.916 [0.891,0.941]	0.672 [0.655,0.690]
KernelSHAP	N=2M+2048,80 superpixels	0.558 [0.540,0.575]	**0.965** [0.949,0.979]	0.818≈ [0.793,0.841]	6.26 [6.16,6.36]	5.81 [5.69,5.92]	4.727 [4.601,4.849]	0.924 [0.896,0.954]	0.688 [0.670,0.708]
RG anchor	τ=1	0.693 [0.676,0.710]	0.752 [0.717,0.788]	0.756 [0.738,0.774]	5.84 [5.70,5.97]	4.71 [4.56,4.86]	4.017 [3.898,4.133]	0.708 [0.651,0.775]	0.508 [0.481,0.541]
RG (loc)	τ=0.9,ε=1	**0.719** [0.701,0.737]	0.763 [0.727,0.796]	0.760 [0.741,0.778]	5.85 [5.72,5.98]	4.70 [4.55,4.85]	4.047 [3.929,4.163]	0.798 [0.749,0.855]	0.560 [0.535,0.586]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
RG (faith)	τ=1,λ=−1,σ^2=1, ε=0.25	0.674 [0.656,0.691]	0.743 [0.708,0.779]	0.738 [0.719,0.756]	5.88 [5.74,6.01]	4.74 [4.59,4.89]	4.030 [3.911,4.144]	0.685 [0.629,0.755]	0.489 [0.460,0.525]

S.4.4.3 Trade-off sweeps

Table S.4.17 includes one-at-a-time sweeps of τ and λ_{sm} around the `loc`-selected ViT-B/16 RG configuration. These sweep rows are evaluated on the test split only after selection and never feed back into it.

S.4.4.4 Cross-Dataset Evaluation: Pascal VOC 2012

This part details the protocol behind the ViT-B/16 cross-dataset row of the summary table in Section 7.5 (all rows in Table S.4.16). The goal is twofold: (i) test whether the RG / RG+Eq configurations selected on ImageNet-S transfer to a different dataset *without* re-tuning, and (ii) put baselines on equal footing by re-running the same per-method validation selection on the new pool.

Dataset. We use Pascal VOC 2012 (Everingham et al., 2009) segmentation annotations: the trainval pool of 2,913 images with pixel-accurate `SegmentationClass` masks across 20 foreground classes plus background. Each image is expanded to one (image, foreground-class) sample per distinct VOC class label present in its mask, yielding 3,422 total samples after dropping the `person` class (no faithful ImageNet-1K target exists for it) and discarding sliver masks whose pixel area on the *original* (pre-resize) annotation is below 64 pixels. Each (image, class) sample carries the same input image and an explanation target equal to the most semantically central ImageNet-1K index for that VOC class (e.g. `horse` → N02389026 SORREL; `tvmonitor` → N03782006 MONITOR); the curated mapping is part of the released code.

Per-image disjoint val/test split. The trainval pool is partitioned randomly with seed 42 at the *image* level (not the (image, class) level) into a 1:5 val/test split: the trainval stems are shuffled deterministically and the first 1/6 is assigned to val. All (image, class) samples for a given image stay in the same partition, preventing the same image from appearing in both tuning and test sets. As on ImageNet-S, samples whose object mask lies entirely outside the 224×224 center crop are excluded after the split (18 val and 105 test samples). The resulting partitions:

	Images	(image, class) samples
val (tuning)	460	556
test (eval)	2,293	2,743

Pixel masks for localization. *Attribution Localization* (AL), *Pointing Game* (PG), and Top-k Intersection are computed against the per-(image, class) *binary VOC2012 segmentation pixel mask* for that class, not bounding boxes. The mask is resized via NEAREST interpolation and center-cropped to 224×224 to match the input pipeline (`Resize(256)` → `CenterCrop(224)`), then passed verbatim into Quantus (Hedström et al., 2023) alongside the attribution map. This avoids the well-known bbox over-credit issue (Selvaraju et al., 2017) where a method that puts mass anywhere inside the box wins the metric; pixel masks reward only mass on the actual foreground object.

Tuning protocol. The protocol mirrors the ImageNet-S protocol of Section A.2, with one deliberate asymmetry: RG and RG+Eq do *not* get a fresh grid search on VOC.

- **Baselines.** Every row of the per-method ViT-B/16 grid of Table S.4.9 is evaluated on the 556-sample val pool, and the `loc` and `faith` configurations are selected per method with the criteria of Section A.2.3. The selected rows are re-evaluated on the 2,743-sample test pool.
- **RG / RG+Eq (transfer test).** The anchor and the `loc`- and `faith`-selected ImageNet-S configurations are evaluated on the VOC test pool unchanged, without any VOC-side re-tuning.

To save compute, VOC is evaluated on localization and faithfulness only (no MaxS/AvgS and no runtime column). Baselines receive VOC-specific configuration selection, whereas the game rows remain fixed at their ImageNet-S settings. Significant wins, significant losses and comparisons without a detected difference are counted per metric with the tests of Section A.2.4; the last category does not establish equivalence.

Tab. S.4.16: ViT-B/16 on the Pascal VOC 2012 test pool (2,743 image–class records from 2,293 images): all rows with 95% confidence intervals from image-cluster bootstrap resampling. Means are weighted over records. RG / RG+Eq use the ImageNet-S-selected configurations without re-tuning; baselines are selected on the VOC val pool. Only localization and faithfulness metrics were evaluated; the unmeasured MaxS and AvgS columns are omitted. Metrics as in Table 7.4; configuration symbols and row labels: Section S.4.4.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
Gradient	fixed	0.295 [0.286,0.304]	0.360 [0.342,0.378]	0.355 [0.343,0.368]	3.45 [3.33,3.56]	3.17 [3.05,3.28]	2.316 [2.234,2.397]
SmGrad (loc)	N=50,σ/range=0.1	0.344 [0.334,0.354]	0.557 [0.538,0.576]	0.511 [0.498,0.523]	3.25 [3.13,3.36]	2.70 [2.60,2.81]	1.940 [1.871,2.011]
SmGrad (faith)	N=50, σ/range=0.15	0.347 [0.337,0.356]	0.555 [0.536,0.574]	0.511 [0.499,0.523]	3.24 [3.12,3.35]	2.67 [2.57,2.77]	1.907 [1.840,1.977]
IntGrad (loc)	N_{steps}=300,black baseline	0.292 [0.283,0.302]	0.388 [0.369,0.406]	0.358 [0.346,0.370]	3.43 [3.32,3.55]	3.21 [3.10,3.32]	2.278 [2.198,2.356]
IntGrad (faith)	N_{steps}=100,black baseline	0.292 [0.283,0.302]	0.389 [0.371,0.407]	0.358 [0.346,0.370]	3.44 [3.32,3.55]	3.21 [3.10,3.32]	2.279 [2.199,2.358]
LRP-ε (loc/faith)	ε=0.1	0.378 [0.368,0.388]	0.604 [0.585,0.623]	0.518 [0.506,0.531]	3.07 [2.96,3.18]	2.63 [2.54,2.73]	1.923 [1.856,1.990]
LRP-γ (loc/faith)	ε=0.1,γ=1	0.403 [0.393,0.414]	0.591 [0.572,0.610]	0.545 [0.532,0.558]	2.99 [2.88,3.10]	2.37 [2.27,2.46]	1.694≈ [1.634,1.756]
AttnLRP (loc/faith)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.05	0.404 [0.393,0.414]	0.602 [0.583,0.620]	0.542 [0.529,0.554]	**2.91** [2.80,3.01]	**2.31** [2.22,2.40]	**1.690** [1.630,1.751]
DAVE (loc)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.25	0.334 [0.324,0.343]	0.547 [0.527,0.566]	0.456 [0.444,0.468]	3.37 [3.26,3.49]	3.01 [2.91,3.12]	2.172 [2.097,2.247]
DAVE (faith)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.5	0.332 [0.322,0.342]	0.535 [0.515,0.554]	0.454 [0.443,0.466]	3.37 [3.26,3.48]	2.98 [2.88,3.09]	2.153 [2.079,2.227]
TIBAV	fixed	0.413 [0.402,0.424]	0.729 [0.713,0.747]	**0.575** [0.561,0.590]	3.04 [2.93,3.15]	2.67 [2.57,2.77]	1.731 [1.665,1.798]
GAE	fixed	0.355 [0.345,0.365]	0.712 [0.695,0.729]	0.548 [0.535,0.562]	3.00 [2.89,3.11]	2.63 [2.52,2.74]	1.754 [1.685,1.824]
AR (loc/faith)	w_r=0.75	0.272 [0.263,0.281]	0.471 [0.452,0.490]	0.411 [0.397,0.425]	3.57 [3.45,3.68]	3.41 [3.30,3.51]	2.146 [2.075,2.216]
LIME	N=1000,80 superpixels,ridge	0.351 [0.341,0.361]	0.785≈ [0.768,0.800]	0.523 [0.508,0.538]	3.45 [3.33,3.56]	3.21 [3.10,3.32]	2.121 [2.051,2.194]
KernelSHAP	N=2M+2048,80 superpixels	0.341 [0.332,0.351]	**0.793** [0.777,0.809]	0.561≈ [0.546,0.576]	3.36 [3.25,3.48]	3.03 [2.93,3.14]	2.077 [2.009,2.147]
RG anchor	τ=1	0.323 [0.313,0.333]	0.517 [0.498,0.536]	0.489 [0.475,0.502]	3.30 [3.19,3.41]	2.67 [2.58,2.77]	1.980 [1.914,2.047]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (loc)	τ=0.4,λ_{sm}=−10, σ^2=5,ε=0.1	**0.459** [0.447,0.472]	0.562 [0.543,0.582]	0.512 [0.498,0.527]	3.44 [3.33,3.56]	3.06 [2.96,3.17]	2.152 [2.083,2.222]
RG (faith)	τ=0.7,λ_{sm}=−5, σ^2=5,ε=0.1	0.351 [0.341,0.361]	0.538 [0.519,0.557]	0.508 [0.495,0.522]	3.27 [3.16,3.38]	2.63 [2.53,2.72]	1.965 [1.899,2.032]
RG+Eq anchor	τ=1,N=50	0.329 [0.319,0.339]	0.574 [0.554,0.593]	0.534 [0.519,0.549]	3.43 [3.31,3.54]	3.06 [2.96,3.17]	1.988 [1.922,2.056]
RG+Eq (loc)	τ=0.5,λ_{sm}=−10, σ^2=2,N=50	0.456 [0.443,0.468]	0.599 [0.579,0.619]	0.571≈ [0.555,0.586]	3.45 [3.34,3.57]	3.14 [3.03,3.25]	2.085 [2.016,2.154]
RG+Eq (faith)	τ=0.7,λ_{sm}=−10, σ^2=2,N=50	0.424 [0.412,0.436]	0.591 [0.572,0.610]	0.562≈ [0.547,0.578]	3.44 [3.32,3.55]	3.08 [2.98,3.19]	2.004 [1.938,2.071]

S.4.4.5 One-factor ablations

Tables S.4.17–S.4.19 vary one parameter at a time around each validation-selected configuration of RG, RG+Eq and SG (bold: the selected value), over the values of the declared grid; all other parameters stay at their selected values. The rows are evaluated on the 565-image test split with the localization and faithfulness metrics and never feed back into the selection. On ViT-B/16, "+Eq" evaluates RG+Eq at the parameters of the selected RG row. Configurations that are inadmissible on the validation split (Section A.2.3) are not included.

Tab. S.4.17: ViT-B/16, ImageNet-S test split: one-factor ablations around the selected configurations. Center: the selected configuration (criterion in parentheses) around which the parameter in the Factor column is varied over the Value column, all other parameters fixed; bold: the center's own value. Symbols: Section S.4.4.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.705	0.802	0.731	8.34	7.60	5.195
		0.4	0.688	0.827	0.773	8.26	7.41	5.095
		0.5	0.665	0.821	0.793	8.22	7.20	4.956
		0.6	0.613	0.814	0.774	8.13	6.88	4.820
		0.7	0.549	0.770	0.741	8.11	6.78	4.808
		0.8	0.507	0.742	0.712	8.14	6.85	4.881
		0.9	0.482	0.697	0.683	8.21	7.02	4.983
		1	0.467	0.648	0.648	8.29	7.25	5.115
RG (faith)	λ_{sm}	-50	0.467	0.556	0.541	8.51	8.13	5.928
		-10	0.556	0.777	0.763	8.16	6.97	4.876
		-5	0.549	0.770	0.741	8.11	6.78	4.808
		-2	0.547	0.743	0.720	8.11	6.81	4.793
		-1	0.546	0.733	0.713	8.11	6.83	4.794
		0	0.546	0.727	0.706	8.13	6.87	4.804
RG (faith)	σ^2	1	0.558	0.738	0.703	8.16	6.90	4.790
		2	0.555	0.743	0.719	8.13	6.81	4.783
		5	0.549	0.770	0.741	8.11	6.78	4.808
		10	0.545	0.765	0.749	8.12	6.82	4.833
RG (faith)	ε	**0.1**	0.549	0.770	0.741	8.11	6.78	4.808
		0.25	0.595	0.804	0.778	8.16	6.95	4.846
		0.5	0.621	0.809	0.792	8.24	7.21	4.944
		1	0.630	0.812	0.794	8.29	7.43	5.051

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	+Eq (N=50)	on	0.557	0.800	0.791	8.38	7.69	4.816
RG (loc)	τ	0.3	0.723	0.816	0.746	8.33	7.64	5.201
		0.4	0.711	0.825	0.788	8.27	7.48	5.109
		0.5	0.691	0.830	0.812	8.23	7.31	4.969
		0.6	0.632	0.827	0.801	8.16	7.00	4.847
		0.7	0.556	0.777	0.763	8.16	6.97	4.876
		0.8	0.506	0.743	0.715	8.22	7.13	5.013
		0.9	0.477	0.683	0.673	8.29	7.28	5.143
		1	0.461	0.628	0.629	8.36	7.42	5.247
RG (loc)	λ_{sm}	-50	0.520	0.543	0.528	8.52	7.99	5.745
		-10	0.711	0.825	0.788	8.27	7.48	5.109
		-5	0.688	0.827	0.773	8.26	7.41	5.095
		-2	0.676	0.827	0.764	8.26	7.39	5.086
		-1	0.673	0.818	0.761	8.26	7.38	5.080
		0	0.671	0.818	0.758	8.26	7.38	5.076
RG (loc)	σ^2	1	0.707	0.823	0.769	8.23	7.35	4.986
		2	0.712	0.823	0.780	8.24	7.39	5.032
		5	0.711	0.825	0.788	8.27	7.48	5.109
		10	0.705	0.837	0.787	8.31	7.58	5.168
RG (loc)	ε	**0.1**	0.711	0.825	0.788	8.27	7.48	5.109
		0.25	0.709	0.842	0.780	8.29	7.55	5.155
		0.5	0.708	0.828	0.776	8.30	7.57	5.171
		1	0.708	0.823	0.773	8.31	7.59	5.181
RG (loc)	+Eq (N=50)	on	0.711	0.846	0.842	8.38	7.82	5.077
RG+Eq (faith)	τ	0.3	0.721	0.805	0.786	8.37	7.75	5.129
		0.4	0.712	0.832	0.822	8.39	7.81	5.049
		0.5	0.698	0.855	0.836	8.39	7.80	5.010
		0.6	0.681	0.846	0.833	8.39	7.77	4.943
		0.7	0.654	0.812	0.820	8.39	7.71	4.856
		0.8	0.610	0.805	0.803	8.38	7.70	4.814
		0.9	0.567	0.796	0.787	8.41	7.71	4.825
		1	0.534	0.777	0.772	8.42	7.74	4.876
RG+Eq (faith)	λ_{sm}	**-10**	0.654	0.812	0.820	8.39	7.71	4.856
		0	0.613	0.814	0.815	8.37	7.73	4.914
RG+Eq (loc)	τ	0.3	0.721	0.805	0.786	8.37	7.75	5.129
		0.4	0.712	0.832	0.822	8.39	7.81	5.049
		0.5	0.698	0.855	0.836	8.39	7.80	5.010
		0.6	0.681	0.846	0.833	8.39	7.77	4.943
		0.7	0.654	0.812	0.820	8.39	7.71	4.856
		0.8	0.610	0.805	0.803	8.38	7.70	4.814
		0.9	0.567	0.796	0.787	8.41	7.71	4.825
		1	0.534	0.777	0.772	8.42	7.74	4.876
RG+Eq (loc)	λ_{sm}	**-10**	0.698	0.855	0.836	8.39	7.80	5.010
		0	0.652	0.848	0.828	8.40	7.85	5.073

Tab. S.4.18: VGG-16, ImageNet-S test split: one-factor ablations around the selected configurations, as in Table S.4.17.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.685	0.736	0.673	14.48	9.64	5.632
		0.4	0.645	0.657	0.641	14.40	9.36	5.574
		0.5	0.626	0.653	0.628	14.27	9.14	5.451
		0.6	0.633	0.651	0.646	14.07	9.00	5.258
		0.7	0.643	0.671	0.674	14.03	8.98	5.142
		0.8	0.640	0.712	0.688	14.05	9.07	5.117
		0.9	0.630	0.742	0.693	14.05	9.17	5.126
		1	0.618	0.749	0.695	14.06	9.25	5.137
RG (faith)	λ	-5	0.518	0.458	0.536	17.60	13.82	7.362
		-2	0.623	0.604	0.631	15.90	11.58	5.852
		-1	0.636	0.676	0.661	15.03	10.35	5.404
		-0.3	0.640	0.694	0.676	14.54	9.70	5.231
		0	0.640	0.712	0.688	14.05	9.07	5.117
RG (faith)	ε	**0.1**	0.640	0.712	0.688	14.05	9.07	5.117
		0.25	0.639	0.692	0.684	14.18	9.23	5.146
		0.5	0.638	0.690	0.680	14.34	9.46	5.183
		1	0.639	0.690	0.676	14.63	9.79	5.249
RG (loc)	τ	**0.3**	0.685	0.736	0.673	14.48	9.64	5.632
		0.4	0.645	0.657	0.641	14.40	9.36	5.574
		0.5	0.626	0.653	0.628	14.27	9.14	5.451
		0.6	0.633	0.651	0.646	14.07	9.00	5.258
		0.7	0.643	0.671	0.674	14.03	8.98	5.142
		0.8	0.640	0.712	0.688	14.05	9.07	5.117
		0.9	0.630	0.742	0.693	14.05	9.17	5.126
		1	0.618	0.749	0.695	14.06	9.25	5.137
RG (loc)	λ	-5	0.561	0.543	0.555	17.05	13.21	6.898
		-2	0.657	0.696	0.634	15.31	10.80	5.885
		-1	0.674	0.735	0.656	14.85	10.13	5.731
		-0.3	0.680	0.727	0.666	14.66	9.87	5.675
		0	0.685	0.736	0.673	14.48	9.64	5.632
RG (loc)	ε	**0.1**	0.685	0.736	0.673	14.48	9.64	5.632
		0.25	0.691	0.747	0.673	14.66	9.91	5.659
		0.5	0.697	0.742	0.672	14.87	10.19	5.705
		1	0.702	0.742	0.671	15.13	10.55	5.767
SG (faith)	λ	-2	0.614	0.710	0.636	18.17	14.83	6.941
		-1	0.609	0.750	0.697	17.91	13.87	6.517
		-0.3	0.598	0.821	0.788	17.08	12.98	6.278
		0	0.547	0.781	0.764	17.63	14.35	6.789
SG (faith)	σ^2	**1**	0.598	0.821	0.788	17.08	12.98	6.278
		2	0.601	0.811	0.765	17.30	13.07	6.323
		5	0.604	0.798	0.731	17.66	13.50	6.417
		10	0.607	0.763	0.706	17.86	13.77	6.500
SG (faith)	gate $\tilde{\Phi}$	**off**	0.598	0.821	0.788	17.08	12.98	6.278

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
		on	0.568	0.823	0.805	17.30	13.74	6.428
SG (loc)	λ	-5	0.568	0.550	0.558	18.17	14.31	7.509
		-2	0.620	0.703	0.681	17.84	13.63	6.404
		-1	0.597	0.770	0.770	17.28	13.32	6.217
		-0.3	0.566	0.830	0.809	17.33	13.75	6.452
		0	0.554	0.827	0.802	17.55	14.16	6.618
SG (loc)	σ^2	1	0.596	0.793	0.766	17.32	13.35	6.246
		2	0.597	0.786	0.766	17.31	13.34	6.232
		5	0.597	0.770	0.770	17.28	13.32	6.217
		10	0.595	0.772	0.776	17.25	13.30	6.206
SG (loc)	gate $\tilde{\Phi}$	off	0.614	0.696	0.628	18.19	15.00	7.050
		on	0.597	0.770	0.770	17.28	13.32	6.217

Tab. S.4.19: ResNet-50, ImageNet-S test split: one-factor ablations around the selected configurations, as in Table S.4.17.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.688	0.727	0.684	6.10	5.10	4.357
		0.4	0.676	0.712	0.692	6.08	5.03	4.340
		0.5	0.677	0.720	0.706	6.06	5.00	4.308
		0.6	0.673	0.704	0.711	5.99	4.96	4.270
		0.7	0.667	0.680	0.707	5.95	4.89	4.219
		0.8	0.665	0.688	0.711	5.89	4.85	4.134
		0.9	0.674	0.720	0.728	5.88	4.79	4.075
		1	0.674	0.743	0.738	5.88	4.74	4.030
RG (faith)	λ	-5	0.667	0.740	0.730	5.90	4.81	4.034
		-2	0.672	0.740	0.736	5.89	4.75	4.034
		-1	0.674	0.743	0.738	5.88	4.74	4.030
		-0.3	0.674	0.747	0.739	5.87	4.75	4.026
		0	0.675	0.747	0.740	5.86	4.74	4.023
RG (faith)	σ^2	**1**	0.674	0.743	0.738	5.88	4.74	4.030
		2	0.673	0.742	0.737	5.87	4.74	4.032
		5	0.671	0.736	0.736	5.88	4.75	4.031
		10	0.670	0.747	0.734	5.89	4.76	4.031
RG (faith)	ε	0.1	0.658	0.745	0.722	5.90	4.79	4.049
		0.25	0.674	0.743	0.738	5.88	4.74	4.030
		0.5	0.692	0.747	0.754	5.86	4.72	4.023
		1	0.712	0.766	0.761	5.85	4.72	4.038
RG (loc)	τ	0.3	0.733	0.761	0.719	6.03	5.04	4.272
		0.4	0.717	0.758	0.718	6.02	4.96	4.243
		0.5	0.709	0.740	0.720	5.98	4.92	4.216
		0.6	0.710	0.733	0.728	5.93	4.87	4.172
		0.7	0.716	0.745	0.741	5.91	4.80	4.125
		0.8	0.720	0.761	0.752	5.87	4.72	4.078
		0.9	0.719	0.763	0.760	5.85	4.70	4.047

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
		1	0.713	0.770	0.763	5.85	4.70	4.035
RG (loc)	λ	-5	0.708	0.765	0.745	5.89	4.80	4.069
		-2	0.716	0.773	0.755	5.87	4.74	4.055
		-1	0.718	0.777	0.757	5.86	4.72	4.054
		-0.3	0.719	0.768	0.758	5.86	4.72	4.050
		0	0.719	0.763	0.760	5.85	4.70	4.047
RG (loc)	ε	0.1	0.656	0.712	0.712	5.89	4.84	4.106
		0.25	0.674	0.717	0.730	5.87	4.77	4.067
		0.5	0.696	0.736	0.749	5.84	4.70	4.042
		1	0.719	0.763	0.760	5.85	4.70	4.047

S.4.4.6 Pointing game at matched granularity

The pointing game of Quantus counts a hit whenever any pixel attaining the map maximum lies in the mask. For piecewise-constant maps (superpixel methods; attention maps on the 14×14 patch grid) the maximum is a region of hundreds to thousands of pixels, for pixel-level maps usually a single pixel. Table S.4.20 reports, for the reported rows, the median size of the maximum set, the expected hit rate of a single maximum pixel drawn uniformly from that set (PG_{1px}), and the pointing game after average-pooling every map to the same 16×16-pixel grid (PG_{16px}). Drawing a single maximum pixel removes most of the advantage of the superpixel methods LIME and KernelSHAP. At the common 16×16 grid the leading methods move closer together: on ViT-B/16 LRP-γ and AttnLRP lead, on VGG-16 the localization-selected SG row and the faithfulness-selected RG row are not significantly below the best baseline (Grad-CAM++), and on ResNet-50 Grad-CAM, LayerCAM and LRP-γ lead (McNemar tests against the best baseline row).

Tab. S.4.20: Pointing game under the Quantus tie rule (PG, as in the main tables) and at matched granularity, ImageNet-S test split. max. set: median number of pixels attaining the map maximum; PG_{1px}: expected hit rate of one pixel drawn uniformly from that set; PG_{16px}: PG after average-pooling the map to 16×16-pixel cells. Rows: the `loc`-selected or fixed configuration of every method, plus the `faith` rows of our games. **Bold**: best per model and column.

Model	Method	PG	max. set (px)	PG_{1px}	PG_{16px}
ViT-B/16	Gradient	0.361	1	0.361	0.382
	SmGrad	0.706	1	0.706	0.835
	IntGrad	0.513	1	0.513	0.543
	LRP-ε	0.834	1	0.834	0.904
	LRP-γ	0.873	1	**0.873**	**0.968**
	AttnLRP	0.855	1	0.855	0.926
	DAVE	0.717	1	0.717	0.781
	TIBAV	0.878	256	0.784	0.878
	GAE	0.743	256	0.670	0.743
	AR	0.565	256	0.516	0.565
	LIME	0.897	1104	0.693	0.786
	KernelSHAP	**0.961**	1031	0.833	0.903
	RG (faith)	0.770	1	0.770	0.899
	RG	0.825	1	0.825	0.888
	RG+Eq (faith)	0.812	1	0.812	0.919
	RG+Eq	0.855	1	0.855	0.919
VGG-16	Gradient	0.738	1	0.738	0.812
	SmGrad	0.782	1	0.782	0.915
	IntGrad	0.770	1	0.770	0.860
	DeepLIFT	0.701	1	0.701	0.784
	LRP-ε	0.832	1	**0.832**	0.906
	LRP-γ	0.692	1	0.692	0.908
	GCAM	0.830	1	0.823	0.920
	GCAM++	0.823	1	0.815	**0.924**
	LCAM	0.720	1	0.715	0.834
	LIME	**0.972**	1103	0.826	0.899
	KernelSHAP	0.954	1011	0.826	0.894
	SG (faith)	0.821	1	0.821	0.883
	SG	0.770	1	0.770	0.908
	RG (faith)	0.712	1	0.712	0.915
	RG	0.736	1	0.736	0.873
ResNet-50	Gradient	0.556	1	0.556	0.630
	SmGrad	0.789	1	0.789	0.919
	IntGrad	0.635	1	0.635	0.743
	DeepLIFT	0.727	1	0.727	0.793
	LRP-ε	0.775	1	0.775	0.880
	LRP-γ	0.842	1	0.842	0.958
	GCAM	0.873	1	**0.862**	**0.959**
	GCAM++	0.834	1	0.827	0.945
	LCAM	0.869	1	0.860	0.958
	LIME	0.926	1058	0.729	0.844
	KernelSHAP	**0.965**	1014	0.825	0.912
	RG (faith)	0.743	1	0.743	0.903
	RG	0.763	1	0.763	0.912

S.4.4.7 Gradient-fallback attention

The RG backward operates by default through a routing rule that holds the attention policy fixed and routes relevance only through the Value path, with zero gradient on Q, K (Value-only routing conditioned on the realized attention matrix, as in CP-LRP (Ali et al., 2022); see Section 5.5.3). The natural alternative is a gradient fallback: pure autograd through the attention block, recovering the zennit-style ε-LRP attention treatment. Table S.4.21 swaps the attention rule at the $\alpha\beta$-LRP anchor and at the two validation-selected RG configurations and evaluates both rules on the 565-image test split. The softmax risk λ_{sm} acts on the Value-path routing and is therefore inactive under the gradient fallback. The gradient fallback lowers all three localization metrics at every configuration, AL and TK significantly throughout, and it does not improve faithfulness at any configuration: PF_5, PF_{20} and Sel are worse in every cell, significantly so in four of the nine.

Tab. S.4.21: ViT-B/16, RG: attention rule ablation on the ImageNet-S test split (565 images), at the anchor and at the loc- and faith-selected configurations. Each cell shows the gradient fallback on top and the composite Value rule of Theorem S.2.3 (gray) below; configuration symbols: Section S.4.4; †: paired difference significant (paired bootstrap / McNemar for PG, Holm-corrected over the table, $p < 0.05$).

Center	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
anchor (τ=1)	0.504†	0.722	0.694†	8.19	6.98†	4.877†
	0.509	0.724	0.699	8.19	6.97	4.854
loc (τ=0.4,λ_{sm}=−10,σ^2=5,ε=0.1)	0.624†	0.743†	0.672†	8.31	7.50	5.364†
	0.711	0.825	0.788	8.27	7.48	5.109
faith (τ=0.7,λ_{sm}=−5,σ^2=5,ε=0.1)	0.543†	0.729†	0.706†	8.13	6.87†	4.816
	0.549	0.770	0.741	8.11	6.78	4.808

S.4.4.8 Qualitative comparison

Figure S.4.2 compares attribution maps for six ImageNet-S examples on ViT-B/16. Each column shows a method at its `loc`-selected configuration (Section A.2.3), or at its fixed configuration if the method has no tunable parameters (Gradient, TIBAV): RG with τ=0.4,λ_{sm}=−10,σ^2=5,ε=0.1 and RG+Eq with τ=0.5,λ_{sm}=−10,σ^2=2,N=50, both with the merged attention–Value rule, i.e. the composite rule of Theorem S.2.3, which routes through the conditioned attention block as one map $X \mapsto O$; SmoothGrad with N=50,σ/range=0.2; Integrated Gradients with N_{steps}=100 and a black baseline; DAVE with N=50,$\theta_{\max}$=0,$s_{\max}$=0,$t_{\max}$=0.5; LRP-γ with ε=0.1,γ=2; and AttnLRP with γ_{conv}=0.25,γ_{lin}=0.05,$\varepsilon_{\text{attn}}$=0.25. As in the evaluation, every map explains the ImageNet-S label, which is the top-1 prediction for all images except the kuvasz (predicted: English setter). The first image is from the test split, the other five are from the 50-image validation split used for the selection. Heatmaps show the raw evaluator-side maps, before the absolute value and max-abs normalization of Section A.2.2 (the 14×14 TIBAV map is upsampled by nearest neighbor, as for the metrics): red marks positive and blue negative attribution; each sign is divided by its 99th percentile over the image and clipped, so the top 1% of pixels saturate and concentrated maps stay legible in print; the same display transform is used for every method.

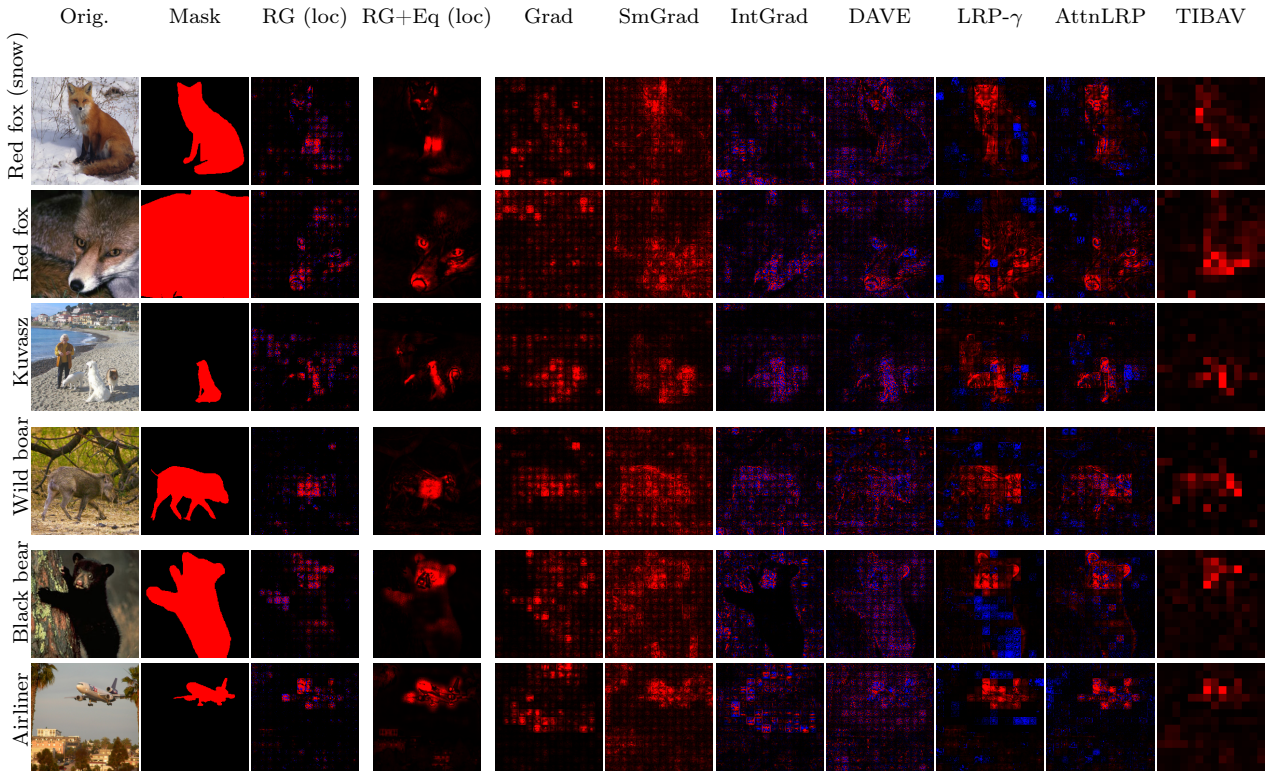


Fig. S.4.2: Dense qualitative comparison on six ImageNet-S examples (ViT-B/16). Columns: Original; ImageNet-S pixel Mask; RG and RG+Eq (merged attention–Value rule); Gradient, SmoothGrad, Integrated Gradients, DAVE, LRP-γ, AttnLRP and TIBAV. Every method is shown at its `loc`-selected configuration, Gradient and TIBAV at their fixed configurations (values in the text). Red: positive, blue: negative attribution, each sign divided by its 99th percentile over the image and clipped.

S.4.5 Numerical Diagnostic of the Transformer DC Construction

The deterministic script `scripts/transformer_dc_demo.py` exercises the construction of Section 5.3 on a three-token, width-three Transformer with one causal attention head, two post-residual LayerNorms, a width-four exact Gaussian-GELU MLP, mean pooling, and two signed output logits. An independent ordinary PyTorch computation uses the same parameters. The diagnostic runs on CPU in float64, without recentering, clipping, or substitution of reference activations into the split computation.

With random seed 23, the sweep combines $\varepsilon \in \{1, 10^{-2}, 10^{-5}\}$, weight scales $\{0.01, 0.1, 0.5\}$, and input scales $\{0.1, 1\}$. Two additional cases add common input-stream offsets of 1 and 10 to the $\varepsilon = 1$, weight-scale-0.01, input-scale-0.1 case. All twenty cases, including failures, are retained in `reports/evidence/transformer-dc/numeric.json`, together with the environment, source hash, per-stage reconstruction error, and stream magnitudes. The command is:

```
python scripts/transformer_dc_demo.py \
    --output RESULT.json
```

The benign case reconstructs the logits within 2.78×10^{-17}, but fifteen of the twenty cases exceed a final maximum absolute error of 10^{-8}. Every tested $\varepsilon = 10^{-5}$ case loses output accuracy, with

stream magnitudes reaching 5.95×10^{111}. Separate primitive checks and zero-variance LayerNorm cases pass. These finite checks diagnose the implementation; they do not prove convexity or numerical stability. The result demonstrates that the exact real-arithmetic construction can suffer catastrophic cancellation even in a small Transformer, so a stable practical implementation remains open.

S.4.6 Formal Verification Details

Both papers use the same finite, varying-width MLP parameter definition in Lean. A network is either an identity output or an affine layer followed by a smaller network; its evaluators specify the ordinary activation or a pair activation. Local preservation of the difference is the only algebraic requirement for a valid pair activation, and induction proves recovery of the complete network. The same definition supports the ReLU, mixing, and Softplus developments. A separate affine readout accounts for an ungated target logit.

The Transformer development defines signed projections, actual masked Query–Key products, coupled softmax, attention–Value products, input-dependent LayerNorm, residual additions, and the feedforward subnetwork. A finite list of blocks represents arbitrary depth, including zero blocks, with an optional final LayerNorm and a signed affine readout. Compilation to an explicit expression language is proved equal to the ordinary forward evaluator. A recursive two-stream evaluator then establishes Theorem 5.5: exact reconstruction and convex, monotone, non-negative components on non-negative doubled inputs. The positive/negative raw-input lift gives globally convex components; raw-input monotonicity is not asserted. These proofs use the actual closed-form softmax and normalization constructions, with positive normalization ε and nonempty allowed-key sets, rather than assuming their structural properties. They concern exact real arithmetic; the numerical diagnostic in Section S.4.5 records the limits of direct evaluation.

A forward trace records the actual gate decisions. A sensitivity row is a linear function taking an input perturbation to the corresponding scalar output change; a pair of rows gives the partial sensitivities with respect to the two input streams. Their half-difference is the derivative along the balanced lift $(x/2, -x/2)$. At strict gate comparisons, the recursively constructed Jacobian is proved to be the actual Fréchet derivative of the complete network.

The shifted backward algorithm is defined layer by layer and proved equal to its closed formula for arbitrary widths and shift parameters. Its source terms are actual derivatives of explicitly defined Q-networks, with the original prefix, one minimum, and a linear absolute-weight tail without biases. The suffix products of shift coefficients, sensitivity-mass law, separate cross-head partial identities, and original-gradient reconstruction are checked. Shifted algorithmic sensitivities are distinguished from derivatives through input-dependent forward re-centering; classical derivatives at gate ties are not asserted.

The game developments define finite sign-labeled paths, their probabilities, discounts, and occupation sums. For the stated policies, the discounted stopping occupations recover the actual MLP gradient; routing occupations recover the complete $\alpha\beta$ recurrence from network-derived contributions. The smooth case includes the actual full Softplus derivative. The attention result treats a fixed attention matrix and the complete composite Value projection, including its bias. It does not assert Query/Key differentiation or a global equilibrium merely from the backward recovery identity. Generic finite layered-game saddle bounds and Bellman uniqueness are checked separately with actual randomized policies.

The trajectory module checks finite path normalization, dynamic-programming identities, geometric-mean overlap, per-terminal-state disagreement, and positive survival conditioning. Ordinary probability laws used for Hellinger comparison are kept distinct from discounted occupation measures. The regularization module checks the finite Gibbs variational maximum, its attaining policy, the binary stop/continue optimizer, and actual Softplus derivatives.

The audit exposed a missing factor 1/2 in the four-partial backward invariant, a shift-product index error, an omitted weight matrix, and overly broad claims about differentiability, multiplicative convexity, re-centering, and nonzero combined gradients. The corrections and counterexamples are documented in `reports/mathematical-corrections.md`. In particular, nonzero raw sensitivity mass does not force a nonzero balanced head gradient.

The source inventory records mathematical statements still outside the checked scope. Neither these proofs nor the corrections certify the Python implementations or empirical experiments. The table adapter independently regenerates the thirteen outputs supported by the revised game's published summaries and checks them against the thesis. Diagnostic and hidden-monotonicity tables whose raw summaries are unavailable retain their published source provenance. To reproduce the mathematical checks, build `formalization/` with `lake build` and run its explicit axiom audit; the README documents the complete commands.

List of References

Achanta, R., A. Shaji, K. Smith, A. Lucchi, P. Fua, S. Süsstrunk (2012): *SLIC superpixels compared to state-of-the-art superpixel methods. IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(11):2274–2282, 2012 (cit. on p. 59).

Achtibat, R., S. M. Vakilzadeh Hatefi, M. Dreyer, A. Jain, T. Wiegand, S. Lapuschkin, W. Samek (2024): AttnLRP: Attention-Aware Layer-Wise Relevance Propagation for Transformers. *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 135–168 (cit. on pp. 58, 60).

Adebayo, J., J. Gilmer, M. Muelly, I. J. Goodfellow, M. Hardt, B. Kim (2018): Sanity Checks for Saliency Maps. *NeurIPS*, 9525–9536 (cit. on p. 51).

Ali, A., T. Schnake, O. Eberle, G. Montavon, K.-R. Müller, L. Wolf (2022): XAI for Transformers: Better Explanations through Conservative Propagation. *Proceedings of the 39th International Conference on Machine Learning (ICML)*. Vol. 162. Proceedings of Machine Learning Research. PMLR, 435–451 (cit. on p. 75).

Anders, C. J., D. Neumann, W. Samek, K.-R. Müller, S. Lapuschkin (2023): *Software for Dataset-wide XAI: From Local Explanations to Global Insights with Zennit, CoRelAy, and ViRelAy*. arXiv: 2106.13200 [cs.LG]. URL: https://arxiv.org/abs/2106.13200 (cit. on p. 21).

Auer, P., N. Cesa-Bianchi, P. Fischer (2002): Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47 (2-3), 235–256. ISSN: 1573-0565. DOI: 10.1023/a:1013689704352. URL: http://dx.doi.org/10.1023/A:1013689704352 (cit. on p. 25).

Bach, S., A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, W. Samek (2015): On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation. *PLOS ONE* 10 (7), 1–46. DOI: 10.1371/journal.pone.0130140. URL: https://doi.org/10.1371/journal.pone.0130140 (cit. on pp. 20, 21, 58).

Binder, A., L. Weber, S. Lapuschkin, G. Montavon, K.-R. Müller, W. Samek (2023): Shortcomings of Top-Down Randomization-Based Sanity Checks for Evaluations of Deep Neural Network Explanations. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 16143–16152. DOI: 10.1109/cvpr52729.2023.01549. URL: http://dx.doi.org/10.1109/CVPR52729.2023.01549 (cit. on p. 51).

Boyd, S., L. Vandenberghe (2004): *Convex Optimization*. Cambridge University Press. ISBN: 9780511804441. DOI: 10.1017/cbo9780511804441. URL: http://dx.doi.org/10.1017/CBO9780511804441 (cit. on pp. 14, 27, 32).

Chattopadhay, A., A. Sarkar, P. Howlader, V. N. Balasubramanian (2018): Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks. *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 839–847. DOI: 10.1109/WACV.2018.00097 (cit. on p. 59).

Chefer, H., S. Gur, L. Wolf (2021a): Generic Attention-Model Explainability for Interpreting Bi-Modal and Encoder-Decoder Transformers. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 397–406 (cit. on p. 58).

Chefer, H., S. Gur, L. Wolf (2021b): Transformer Interpretability Beyond Attention Visualization. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 782–791 (cit. on p. 58).

Deng, J., W. Dong, R. Socher,
bibinitperiod Li, K. Li, L. Fei-Fei (2009): ImageNet: A Large-Scale Hierarchical Image Database. English (US). *2009 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2009*. 2009 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2009. Publisher Copyright: © 2009 IEEE.; 2009 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2009 ; Conference date: 20-06-2009 Through 25-06-2009. United States: IEEE Computer Society, 248–255. ISBN: 9781424439911. DOI: 10.1109/CVPR.2009.5206848 (cit. on p. 58).

Dombrowski, A.-K., C. J. Anders, K.-R. Müller, P. Kessel (2022): Towards robust explanations for deep neural networks. *Pattern Recognition* 121, 108194. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2021.108194. URL: http://dx.doi.org/10.1016/j.patcog.2021.108194 (cit. on p. 29).

Everingham, M., L. Van Gool, C. K. I. Williams, J. Winn, A. Zisserman (2009): The Pascal Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision* 88 (2), 303–338. ISSN: 1573-1405. DOI: 10.1007/s11263-009-0275-4. URL: http://dx.doi.org/10.1007/s11263-009-0275-4 (cit. on pp. 58, 67).

Fei, Y., Z. Yang, Y. Chen, Z. Wang, Q. Xie (2020): Risk-Sensitive Reinforcement Learning: Near-Optimal Risk-Sample Tradeoff in Regret. *NeurIPS* (cit. on p. 25).

Gao, S., Z.-Y. Li, M.-H. Yang, M.-M. Cheng, J. Han, P. Torr (2023): Large-Scale Unsupervised Semantic Segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45 (6), 7457–7476. ISSN: 1939-3539. DOI: 10.1109/tpami.2022.3218275. URL: http://dx.doi.org/10.1109/TPAMI.2022.3218275 (cit. on p. 58).

Gast, J., S. Roth (2018): Lightweight Probabilistic Deep Networks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 3369–3378. DOI: 10.1109/cvpr.2018.00355. URL: http://dx.doi.org/10.1109/cvpr.2018.00355 (cit. on p. 25).

Gaubert, S., Y. Vlassopoulos (2025): *Relu and softplus neural nets as zero-sum turn-based games*. arXiv: 2512.20582 [cs.LG]. URL: https://arxiv.org/abs/2512.20582 (cit. on pp. 5, 26).

Hedström, A., L. Weber, D. Krakowczyk, D. Bareeva, F. Motzkus, W. Samek, S. Lapuschkin, M. M.-C. Müller (2023): Quantus: An Explainable AI Toolkit for Responsible Evaluation of Neural Network Explanations and Beyond. *Journal of Machine Learning Research* 24 (34), 1–11. URL: http://jmlr.org/papers/v24/22-0142.html (cit. on pp. 58, 67).

Hendrycks, D., K. Gimpel (2016): Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*. URL: https://arxiv.org/abs/1606.08415 (cit. on p. 26).

Jiang, P.-T., C.-B. Zhang, Q. Hou, M.-M. Cheng, Y. Wei (2021): LayerCAM: Exploring Hierarchical Class Activation Maps for Localization. *IEEE Transactions on Image Processing* 30, 5875–5888. DOI: 10.1109/TIP.2021.3089943 (cit. on p. 59).

Kohlbrenner, M., A. Bauer, S. Nakajima, A. Binder, W. Samek, S. Lapuschkin (2020): Towards Best Practice in Explaining Neural Network Decisions with LRP. *IJCNN*. IEEE, 1–7 (cit. on p. 58).

Lundberg, S. M., S.-I. Lee (2017): A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett. Vol. 30. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf (cit. on p. 59).

Mukherjee, A., A. Basu, R. Arora, P. Mianjy (2018): Understanding Deep Neural Networks with Rectified Linear Units. English. International Conference on Learning Representations , ICLR ; Conference date: 30-04-2018 Through 03-05-2018. URL: https://iclr.cc/Conferences/2018 (cit. on p. 3).

Ribeiro, M. T., S. Singh, C. Guestrin (2016): "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. San Francisco, California, USA: Association for Computing Machinery, 1135–1144. ISBN: 9781450342322. DOI: 10.1145/2939672.2939778. URL: https://doi.org/10.1145/2939672.2939778 (cit. on p. 59).

Russakovsky, O., J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, L. Fei-Fei (2015): ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision* 115 (3), 211–252. ISSN: 1573-1405. DOI: 10.1007/s11263-015-0816-y. URL: http://dx.doi.org/10.1007/s11263-015-0816-y (cit. on p. 58).

Selvaraju, R. R., M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra (2017): Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *2017 IEEE International Conference on Computer Vision (ICCV)*, 618–626. DOI: 10.1109/ICCV.2017.74 (cit. on pp. 59, 67).

Shrikumar, A., P. Greenside, A. Kundaje (2017): Learning Important Features Through Propagating Activation Differences. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 3145–3153. URL: https://proceedings.mlr.press/v70/shrikumar17a.html (cit. on p. 58).

Simonyan, K., A. Vedaldi, A. Zisserman (2014): Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Workshop Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1312.6034 (cit. on p. 58).

Smilkov, D., N. Thorat, B. Kim, F. Viégas, M. Wattenberg (2017): *SmoothGrad: removing noise by adding noise*. arXiv: 1706.03825 [cs.LG]. URL: https://arxiv.org/abs/1706.03825 (cit. on p. 58).

Sundararajan, M., A. Taly, Q. Yan (2017): Axiomatic Attribution for Deep Networks. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 3319–3328. URL: https://proceedings.mlr.press/v70/sundararajan17a.html (cit. on p. 58).

Wróbel, A., S. Gairola, J. Tabor, B. Schiele, B. Zieliński, D. Rymarczyk (2026): *DAVE: Distribution-aware Attribution via ViT Gradient Decomposition*. arXiv: 2602.06613 [cs.CV]. URL: https://arxiv.org/abs/2602.06613 (cit. on p. 58).

Zimmermann, J. P., G. Loho (2026): Hidden Monotonicity: Explaining Deep Neural Networks via their DC Decomposition. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Highlight, 24107–24117. arXiv: 2601.07700 [cs.CV]. URL: https://openaccess.thecvf.com/content/CVPR2026/html/Zimmermann_Hidden_Monotonicity_Explaining_Deep_Neural_Networks_via_their_DC_Decomposition_CVPR_2026_paper.html (cit. on p. 3).